

A Multi-Start Path-Cover Matheuristic for Mixed-Fleet Bus Scheduling in MINOA

MIRZA SHAHEEN IQUBAL

Matriculation number: 22998316

Academic paper to obtain the degree of
Master of Science (M.Sc.)

in the study program
Data Science

First Supervisor

Prof. Dr. Frauke Liers
Professur für Optimization under
Uncertainty & Data Analysis

Second Supervisor

Prof. Dr. Michael Stingl
Chair of Applied Mathematics
(Continuous Optimization)

at the Faculty of Sciences of
Friedrich-Alexander-Universität Erlangen-Nürnberg

21 July 2026

Declaration of Authorship

I confirm that I have written this thesis independently and without unauthorized assistance from third parties and without the use of undisclosed or unauthorized aids. This thesis has not been submitted in the same or a similar form to another examination authority and has not been accepted as part of an examination by another examination authority. All content taken from other sources, either verbatim or in substance, has been acknowledged and referenced. This also applies to drawings, sketches, diagrams, visual material, and Internet sources.

I confirm that I have taken note of the regulation on the use of artificial intelligence in theses and will comply with it. AI-based tools were used solely as supporting tools for partial editing and polishing of text and figures with respect to spelling, grammar, wording, formatting, and stylistic improvement.

All suggestions from these tools were reviewed and revised by me before inclusion. The research question, methodology, implementation, experiments, validation, numerical results, figures, tables, interpretation, and conclusions remain my own responsibility.

Erlangen, 21 July 2026

Mirza Shaheen Iqbal

Abstract

Urban bus operators increasingly have to plan conventional and electric buses in one common system. This creates an integrated planning problem: candidate trips must be selected, trips must be chained into vehicle blocks, vehicle types must be assigned, and electric buses must be charged without violating terminal capacities. This thesis studies this problem for the MINOA Senior benchmark. The goal is to produce high-quality feasible schedules for mixed ICE/EV fleets under headway rules, vehicle-block compatibility, battery autonomy, charging requirements, parking and charger capacities, and the official vehicle-scheduling cost structure.

I develop a multi-start path-cover matheuristic for this setting. For each timetable variant, the method builds a compatibility graph in which selected trips are nodes and feasible vehicle continuations are directed edges. Vehicle blocking is then modeled as a path-cover problem. The final method combines graph matching, charging-aware edge scoring, EV/ICE block assignment, battery simulation, charging insertion, and a no-regression archive. The matching step is exact for a fixed graph, while the complete method remains heuristic because timetable generation, edge scoring, fleet assignment, and charging decisions are explored through bounded candidate variants.

Four algorithmic variants are implemented and compared: a greedy constructive heuristic, an unweighted path-cover method, a weighted path-cover method, and the final multi-start weighted path-cover matheuristic. The implementation computes the official objective internally and writes MINOA-compatible output files. The experiments were run in a CPU-only local environment without GPU or HPC resources, which is encouraging from a reproducibility perspective. Final outputs are checked by an independent feasibility and objective audit. The best validated headline results produced by the implemented method are 162.44 for Small, 371.35 for Medium, and 1163.35 for Large, using 2, 5, and 15 vehicles. For all Senior instances, the final archive reports total validated cost 10000.48 and 126 vehicles.

Keywords: integrated timetabling, vehicle scheduling, mixed bus fleets, electric buses, graph matching, path cover, charging coordination, no-regression archive, MINOA Senior benchmark.

Contents

Declaration of Authorship	I
Abstract	II
Contents	III
List of Figures	VII
List of Tables	VIII
List of Abbreviations	IX
1 Introduction	1
1.1 Background and Motivation	1
1.2 Public Transport Planning Context	1
1.3 MINOA Challenge Setting	2
1.4 Research Objectives and Questions	3
1.5 Scope and Delimitations	3
1.6 Research Contributions	4
1.7 Structure of the Thesis	5
2 Literature Review	6
2.1 Timetabling in Public Transport	6
2.2 Vehicle Scheduling	7
2.3 Integrated Timetabling and Vehicle Scheduling	9
2.4 Electric Bus Scheduling and Charging	10
2.5 Mixed-Fleet Scheduling	11
2.6 Graph Algorithms and Positioning	12
3 Problem Definition and Benchmark	14
3.1 Study Scope and Input Assumptions	14
3.2 Network and Service Representation	15
3.2.1 Lines, Directions, Trips, and Terminals	15
3.2.2 Depot, Nodes, and Deadhead Movements	16
3.2.3 Planning Horizon and Time Windows	16
3.3 Timetable Selection Layer	17
3.3.1 Candidate Trips and Selected Timetable	17
3.3.2 Headway Feasibility	17
3.3.3 Initial and Final Trip Requirements	19
3.4 Vehicle-Scheduling Layer	19
3.4.1 Vehicle Blocks and Trip Coverage	19
3.4.2 In-Line and Depot-Bridge Compatibility	20

3.4.3	Break, Waiting, and Stopping-Time Requirements	21
3.5	Electric-Bus and Charging Layer	21
3.5.1	Battery Autonomy	21
3.5.2	Slow and Fast Charging	22
3.5.3	Minimum Charging Time and Partial Recharging	23
3.5.4	Parking and Charging Capacity at Nodes	23
3.6	Objective and Cost Structure	24
3.7	Benchmark Instances and Unit Conventions	24
4	Mathematical Formulation and Graph Model	28
4.1	Notation and Planning Objects	28
4.1.1	Sets, Parameters, and Indices	28
4.1.2	Layered Decision Structure	30
4.2	Compatibility-Graph Formulation	30
4.2.1	Trip Nodes and Time-Ordered Candidate Pairs	30
4.2.2	In-Line Continuation Edges	30
4.2.3	Depot-Bridge Continuation Edges	31
4.2.4	Arc Set and Edge Attributes	32
4.3	Decision Variables and State Variables	33
4.3.1	Timetable Selection Variables	33
4.3.2	Coverage and Chain Variables	33
4.3.3	Vehicle-Type Assignment Variables	34
4.3.4	Battery, Charging, and Break Variables	34
4.4	Objective Function	35
4.4.1	Reported Vehicle-Scheduling Cost	35
4.4.2	Objective Decomposition	36
4.4.3	Internal Edge Scores	36
4.5	Core Feasibility Constraints	37
4.5.1	Timetable Feasibility Constraints	37
4.5.2	Timetable–Vehicle Schedule Linking	38
4.5.3	Matching Constraints for Path Cover	38
4.5.4	Path-Cover Theorem for the Fixed Compatibility Graph	40
4.5.5	Vehicle-First Weighted Matching	40
4.6	Electric Fleet, Charging, and Capacity Constraints	41
4.6.1	Electric Fleet Availability	41
4.6.2	Battery Propagation	42
4.6.3	Charging Feasibility	42
4.6.4	Parking and Charging Capacity	43

4.7	Paid-Break Accounting and Linearization	45
4.7.1	Direct Paid-Break Calculation	45
4.7.2	Big-M View of the Same Logic	46
4.8	Timetable Variant Scoring	47
4.8.1	Sparse Headway Paths	47
4.8.2	Examples of Variant Scores	47
4.9	Model Size and Complexity Discussion	48
4.9.1	Graph Size	48
4.9.2	Matching Problem Size	48
4.9.3	Why the Matheuristic is Practical	49
4.10	Exact Subproblems and Heuristic Coupling	49
4.10.1	Fixed-Graph Path-Cover Layer	49
4.10.2	Integrated Candidate Layer	50
4.10.3	Bound Interpretation	51
5	Solution Methodology	53
5.1	Shared Construction Layer	53
5.2	Vehicle Blocks as a Path Cover	55
5.3	Four Implemented Algorithms	57
5.3.1	Greedy Constructive Heuristic	58
5.3.2	Unweighted Path Cover	61
5.3.3	Weighted Path Cover	63
5.3.4	Multi-Start Weighted Path-Cover Matheuristic	65
5.3.5	Rationale for Selecting the Multi-Start Path-Cover Matheuristic	68
5.4	EV Assignment and Charging Feasibility	69
5.5	Comparison of Methodological Roles	71
6	Software Implementation and Experimental Design	72
6.1	Software Architecture	72
6.2	Internal Objective and External Audit	75
6.3	Runtime and Environment	76
6.4	Experiment Commands and Scopes	77
6.5	Reproducibility Controls	78
7	Computational Results	79
7.1	Evaluation Metrics	79
7.2	Comparison of the Four Algorithms	80
7.3	Headline Instances	84
7.4	Complete Senior Benchmark	85
7.5	Archive Improvements and Ablation Evidence	91

7.6	Lower Bounds, Runtime, and Objective Audit	92
8	Discussion	100
8.1	Interpretation of the Main Findings	100
8.2	Benefits and Drawbacks of the Implemented Methods	102
8.3	Validation, Bounds, and Optimality	103
8.4	Limitations and Future Work	104
9	Conclusion	107
9.1	Summary of the Thesis	107
9.2	Main Methodological Contribution	107
9.3	Synthesis of the Main Findings	108
9.4	Scientific and Practical Implications	108
9.5	Final Remarks	109
	References	110
A	Running the Code	113
A.1	System and Software Requirements	113
A.2	Repository Structure	113
A.3	Main Experiment Commands	114
A.4	Output Files and External Audit	114
A.5	Tables, Figures, and Lower Bounds	114
A.6	Quick-Start Recipe	115
A.7	Troubleshooting	115
B	Complete Algorithm Comparison on All Senior Instances	116
C	Selected Instance Profiles	118
D	Core Pseudocode	119
D.1	Timetable Selection	119
D.2	Compatibility Graph and Path Cover	119
D.3	Final Multi-Start Matheuristic	120
D.4	EV Feasibility and Charging Assignment	120
E	Supporting Mathematical Derivations	121
E.1	Why Matching Gives a Minimum Path Cover	121
E.2	Weighted Matching as a Vehicle-First Score	121
E.3	Battery and Charging Propagation	122
E.4	Capacity Checking by Event Times	122
E.5	Cost Decomposition for One Block	123
E.6	Global and Selected-Timetable Lower Bounds	123

List of Figures

Figure 1.1:	Planning story of the MINOA path-cover method.	2
Figure 2.1:	Positioning of the implemented algorithms.	13
Figure 3.1:	Timetable selection under headway constraints.	18
Figure 3.2:	From benchmark data to solver objects.	25
Figure 4.1:	Compatibility-graph formulation.	32
Figure 4.2:	Bipartite matching transformation for path cover.	39
Figure 4.3:	Capacity ledger at one terminal node.	44
Figure 4.4:	Paid-break accounting between two passenger trips.	45
Figure 5.1:	Network interpretation of local and graph-based construction.	57
Figure 5.2:	Workflow comparison of the four implemented algorithms.	58
Figure 5.3:	EV-feasibility check for one vehicle block.	70
Figure 6.1:	Experiment pipeline used for reproducible result generation.	73
Figure 6.2:	Final solver workflow from input data to archived output.	74
Figure 7.1:	Algorithm comparison on the headline instances.	81
Figure 7.2:	Cost and vehicle trade-off for the headline algorithm variants.	82
Figure 7.3:	Instance-level improvement from the baseline archive to the final no- regression archive.	83
Figure 7.4:	All-instance comparison of the four implemented algorithms.	83
Figure 7.5:	Cost, fleet mix, and efficiency over all Senior instances.	87
Figure 7.6:	Official cost components across all Senior instances.	88
Figure 7.7:	Operational pressure indicators for the Senior instances.	89
Figure 7.8:	Cost per trip and vehicle productivity for all Senior instances.	90
Figure 7.9:	Upper bounds and lower bounds for the headline instances.	93
Figure 7.10:	Global and selected-timetable diagnostic gaps.	94
Figure 7.11:	Runtime of the lower-bound computations.	95
Figure 7.12:	Vehicle lower-bound comparison for the headline instances.	96
Figure 7.13:	Diagnostic gap compared with candidate-trip scale.	97
Figure 7.14:	Fleet mix and operational activity profile of the final schedules.	99
Figure 8.1:	Final result summary of the implemented multi-start path-cover matheuristic.	101

List of Tables

Table 3.1: Main unit conventions used in the thesis.	25
Table 3.2: Structural characteristics of the Senior benchmark instances.	26
Table 4.1: Main notation used in the mathematical formulation.	29
Table 5.1: Pseudocode of the multi-start path-cover matheuristic.	68
Table 5.2: Methodological comparison of the four implemented algorithms.	71
Table 6.1: Main implementation components.	72
Table 6.2: Computing environment used for the reported experiments.	76
Table 7.1: Aggregate comparison of the four implemented algorithms.	80
Table 7.2: Final validated results for Small, Medium, and Large.	84
Table 7.3: Normalized interpretation of the headline results.	84
Table 7.4: Final validated results for all twelve Senior instances.	85
Table 7.5: Baseline archive versus final archive.	91
Table 7.6: Ablation evidence from adaptive candidate families.	91
Table 7.7: Upper and lower bounds for the headline instances.	92
Table 7.8: Internal and external objective reconciliation.	98
Table 7.9: Objective component audit for the final archive.	98
Table A.1: Common execution issues and practical fixes.	115
Table B.1: Complete all-instance comparison of the four implemented algorithms. .	117
Table D.1: Pseudocode for timetable selection in one direction.	119
Table D.2: Pseudocode for compatibility-graph construction and path cover.	119
Table D.3: Pseudocode for the final multi-start path-cover matheuristic.	120
Table D.4: Pseudocode for EV feasibility and charging assignment.	120

List of Abbreviations

Abbreviation	Meaning
MINOA	Mixed Integer Nonlinear Optimization Applications
EV	Electric vehicle
ICE	Internal-combustion-engine vehicle
TT	Timetable
VS	Vehicle schedule
ITTVS	Integrated timetabling and vehicle scheduling
MIP	Mixed-integer programming
DAG	Directed acyclic graph

Chapter 1

Introduction

1.1 Background and Motivation

Urban bus planning has become more connected as electric buses enter fleets that were once operated mainly by conventional vehicles. For an internal combustion engine (ICE) bus, the next trip is mostly a question of time and reachability. For an electric vehicle (EV), the same decision also depends on remaining autonomy, possible charging time, and the availability of a free charger [1–3]. A feasible timetable is therefore not enough on its own.

These links are the focus of this thesis in the MINOA Senior challenge. The problem combines timetable selection, vehicle chaining, fleet assignment, charging decisions, and terminal capacity, and these decisions influence each other throughout the day. A timetable choice changes the trips that must be served; a vehicle connection changes the later battery state; and a charging decision can occupy a limited terminal resource. One challenge in this work was that timetable feasibility and vehicle feasibility first looked separate, but in the benchmark they were strongly linked. The aim is therefore to construct feasible and cost-efficient schedules while keeping these interactions visible.

1.2 Public Transport Planning Context

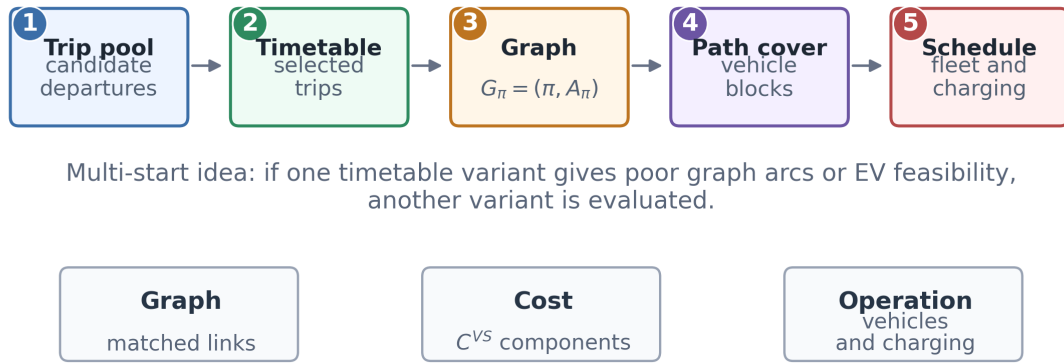
Public transport planning is often divided into levels. Network and line planning define the operated lines and terminals. Timetabling chooses the passenger trips and their departure times. Vehicle scheduling then connects the selected trips into vehicle blocks [4–6]. In a purely sequential process, the timetable is fixed before the vehicle schedule is built. This is easier to handle, but it can remove timetable alternatives that would need fewer vehicles or create better charging opportunities [7, 8].

The MINOA setting sits at the boundary between timetabling and vehicle scheduling. The input contains many candidate trips, but only a feasible subset must be selected. The selected trips must satisfy headway rules, and every selected trip must then be covered exactly once by a vehicle block. Because EV blocks depend on autonomy and charging

opportunities, a good solution must coordinate the passenger-service side and the vehicle-operation side. This is a small but important shift from a purely sequential workflow: the timetable is not only a service plan, but also the input to the later vehicle and charging decisions. This was not immediately obvious at the beginning, because timetable feasibility and vehicle feasibility look like separate questions until the larger instances are actually constructed.

Planning Story of the MINOA Path-Cover Method

The figure follows one schedule candidate from raw trips to the final mixed-fleet vehicle schedule.



The final thesis result is a reproducible route through these objects on all senior instances.

Figure 1.1: Planning story of the MINOA path-cover method.

Figure 1.1 summarizes the planning logic used throughout the thesis. The solver first chooses a feasible timetable from the candidate trip pool. It then asks a vehicle-scheduling question: which selected trip can follow another selected trip? This question becomes a directed compatibility graph. A path cover of this graph gives vehicle blocks. Finally, the solver checks whether each block can be operated by an EV or must remain ICE, including charging time, battery autonomy, and terminal capacity. The main message is that no single layer is sufficient on its own: a timetable must be operable, a vehicle block must respect time and space, and an EV assignment must remain feasible over the full block. The same three decisions return later as timetable paths, compatibility graphs, and mixed-fleet vehicle schedules.

1.3 MINOA Challenge Setting

The MINOA Senior input contains line directions, candidate trips, time windows, terminal nodes, deadhead arcs, vehicle types, charging rates, cost coefficients, and capacity limits.

The output must contain the selected timetable and complete vehicle blocks. Each block starts and ends at the depot, contains selected passenger trips, and may include pull movements, waiting, breaks, parking, and charging activities.

The computational study reports Small, Medium, and Large as the headline instances. It also uses all available Senior instances as a wider benchmark. The additional instances have different numbers of lines, directions, candidate trips, and headway windows. They help to check whether the implementation works only for the three headline files or also for a broader set of Senior inputs.

1.4 Research Objectives and Questions

The main objective is to design and evaluate a solver for integrated timetabling and mixed-fleet vehicle scheduling. The solver should combine timetable selection, vehicle chaining, fleet assignment, charging decisions, and independent feasibility checking in one reproducible workflow.

The work is guided by three research questions:

1. How can a feasible timetable be selected from the candidate trips while respecting headway rules and required boundary trips?
2. How can selected trips be transformed into vehicle blocks in a way that reduces vehicles and operational cost?
3. How much solution quality is gained when a simple greedy construction is extended to graph-based path cover, weighted path cover, and finally a multi-start path-cover matheuristic?

1.5 Scope and Delimitations

The work focuses on the MINOA Senior problem data and its timetable, vehicle-scheduling, charging, capacity, and cost rules. Crew scheduling, stochastic delays, real-time dispatching, passenger assignment, and multi-depot extensions are outside the scope. This restriction keeps the methodology close to the given challenge and makes the computational results easier to interpret.

The method is heuristic for the full integrated problem and does not prove global optimality. However, it uses exact graph subproblems inside the construction, especially matching-based path cover [9–11]. The reported schedules are feasible solutions produced by a reproducible matheuristic. They can also provide useful upper bounds and warm starts for a future exact method, as discussed in [Section 8.4](#).

1.6 Research Contributions

The main contributions are:

1. It formulates the MINOA planning task using consistent notation for timetable selection, compatibility graphs, path-cover vehicle blocks, fleet assignment, battery propagation, charging, capacity, and cost.
2. It implements four algorithmic variants: a greedy constructive baseline, an unweighted path-cover algorithm, a weighted path-cover algorithm, and a multi-start path-cover matheuristic. The final variant also uses bounded charging-aware edge scoring, EV charging look-ahead, and adaptive candidate-family settings.
3. It gives detailed benchmark tables and figures for Small, Medium, Large, and all twelve Senior instances, including graph statistics, cost, vehicle count, EV share, selected-trip percentage, and runtime.
4. It provides a reproducible implementation with a single experiment command and automated result reporting.

1.7 Structure of the Thesis

The thesis is organized so that the reader first sees the planning context and then reaches the algorithmic and computational parts. This order matters because the MINOA problem combines public-transport planning, graph-based vehicle scheduling, and electric-bus feasibility. [Chapter 2](#) starts with the research background. It reviews timetabling, vehicle scheduling, integrated planning, electric-bus scheduling, mixed fleets, and graph-based optimization. The focus is on the ideas needed for this thesis: headway-based timetable selection, trip-to-trip compatibility, path covers, matching, and charging-aware scheduling.

After this background, [Chapter 3](#) introduces the MINOA Senior benchmark in operational terms. It describes the input data, candidate trips, time windows, timetable rules, vehicle-block requirements, depot movements, electric-vehicle autonomy restrictions, charging capacities, and cost components. [Chapter 4](#) then translates these planning rules into mathematical notation. It defines the compatibility graph, path-cover structure, vehicle-type assignment, battery propagation, charging feasibility, capacity restrictions, paid-break accounting, and timetable-variant scoring. These definitions give the precise language used by the solution methodology.

The algorithmic part begins in [Chapter 5](#). The chapter develops the four implemented approaches in increasing order of strength: greedy construction, unweighted path cover, weighted path cover, and the final multi-start path-cover matheuristic. This sequence is useful because it shows why the final method was chosen. The greedy method gives a simple feasible baseline, the path-cover methods use the compatibility graph more globally, and the multi-start method adds timetable and edge-scoring variation. [Chapter 6](#) connects these ideas to the Python software. It describes the repository structure, modules, experiment commands, hardware setting, validation workflow, and reproducibility process.

The empirical part is presented in [Chapter 7](#). It reports the validated results for Small, Medium, Large, and the full Senior benchmark, including objective values, vehicle counts, EV/ICE fleet mix, runtime, lower bounds, charging, and break minutes. [Chapter 8](#) interprets the evidence by comparing the benefits and limits of the methods. Finally, [Chapter 9](#) summarizes the methodological contribution, the main findings, and possible exact-certification extensions.

Chapter 2

Literature Review

Public transport planning is usually divided into several stages. Strategic planning defines the network, tactical planning defines frequencies and timetables, and operational planning assigns vehicles and crews to the planned service. Ceder describes this staged view as a practical way to organize public transport decisions, while Desaulniers and Hickman present public transit as a large optimization field in which planning layers are strongly connected [4, 5]. The separation is useful because real transport systems are too large to optimize in one step without structure. However, it can also hide important links between decisions. A timetable that looks regular from the passenger side may still create many vehicle starts, long depot movements, or charging conflicts when it has to be operated by a mixed fleet.

The MINOA Senior benchmark is located at this interface. It combines trip selection, vehicle-block construction, fleet-type assignment, battery autonomy, charging opportunities, terminal capacities, and cost evaluation [12, 13]. This literature review therefore focuses on the planning ideas that support the thesis method: timetabling, vehicle scheduling, integrated planning, electric-bus scheduling, mixed fleets, and graph-based optimization. The aim is not to reproduce all models from the literature. Instead, the chapter explains which concepts are needed to understand why a multi-start path-cover matheuristic is a suitable solution approach for the studied benchmark.

2.1 Timetabling in Public Transport

Timetabling determines when passenger trips are operated. A timetable is not only a list of departure times. It also decides the time structure that later vehicle schedules must follow. One of the most common service-quality ideas is regularity. In a line or direction with consecutive trips k and $k + 1$, the basic headway can be written as

$$h_k = t_{k+1} - t_k.$$

This expression captures the simple idea behind headway-based service: passengers experience service frequency through the time gap between two consecutive trips. Ceder

discusses headways as a central element of public transport planning because they influence waiting time and perceived service quality [4]. Schobel also treats timetable quality as a central optimization issue, especially when regular departures and passenger waiting times are considered together [14]. Broader timetabling literature studies regularity, balanced departures, and connections between planning layers [7, 8, 15].

Regularity alone is not enough for the problem studied in this thesis. A perfectly regular timetable may still be expensive to operate if the selected trips cannot be chained efficiently by vehicles. For example, two trips may be well spaced for passengers but badly spaced for operations if a bus finishes one trip just too late to serve the next one. In this case, the timetable creates a new vehicle requirement although the passenger-facing pattern looks reasonable. This shows the tension between passenger-oriented timetable quality and vehicle-scheduling quality.

In many classical settings, the timetable is fixed before vehicle scheduling starts. The planner then receives a trip list and must find a vehicle schedule for it. In the MINOA Senior benchmark, this is different: the input contains candidate trips, and the timetable is obtained by selecting a feasible subset of them [12, 13]. This makes trip selection an optimization decision. Selecting more trips may improve service density, but it also increases the number of activities that vehicles must cover. Selecting fewer trips can reduce vehicle effort, but it may violate headway or border-effect rules. The selected timetable therefore becomes the first important decision layer of the implemented method.

The main limitation of timetable-only planning is that it does not fully see the later operating consequences. A regular timetable may require several vehicles to start at similar times, may create poor terminal balances, or may increase deadhead movement. This problem is already visible for conventional fleets, and it becomes stronger for electric fleets because a small shift in a trip time can change whether a vehicle has enough idle time at a charging location. For this reason, timetable quality in this thesis is not judged only by regularity. The selected trips also shape the compatibility graph used for vehicle-block construction.

2.2 Vehicle Scheduling

Vehicle scheduling assigns buses to the selected trips. The main formal object is a vehicle block: an ordered sequence of activities served by one bus. A block usually starts with a pull-out from the depot, continues with passenger trips, waiting, and possible deadhead

movements, and ends with a pull-in to the depot. Bunte and Kliwer review several vehicle-scheduling model families for public transport, including network-flow and set-partitioning formulations [6]. Kliwer, Mellouli, and Suhl show how time-space networks can represent multi-depot bus scheduling exactly when the timetable is fixed [16].

A standard coverage idea in vehicle scheduling is that each trip must be covered by exactly one selected block. In a set-partitioning view this can be written conceptually as

$$\sum_{b \in \mathcal{B}: i \in b} x_b = 1 \quad \forall i.$$

Here, $\mathcal{B}$ is a set of feasible candidate blocks and x_b indicates whether block b is used. The equation is simple, but it shows the core scheduling logic: every selected trip must appear once and only once in the final vehicle schedule. Desaulniers, Desrosiers, and Solomon discuss column generation as one way to handle models where the set of possible blocks is too large to list explicitly [17]. In that setting, the optimization model selects useful columns, while a pricing problem generates new candidate blocks.

Vehicle blocks also provide the link between the mathematical model and the reported operational results. In the single-day schedules studied here, each used block corresponds to one used vehicle, because a physical bus operates one chronological block during the planning horizon. This is why vehicle count is a major result metric in the computational chapter. A lower number of blocks usually means fewer vehicles, lower fixed cost, and less depot movement. However, a block is useful only if it remains feasible with respect to time, stopping rules, and later EV restrictions.

Compatibility graphs give a clear way to construct such blocks. Each selected trip becomes a node. A directed arc from trip i to trip j means that one vehicle can finish i , perform the required waiting or depot movement, and start j on time. A path in this graph represents one vehicle block, so fewer paths usually mean fewer used vehicles. This graph view is closely linked to network-flow and matching theory [9, 18, 19].

Classical vehicle scheduling becomes relatively clean once the timetable is fixed. The planner builds the compatibility graph and then solves a flow, matching, or set-partitioning problem. The limitation is that these models do not automatically include electric-bus feasibility. A block can be feasible for an internal-combustion vehicle but infeasible for an electric bus because the battery becomes too low or because no charger is available at the right time [20]. This motivates the thesis design: construct strong vehicle blocks through graph methods, but check EV feasibility on the complete chronological block.

2.3 Integrated Timetabling and Vehicle Scheduling

Integrated timetabling and vehicle scheduling connects two decisions that are often solved one after another: which trips are operated and how vehicles are chained through those trips. Sequential planning is attractive because it keeps the planning process manageable. The timetable can be designed first, and the vehicle schedule can be optimized afterwards. The drawback is that a timetable decision may look good locally while creating high downstream operating cost. Integrated planning tries to avoid this by considering timetable and vehicle effects together.

Michaelis and Schobel study the integration of line planning, timetabling, and vehicle scheduling and show that these planning tasks influence one another [7]. Schmid and Ehmke focus on integrated timetabling and vehicle scheduling with balanced departure times, which is close in spirit to the idea that passenger regularity and vehicle efficiency should be considered together [8]. Schobel later discusses an iterative model in which line planning, timetabling, and vehicle scheduling are repeatedly adjusted [15]. These studies support the view that timetable decisions should not be treated as completely independent from operational scheduling.

Exact mixed-integer programming formulations are attractive because they can represent trip selection, vehicle-flow decisions, vehicle types, battery states, charging activities, and resource capacities in one model. General linear and integer-programming theory gives the language for such formulations [21, 22]. The difficulty is size. Once timetable selection, mixed-fleet assignment, and charging feasibility are all included, the number of binary decisions and linking constraints can grow quickly. This is one reason why exact integrated models may become difficult on larger benchmark instances.

Matheuristics offer a practical middle ground. They use mathematical optimization ideas for subproblems where these ideas are strong, and heuristic search for the parts that are too large or too coupled to solve exactly. Glover and Kochenberger describe metaheuristic search as a way to explore large solution spaces [23], while Maniezzo, Stutzle, and Voss discuss matheuristics as hybrids of heuristic search and mathematical programming [24]. This thesis follows that direction. It keeps a clear graph-optimization subproblem for fixed timetable variants, but it uses controlled multi-start search to explore different timetable and edge-scoring choices.

This positioning is important for interpreting the contribution. The thesis does not claim that the full integrated problem is solved exactly. Instead, it uses exact graph matching inside a heuristic framework. This gives a stronger construction step than a purely local

greedy rule, while keeping the overall method practical for all Senior instances. The literature therefore motivates both sides of the design: integration is valuable, but full integration can be computationally heavy; graph-based matheuristics are one way to balance these two facts.

2.4 Electric Bus Scheduling and Charging

Electric bus scheduling adds battery autonomy, charging duration, charging location, and charger-capacity limits to the vehicle-scheduling problem. A simple battery-state update can be written as

$$q_{r+1} = q_r - e_r + c_r.$$

The next battery state q_{r+1} depends on the previous state q_r , the energy consumed by driving e_r , and the energy gained by charging c_r . This compact expression explains why EV scheduling is chronological: the feasibility of a later trip depends on all earlier driving and charging decisions. Schneider, Stenger, and Goeke introduce the electric vehicle-routing problem with time windows and recharging stations, where this kind of time-dependent energy logic is central [20]. Keskin and Catay show that partial recharge strategies can be useful, which means that charging duration can become a decision rather than a fixed activity [25].

Total distance alone is therefore not enough to judge EV feasibility. A block with moderate distance may still fail if most driving happens before the first charging opportunity. A longer block may be feasible if it contains well-placed breaks at charging-capable terminals. Li studies transit bus scheduling with limited energy and shows how energy restrictions change the operational problem [1]. Van Kooten Niekerk, van den Akker, and Hoogeveen also emphasize that electric-vehicle scheduling must consider time and energy together [2]. Public-transport EV scheduling studies make the same point: vehicle location, available break duration, and charger capacity are all part of the feasibility decision [1, 2, 26].

Charging location is especially important in a public transport setting. A bus cannot charge whenever it is idle; it must be at a node where a charger exists, and the charger must not already be occupied. Charging time is also not free. If a charging activity uses part of a layover, less time remains for recovery or paid break. If charging extends the layover, it may destroy a useful trip connection. The scheduling problem is therefore not

only about whether an EV has enough energy in total. It is about whether the block contains the right charging opportunities at the right moments.

The literature also shows a trade-off between modelling detail and tractability. Detailed charging models can describe charger types, charging curves, and state-of-charge decisions more realistically. Such detail is valuable, but it can make an integrated timetable and vehicle-scheduling model much larger. Olsen and Kliwer discuss how assumptions about charging processes influence electric-bus scheduling results [27]. For this thesis, the important lesson is to keep the EV check chronological and capacity-aware while still allowing many candidate vehicle blocks to be tested.

2.5 Mixed-Fleet Scheduling

A mixed fleet contains electric vehicles and conventional internal-combustion vehicles. ICE vehicles are operationally flexible because they do not require charging during the planning horizon. EVs can reduce the emission-related cost component, but they are restricted by autonomy and charging resources. A simple block-assignment view is

$$z_b^{EV} + z_b^{ICE} = 1.$$

This expression says that block b is assigned to one vehicle type. In a real mixed-fleet schedule, the choice is not only a label. An EV assignment is useful only if the block has enough autonomy and feasible charging opportunities. Rogge, van der Hurk, Larsen, and Sauer study electric bus fleet size and mix decisions together with charging infrastructure, which shows how fleet composition and charging resources interact [3].

The literature points to an important trade-off: maximizing EV use is not always the same as minimizing total cost. An EV block can be attractive when charging fits naturally into existing breaks. It can be unattractive when it forces additional waiting, creates charger conflicts, or requires an additional vehicle. Tang, Lin, and He study robust scheduling strategies for electric buses under stochastic traffic conditions, which also shows that operational uncertainty can make EV scheduling more delicate [28]. Olsen and Kliwer similarly show that modelling assumptions about charging can change the resulting schedule [27].

This view supports the mixed-fleet logic used in the thesis. Vehicle blocks are first constructed from timetable and compatibility information. Then the algorithm checks

which of these blocks can be operated by electric buses. Electric vehicles are used where the chronological block structure supports them, while conventional vehicles remain necessary for blocks with unfavorable battery or charging conditions. This is a cautious design, but it avoids reporting EV-heavy schedules that are not operationally feasible.

The mixed-fleet setting also explains why the objective cannot be interpreted through one metric alone. A solution with many EVs may still have a high total cost if it uses more vehicles or creates long paid breaks. A solution with fewer EVs may be cheaper but less attractive from an emission perspective. The reported results therefore include total cost, vehicle count, EV/ICE fleet composition, deadhead time, break time, and charging time. These measures belong together because the mixed-fleet decision is both economic and operational.

2.6 Graph Algorithms and Positioning

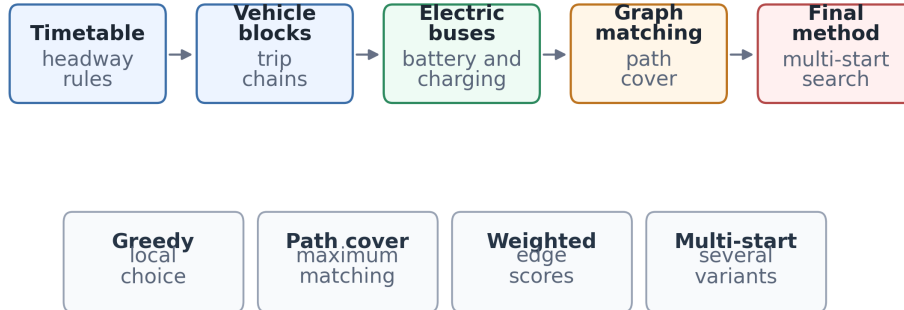
Graph algorithms provide a practical bridge between local construction and large exact formulations. In a directed acyclic compatibility graph, selected trips can be connected into vehicle blocks. For a fixed graph, the path-cover idea can be linked to matching through the relation

$$|\mathcal{P}| = |V| - |M|.$$

Here, V is the set of trip nodes, M is a matching in the bipartite transformation, and $\mathcal{P}$ is the resulting set of paths. The formula shows why maximizing compatible links reduces the number of vehicle paths. Ford and Fulkerson provide a classical basis for network-flow reasoning, while Dulmage and Mendelsohn study coverings of bipartite graphs [9, 18]. Edmonds' matching work and Galil's survey explain why matching is a central combinatorial optimization problem [10, 11]. Lawler gives a broader view of networks and matroids in combinatorial optimization [19].

Positioning of the Implemented Algorithms

The thesis moves from standard planning components to a graph-based matheuristic for mixed fleets.



The four experimental algorithms are therefore not isolated: each one adds one modeling idea to the previous one.

Figure 2.1: Positioning of the implemented algorithms.

Figure 2.1 positions the implemented methods in relation to the reviewed literature. The upper row shows the planning ideas that motivate the work: timetable construction, vehicle blocks, electric-bus feasibility, graph matching, and multi-start search. The lower row shows how the implemented algorithmic variants add these ideas step by step.

The strength of the graph view is transparency. If trip j follows trip i , the reason can be checked through time feasibility, depot movement, and stopping-time data. It also gives a strong fixed-timetable subproblem. Matching can choose compatible continuations jointly, while a greedy rule chooses only one local successor at a time. This explains why graph-based methods are useful for vehicle scheduling [6].

The limitation is equally important. Exact matching is exact only for the fixed compatibility graph that is given to it. It does not by itself solve timetable selection, EV assignment, battery propagation, or charging-capacity decisions. Those decisions remain linked to the constructed blocks. The literature therefore supports the central design of this thesis: use a transparent path-cover structure for vehicle-block construction, but combine it with multi-start timetable variants and charging-aware feasibility checks [3, 8, 26].

Chapter 3

Problem Definition and Benchmark

The planning problem consists of fixed benchmark data, a selected timetable, and a vehicle schedule that operates exactly the selected trips. The main objects are the public-transport network, candidate trips, time-window rules, vehicle-block compatibility, electric-bus autonomy, charging resources, and cost coefficients.

3.1 Study Scope and Input Assumptions

The solver works on fixed MINOA Senior input files [12, 13, 29]. It does not change the network, candidate trips, deadhead arcs, vehicle data, charging rates, node capacities, or cost coefficients. The decisions start after the data are fixed: the solver selects a feasible timetable from the candidate trips and then constructs a feasible vehicle schedule for that timetable.

Let I be one benchmark instance. Its input data can be written as

$$I = (T, D, N, A^{dh}, V, V_E, \mathcal{W}, \mathcal{R}, \mathcal{K}), \quad (3.1)$$

where T is the candidate trip set, D is the set of line directions, N is the set of nodes, A^{dh} is the deadhead-arc data, V is the set of vehicle types, and $V_E \subseteq V$ is the set of electric vehicle types. The tested Senior instances contain one electric type and one ICE type. The formulation also allows several vehicle types, as permitted by the problem statement. The set $\mathcal{W}$ contains time-window dependent rules, $\mathcal{R}$ contains charging and resource-capacity data, and $\mathcal{K}$ contains the cost coefficients [12, 13].

The output contains two coupled objects:

$$S = (\pi, \Omega), \quad (3.2)$$

where $\pi \subseteq T$ is the selected timetable and Ω is the vehicle schedule. A solution is feasible only if the selected timetable and the vehicle schedule describe the same operated passenger trips:

$$T(\Omega) = \pi. \quad (3.3)$$

Equation (3.3) is the central link between timetabling and vehicle scheduling. It means that every selected trip must appear in the vehicle schedule and no unselected candidate trip may appear there. This prevents the solver from choosing one timetable and operating a different one. The formal coverage constraints are stated later in Equations (4.32) and (4.33).

3.2 Network and Service Representation

3.2.1 Lines, Directions, Trips, and Terminals

The candidate passenger trips are grouped by line direction. For each direction $d \in D$, let $T_d \subseteq T$ be the trips of that direction. The direction sets form a partition of the candidate pool:

$$T = \bigcup_{d \in D} T_d, \quad T_d \cap T_{d'} = \emptyset \quad \forall d \neq d'. \quad (3.4)$$

Each trip $i \in T$ has a start node $sn(i)$, an end node $en(i)$, a start time $st(i)$, an end time $et(i)$, and a main-stop time $a(i)$. The headway rules use $a(i)$ because the problem defines service regularity at a main stop, not only at the terminal departure time.

The duration of a passenger trip is

$$\tau_i^{trip} = et(i) - st(i), \quad \tau_i^{trip} > 0. \quad (3.5)$$

The planning model also uses the start and end nodes to decide whether two trips can be connected directly or must be connected through the depot.

3.2.2 Depot, Nodes, and Deadhead Movements

The depot is the location where every vehicle block starts and ends. Terminal nodes are the locations where passenger trips start or end and where vehicles may wait, park, or charge. A deadhead arc describes a non-passenger movement. For a deadhead movement $a \in A^{dh}$, I write travel time and distance as

$$\tau_a^{dh}(t), \quad \ell_a^{dh}, \quad (3.6)$$

where t is the relevant time at which the deadhead time window is chosen. The travel time $\tau_a^{dh}(t)$ is measured in seconds and may depend on the time window. The distance ℓ_a^{dh} is measured in kilometres and does not depend on the time of day [13]. Pull-out and pull-in movements are special deadheads:

$$t_b^{po} = \tau_{dep, sn(i_1)}^{dh}(st(i_1)), \quad t_b^{pi} = \tau_{en(i_q), dep}^{dh}(et(i_q)), \quad (3.7)$$

for a block $b = (i_1, \dots, i_q)$. The notation t_b^{po} denotes pull-out duration from the depot to the first trip, and t_b^{pi} denotes pull-in duration from the last trip back to the depot.

3.2.3 Planning Horizon and Time Windows

All activities of a vehicle block must lie inside the planning horizon of the instance. The horizon is divided into time windows $\mathcal{H} = \{1, \dots, k\}$ by time instants $t_0 < t_1 < \dots < t_k$. The time-window index is

$$\omega(t) = h \iff t_{h-1} < t \leq t_h \quad (3.8)$$

for the time-window index containing time t . At the first boundary, $\omega(t_0) = 1$ is used by the implementation, and times outside the horizon are mapped to the nearest boundary window, matching the input/output convention for external times [13]. If $H = [t^{min}, t^{max}]$, then every activity interval $[s_a, e_a]$ in the vehicle schedule must satisfy

$$t^{min} \leq s_a < e_a \leq t^{max}. \quad (3.9)$$

The headway rules, pull-in times, pull-out times, and stopping-time bounds are indexed by $\omega(\cdot)$. For example, the maximum headway of direction d in window h is denoted by $h_{d,h}^{max}$, while the minimum and maximum stopping times at node n are $\delta_{n,h}^{min}$ and $\delta_{n,h}^{max}$. The relevant window for an in-line stop is selected by the arrival time at the node, here $et(i)$. The same time-window choices appear in the in-line and depot-bridge constraints in Equations (4.5) and (4.7).

3.3 Timetable Selection Layer

3.3.1 Candidate Trips and Selected Timetable

For each direction d , the selected timetable is a subset $\pi_d \subseteq T_d$. The complete selected timetable is

$$\pi = \bigcup_{d \in D} \pi_d. \quad (3.10)$$

The problem is not to operate all candidate trips. The input contains many possible departures, and the solver selects enough of them to satisfy the service rules. The selected-trip percentage is therefore

$$p_T = \frac{|\pi|}{|T|} \cdot 100\%. \quad (3.11)$$

The selected-trip percentage measures the density of the final timetable relative to the candidate pool.

3.3.2 Headway Feasibility

For two consecutive selected trips $i, j \in \pi_d$, the headway is measured at the main stop:

$$w_{ij} = a(j) - a(i). \quad (3.12)$$

If I_{ij}^{max} is the maximum permitted headway for this pair and time window, the feasibility condition is

$$0 < w_{ij} \leq I_{ij}^{max}. \quad (3.13)$$

The value I_{ij}^{max} follows the border-effect rule in the MINOA problem statement. If both main-stop times lie in the same time window, the maximum headway of that window is used. If the two trips lie in two consecutive time windows, the allowed bound is the larger of the two adjacent maximum headways. This avoids making a pair infeasible only because the two main-stop times fall on different sides of a time-window border. The same rule is used in the formal headway graph in [Equations \(4.27\) and \(4.29\)](#).

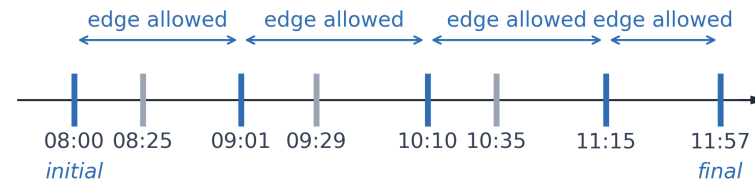
Equivalently, each direction can be represented by a directed headway graph

$$H_d = (T_d, E_d), \quad (i, j) \in E_d \iff a(i) < a(j), \quad a(j) - a(i) \leq I_{ij}^{max}. \quad (3.14)$$

The selected timetable π_d is then an initial-to-final path in H_d . This graph interpretation is important because it connects the passenger-service rule to the algorithmic timetable-selection step. Graph and network representations are also standard tools in vehicle scheduling and network-flow models [\[16, 18, 19\]](#).

Timetable Selection as a Shortest Path in the Headway Graph

Blue trips are selected; grey trips remain alternatives in the dense candidate pool.



Selection rule: minimize $|\pi_d|$ first; use $Q_h(P)$ only to break ties among equally sparse feasible paths.

Figure 3.1: Timetable selection under headway constraints.

[Figure 3.1](#) shows the idea for one direction. Grey departures are available alternatives, while the blue departures form the selected timetable path. Consecutive blue trips must stay within the maximum headway, and the first and last selected trips must satisfy the boundary requirements. The figure explains why timetable selection is a graph path problem in this thesis: the chosen blue trips form a feasible path through the candidate departures, and every later vehicle block is built only from this selected path. This is the first decision layer of the problem: before any vehicle can be scheduled, the solver

must choose a passenger timetable that does not leave excessive gaps between consecutive departures in one direction.

3.3.3 Initial and Final Trip Requirements

The selected timetable must also satisfy boundary requirements. Let $T_d^{start} \subseteq T_d$ be the admissible initial trips and $T_d^{end} \subseteq T_d$ be the admissible final trips for direction d . If $\pi_d = (i_1, \dots, i_q)$, then

$$i_1 \in T_d^{start}, \quad i_q \in T_d^{end}. \quad (3.15)$$

Thus, timetable selection is not only a sparse subset choice. It is a path choice:

$$\pi_d = (i_1, \dots, i_q) \quad \text{with} \quad (i_r, i_{r+1}) \in E_d \quad r = 1, \dots, q-1. \quad (3.16)$$

[Chapter 4](#) uses this path view in the formal selected-timetable definition in [Equation \(4.30\)](#) and in the dynamic recursion [Equation \(4.31\)](#).

3.4 Vehicle-Scheduling Layer

3.4.1 Vehicle Blocks and Trip Coverage

A vehicle block is an ordered sequence of vehicle activities. In its passenger trip view, a block can be written as

$$b = (i_1, i_2, \dots, i_q), \quad i_r \in \pi. \quad (3.17)$$

The set of all blocks is B , and the vehicle schedule is $\Omega = \{b : b \in B\}$. Every selected trip must be covered exactly once:

$$\sum_{b \in B} y_{ib} = 1 \quad \forall i \in \pi, \quad (3.18)$$

where $y_{ib} = 1$ if trip i is contained in block b . Unselected trips are not covered:

$$\sum_{b \in B} y_{ib} = 0 \quad \forall i \in T \setminus \pi. \quad (3.19)$$

The number of utilized vehicles equals the number of blocks:

$$N^{veh} = |B|. \quad (3.20)$$

The exact-once coverage rule and the exclusion of unselected trips are written formally in [Equations \(4.32\)](#) and [\(4.33\)](#). These constraints are the vehicle-schedule side of the timetable–vehicle link.

3.4.2 In-Line and Depot-Bridge Compatibility

Two selected trips i and j can be served by the same vehicle only if the vehicle can finish i and start j on time. If $en(i) = sn(j)$, the connection is in-line. The stop duration is

$$\Delta_{ij} = st(j) - et(i). \quad (3.21)$$

The in-line compatibility condition is

$$en(i) = sn(j), \quad \delta_{en(i), \omega(et(i))}^{min} \leq \Delta_{ij} \leq \delta_{en(i), \omega(et(i))}^{max}. \quad (3.22)$$

If the trips do not meet at the same terminal, the vehicle may return to the depot and leave again. With depot dep , the depot-bridge condition is

$$et(i) + \tau_{en(i), \omega(et(i))}^{PI} + \delta_{dep, \omega(et(i) + \tau_{en(i), \omega(et(i))}^{PI})}^{min} + \tau_{sn(j), \omega(st(j))}^{PO} \leq st(j). \quad (3.23)$$

The condition is written with a depot stopping term δ_{dep}^{min} . If no positive depot stopping time is specified in an instance, $\delta_{dep}^{min} = 0$. Keeping it in the equation makes the formulation valid for instances where such a minimum stop is present. The formal graph edge tests are given in [Equations \(4.5\)](#) and [\(4.7\)](#).

The compatibility graph built from these tests is a directed acyclic graph. The direction comes from time: an edge can only point from an earlier trip to a later trip. This simplifies vehicle-block construction. A path in the graph is already a chronological sequence of trips,

and a path cover is a set of such paths that covers the selected timetable. The formal graph definition and its path-cover use are introduced in [Section 4.2](#).

3.4.3 Break, Waiting, and Stopping-Time Requirements

Stopping time is first a feasibility rule. Paid break time is then a cost component. For a stop activity a , let τ_a be the total stop duration, τ_a^{charge} the part used for charging, and $\delta_{n(a),w(a)}^{min}$ the required minimum stop at node $n(a)$ in the stop-start window $w(a)$. The paid break part is

$$p_a = \begin{cases} \max \{0, \tau_a - \max(\tau_a^{charge}, \delta_{n(a),w(a)}^{min})\}, & \text{in-line stop,} \\ 0, & \text{stop adjacent to pull-in or pull-out.} \end{cases} \quad (3.24)$$

Thus, the same waiting interval can play different roles. One part may satisfy minimum stopping time, another part may be used for charging, and only the remaining part contributes to paid break cost. The corresponding paid-break constraints are stated in [Equations \(4.64\)](#) and [\(4.65\)](#).

3.5 Electric-Bus and Charging Layer

3.5.1 Battery Autonomy

For an EV block, residual autonomy is simulated along the ordered activity sequence. Let r_{ba}^{pre} and r_{ba}^{post} be the residual autonomy before and after activity a in block b . The vehicle starts from the depot with full autonomy. For a driving activity,

$$r_{ba}^{post} = r_{ba}^{pre} - \ell_a. \quad (3.25)$$

For a charging activity, $r_{ba}^{post} = r_{ba}^{pre} + \Delta r_{ba}^{charge}$, subject to the autonomy upper bound.

An EV assignment is feasible only if

$$0 \leq r_{ba}^{pre}, r_{ba}^{post} \leq A_v^{max} \quad \forall a \in A_b. \quad (3.26)$$

This explains why EV feasibility is checked after a vehicle block has been constructed. The same set of trips may be easy or difficult for an EV depending on the order, deadhead movements, and available charging breaks [1, 2, 28]. The formal battery propagation constraints are given in Equations (4.48) to (4.51).

ICE vehicles do not have battery or charging constraints. This does not mean that an ICE block can ignore the other rules. The selected timetable must first be feasible, and the block must still satisfy chronological order, pull-in and pull-out timing, in-line or depot-bridge compatibility, stopping-time rules, and parking capacity where a break is used. Only after these conventional vehicle-schedule conditions are satisfied can the battery and charging restrictions be omitted for an ICE assignment. The EV fleet availability rule is formalized in Equation (4.46); the ICE count relation is given in Equation (4.47).

3.5.2 Slow and Fast Charging

Let q_a^{slow} and q_a^{fast} be the slow and fast charging durations inserted during break activity a . Charging must fit inside the available stop:

$$q_a^{slow} + q_a^{fast} \leq \tau_a. \quad (3.27)$$

For slow charging of vehicle type v , the autonomy gain is

$$\Delta r_a^{slow} = A_v^{max} \frac{q_a^{slow}}{T_v^{slow}}. \quad (3.28)$$

For fast charging, the charging time is scaled by the fast-charging multiplier $\phi \in (0, 1)$:

$$\Delta r_a^{fast} = A_v^{max} \frac{q_a^{fast}}{\phi_v T_v^{slow}}. \quad (3.29)$$

The implemented schedule chooses either no charging, slow charging, or fast charging for a charging activity. This can be expressed as

$$\Delta r_a^{charge} \in \{0, \Delta r_a^{slow}, \Delta r_a^{fast}\}. \quad (3.30)$$

The corresponding duration and gain constraints are written in Equations (4.52) and (4.54) to (4.56).

3.5.3 Minimum Charging Time and Partial Recharging

If a charging activity is used, it must respect the minimum charging duration of the EV type:

$$q_a^m = 0 \quad \text{or} \quad q_a^m \geq q_v^{min,m}, \quad m \in \{\text{slow}, \text{fast}\}. \quad (3.31)$$

Partial charging is allowed because the vehicle does not need to return to full autonomy after every stop. The upper bound is the maximum battery autonomy:

$$r_{ba}^{post} \leq A_v^{max}. \quad (3.32)$$

The minimum charging-time rule and the minute-granularity rule are stated formally in [Equations \(4.53\)](#) and [\(4.57\)](#).

3.5.4 Parking and Charging Capacity at Nodes

Capacity is checked by overlapping activity intervals. Let o_a^{park} , o_a^{slow} , and o_a^{fast} indicate which resource a break window occupies. Let $\mathcal{A}_n(t)$ be the break windows active at node n and time t . The capacity rules are

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{park} \leq K_n^{park}, \quad (3.33)$$

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{slow} \leq K_n^{slow}, \quad (3.34)$$

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{fast} \leq K_n^{fast}. \quad (3.35)$$

These constraints are important because a block can be feasible in isolation but infeasible together with other blocks if too many vehicles use the same terminal resource at the same time. The output field `typeSpot` specifies the occupied resource, while `isCharging` specifies whether active charging occurs in that occupied slot [\[13\]](#). The formal resource-occupation and capacity constraints are given in [Equations \(4.58\)](#) and [\(4.60\)](#) to [\(4.63\)](#).

3.6 Objective and Cost Structure

The reported objective is the vehicle-scheduling cost C^{VS} . It contains fixed vehicle cost, paid break cost, pull-in and pull-out cost, and the CO₂ cost for conventional driving [12]. Fixed vehicle cost is charged for every used vehicle block. Paid-break cost is charged for residual break time after mandatory stopping time and active charging have been accounted for. Pull cost covers the pull-out and pull-in movements between the depot and the first or last serviced terminal of a block. In the reported objective decomposition, the CO₂ term applies the vehicle-specific coefficient to the passenger-service driving duration $d(b)$. For EV types $v \in V_E$, the input coefficient $c_v^{\text{CO}_2}$ is zero.

Chapter 4 gives the formal objective C^{VS} in Equation (4.19) and decomposes it into Equations (4.22) to (4.25).

3.7 Benchmark Instances and Unit Conventions

Times are stored in seconds in the input and output files, while result tables usually report operational durations in minutes:

$$t^{min} = \frac{t^{sec}}{60}. \quad (3.36)$$

All feasibility checks use seconds. Charging windows are written in whole-minute durations, so a computed required charging duration is rounded up to the next multiple of 60 seconds before it is inserted into the output. Rounding upward is used because rounding downward could undercharge an EV block. Capacity is then checked on these minute-aligned charging and parking intervals. Result tables divide the final accumulated seconds by 60 for readability; the displayed minutes do not change the validator-facing schedule.

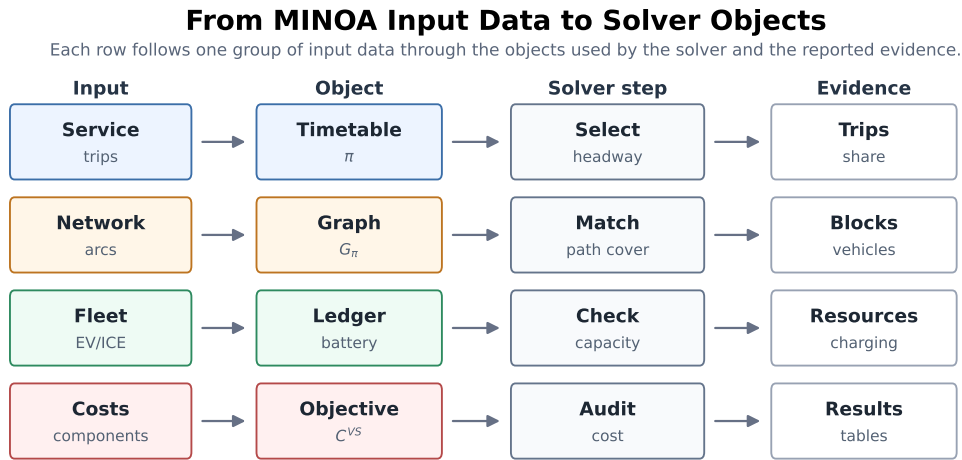
Distances and autonomy follow the units provided in the instance files. Costs are reported in the same cost units as the problem statement and the checking output.

Table 3.1 summarizes the unit conventions used in the formulation, implementation, and reported results.

Table 3.1: Main unit conventions used in the thesis.

Symbol	Meaning	Unit
ℓ_i	passenger-trip distance	km
ℓ_a^{dh}	deadhead-arc distance	km
τ_i^{trip}	passenger-trip driving time	s
τ_a^{dh}	deadhead driving time	s
$d(b)$	passenger-service driving duration used in the CO ₂ audit	s
A_v^{max}	maximum EV autonomy of type v	km
q_a	charging duration in a break window	s
$c_v^{CO_2}$	conventional driving cost coefficient	cost/s

The main comparison instances are Small, Medium, and Large. In addition, the implementation is evaluated on all available Senior instances to check how the method behaves beyond the headline files. Table 3.2 lists the twelve instances used in the thesis. The table reports the structural features that later influence graph size, vehicle chaining, and EV feasibility: line directions, terminal nodes, deadhead arcs, headway windows, candidate trips, planning horizon, EV fleet cap, and EV autonomy.



Reading guide: the same benchmark data feed all algorithms; the algorithms differ in timetable variants, graph weights, and EV charging checks.

Figure 3.2: From benchmark data to solver objects.

Figure 3.2 is used as the reference map for the following chapters. The benchmark data are first converted into timetable, graph, and vehicle-operation objects. The mathematical model formalizes these objects, and the methodology chapter explains how the solver constructs them. The main interpretation is that the solver does not work directly on a flat JSON file. It first turns the input into structured objects, and each object is responsible for one part of the MINOA rules. The raw benchmark properties become solver objects

such as timetable paths, compatibility arcs, vehicle blocks, charging intervals, and audit outputs.

Table 3.2: Structural characteristics of the Senior benchmark instances.

Instance	Dir.	Nodes	Deadhead arcs	Headway windows	Cand. trips	Horizon (min)	EV cap	Auton. (km)
Small	2	3	4	6	1,064	568	5	90
Medium	4	4	6	12	3,368	870	5	120
Large	10	7	12	30	6,960	743	5	80
Toy Example	4	4	6	12	1,448	390	5	60
1line	2	3	4	6	1,920	959	5	85
1line 6timeWindow	2	3	4	12	1,920	959	5	85
2lines	4	4	6	12	3,840	959	5	85
2lines 6 timeWindows	4	4	6	24	3,840	959	5	85
3lines	6	5	8	18	5,760	959	5	85
3linesTriangle	6	4	6	18	5,760	959	5	85
5lines	10	7	12	30	9,600	959	5	85
8lines	16	10	18	48	15,360	959	5	85

The first three rows show why Small, Medium, and Large form a natural headline set. They increase from 2 to 10 directions and from 1,064 to 6,960 candidate trips, while EV autonomy also changes. The additional instances extend the range up to 16 directions and 15,360 candidate trips. They are used to study robustness and scaling behavior.

The structural columns in [Table 3.2](#) affect different parts of the solution method. The number of directions determines how many independent timetable-selection sequences have to be built. The number of terminal nodes and deadhead arcs determines how many depot-mediated continuations may exist. The number of headway windows determines how often the service-frequency rule changes over the planning horizon. The number of candidate trips controls the size of the initial timetable-selection problem.

The candidate-trip count is much larger than the number of selected trips in the final solutions. This is expected. The input contains many potential departures, while the timetable rules require a feasible subset. For example, Large contains 6,960 candidate trips, but the final selected timetable contains 260 trips. The optimization task is therefore not to operate all candidates. It is to choose a timetable that satisfies the service rules and can be operated with a feasible vehicle schedule.

The EV fleet cap and autonomy columns explain why the same algorithm can lead to different EV shares across instances. A high EV cap does not mean that all vehicles can become electric. The vehicle blocks must also fit the autonomy limit and must contain feasible charging opportunities. Conversely, an instance with moderate autonomy can still use several EVs if the constructed blocks are short enough or if charging breaks appear at useful nodes.

The planning horizon also matters. A longer horizon does not automatically mean that the instance is harder. If trips are spread over a long day, vehicles may have more opportunities to wait or charge. If many trips are concentrated in overlapping time windows, more vehicles are needed even if the horizon is short. This is why the computational chapter reports not only total cost but also selected trips, vehicle count, deadhead minutes, break minutes, and charge minutes.

The benchmark structure motivates the graph formulation in [Chapter 4](#). Each selected trip becomes a node, and every feasible continuation becomes an arc. The size and density of this graph depend on the time windows, terminal layout, and deadhead movements listed in [Table 3.2](#). The graph is therefore not an abstract modeling device only; it is a direct representation of the operational structure of each instance.

Chapter 4

Mathematical Formulation and Graph Model

The mathematical formulation connects timetable selection, vehicle chaining, fleet assignment, battery feasibility, charging, parking capacity, and cost calculation. Solving all of these decisions in one exact model would be very large. The formulation is therefore used to derive the graph and matching subproblems solved inside the matheuristic. Here π denotes the selected timetable, G_π the compatibility graph, B the set of vehicle blocks, and C^{VS} the reported vehicle-scheduling cost.

4.1 Notation and Planning Objects

4.1.1 Sets, Parameters, and Indices

[Table 4.1](#) summarizes the notation used in the mathematical chapters.

Table 4.1: Main notation used in the mathematical formulation.

Symbol	Meaning	Unit
T, D, N	Sets of candidate trips, directions, and nodes.	—
V, V_E	Sets of vehicle types and electric vehicle types, with $V_E \subseteq V$.	—
i, j, b, a, v	Indices for trips, vehicle blocks, activities, and vehicle types.	—
π_d, π	Selected timetable of direction d , and complete selected timetable.	—
$st(i), et(i), a(i)$	Trip start time, trip end time, and main-stop time.	s
$sn(i), en(i)$	Start and end node of trip i .	—
ℓ_i	Passenger-trip distance.	km
ℓ_a^{dh}	Deadhead distance of activity or arc a .	km
τ_i^{trip}	Passenger-trip driving time $et(i) - st(i)$.	s
τ_a^{dh}	Deadhead driving time.	s
$\omega(t)$	Time-window index containing time t .	—
$h_{d,\omega}^{max}$	Maximum headway for direction d in time window ω .	s
$\delta_{n,\omega}^{min}, \delta_{n,\omega}^{max}$	Minimum and maximum stopping time at node n .	s
$t_b^{pi}, t_b^{po}, d(b)$	Pull-in duration, pull-out duration, and passenger-service duration used for the CO ₂ term.	s
B, G, π, H_d	Vehicle-block set, compatibility graph, and headway graph.	—
u_{bv}	Binary vehicle-type variable; equals 1 if block b uses vehicle type v .	—
z_{ij}	Binary matching variable; equals 1 if trip j directly follows trip i .	—
y_{ib}	Binary coverage variable; equals 1 if trip i is served by block b .	—
$r_{ba}^{pre}, r_{ba}^{post}$	Residual EV autonomy before and after activity a in block b .	km
q_a^{slow}, q_a^{fast}	Slow and fast charging duration in break activity a .	s
$o_a^{park}, o_a^{slow}, o_a^{fast}$	Resource-occupancy variables for parking, slow charging, and fast charging.	—
g_a	Binary or state variable indicating active charging in break window a .	—
p_a	Paid-break duration of break activity a .	s
$A_v^{max}, T_v^{slow}, q_v^{min,m}$	EV autonomy, full slow-charge time, and minimum charging duration.	mixed
ϕ_v	Fast-charging multiplier for EV type v .	—
$K_n^{park}, K_n^{slow}, K_n^{fast}$	Parking, slow-charge, and fast-charge capacities at node n .	vehicles
$c_v^{fix}, c_v^{break}, c_v^{pio}, c_v^{CO_2}$	Fixed, paid-break, pull-in/out, and CO ₂ cost coefficients.	mixed
$C^{VS}, C^{fix}, C^{break}, C^{pull}, C^{CO_2}$	Total vehicle-scheduling cost and its fixed, paid-break, pull, and CO ₂ components.	cost
$UB, LB^{global}, LB^{TT}(\pi)$	Validated upper bound, global lower bound, and selected-timetable diagnostic lower bound.	cost
$N_{veh}, s_{EV}, \eta_{veh}, \bar{c}_T$	Used vehicles, EV share, vehicle productivity, and average cost per selected trip.	mixed
M^{veh}, M^{break}	Vehicle-first matching constant and paid-break linearization constant.	s
$Q_h(P)$	Timetable tie-breaking score for path P under start h .	score

4.1.2 Layered Decision Structure

The solver builds a solution through a sequence of related objects:

$$T \longrightarrow \pi \longrightarrow G_\pi = (\pi, A_\pi) \longrightarrow B \longrightarrow (\Omega, C^{VS}). \quad (4.1)$$

Equation (4.1) is the main modeling idea of the thesis. First, a feasible timetable is selected. Second, feasible trip-to-trip continuations are written as a graph. Third, a path cover of this graph gives vehicle blocks. Fourth, the blocks are assigned to EV or ICE and checked for charging and capacity feasibility. The final cost is computed only after the complete schedule exists.

4.2 Compatibility-Graph Formulation

4.2.1 Trip Nodes and Time-Ordered Candidate Pairs

For a fixed selected timetable π , every selected trip becomes a graph node. Only time-ordered trip pairs can become graph arcs:

$$P_\pi^{time} = \{(i, j) \in \pi \times \pi : et(i) < st(j)\}. \quad (4.2)$$

The compatibility arc set is a subset of these ordered pairs:

$$A_\pi \subseteq P_\pi^{time}. \quad (4.3)$$

The graph is therefore directed and acyclic under the time ordering. This property is important because every path in the graph can be interpreted as a vehicle block.

4.2.2 In-Line Continuation Edges

An in-line edge is possible when the end node of the first trip is the start node of the second trip. The available stop duration is

$$\Delta_{ij} = st(j) - et(i). \quad (4.4)$$

The in-line compatibility rule is

$$en(i) = sn(j), \quad \delta_{en(i),\omega(et(i))}^{min} \leq \Delta_{ij} \leq \delta_{en(i),\omega(et(i))}^{max}. \quad (4.5)$$

The time window is selected by the arrival time at the terminal, $et(i)$. This follows the problem statement: if a break crosses several time windows, the stopping-time bounds are those of the first window, i.e., the arrival window at the node [12].

If $\chi_{ij}^{in} = 1$ denotes in-line compatibility, then

$$\chi_{ij}^{in} = \mathbf{1} \left[en(i) = sn(j) \wedge \delta_{en(i),\omega(et(i))}^{min} \leq \Delta_{ij} \leq \delta_{en(i),\omega(et(i))}^{max} \right]. \quad (4.6)$$

4.2.3 Depot-Bridge Continuation Edges

If two trips cannot be connected directly at the same terminal, the vehicle can return to the depot and leave again. This creates a depot-bridge edge. With depot node dep , the feasibility condition is

$$en(i) \neq sn(j), \quad et(i) + \tau_{en(i),\omega(et(i))}^{PI} + \delta_{dep,\omega(et(i)+\tau_{en(i),\omega(et(i))}^{PI})}^{min} + \tau_{sn(j),\omega(st(j))}^{PO} \leq st(j). \quad (4.7)$$

This inequality is the basic temporal test for a depot bridge. It does not by itself describe all EV charging side conditions. If a vehicle stops at a terminal before pull-in or after pull-out, that stop is allowed only for charging and must also satisfy the corresponding charging, maximum stopping time, and capacity rules [12]. In the implementation, a depot bridge is written as pull-in, possible depot break, and pull-out. Terminal charging adjacent to pull-in or pull-out is not inserted by the current heuristic.

The corresponding indicator is

$$\begin{aligned} \chi_{ij}^{dep} = \mathbf{1} \left[en(i) \neq sn(j) \wedge et(i) + \tau_{en(i),\omega(et(i))}^{PI} \right. \\ \left. + \delta_{dep,\omega(et(i)+\tau_{en(i),\omega(et(i))}^{PI})}^{min} + \tau_{sn(j),\omega(st(j))}^{PO} \leq st(j) \right]. \end{aligned} \quad (4.8)$$

If the instance does not define a positive depot stopping time, then $\delta_{dep}^{min} = 0$. The term remains in the model so that the condition is correct for general input instances with a positive depot stop.

4.2.4 Arc Set and Edge Attributes

A directed arc is accepted if at least one of the two compatibility tests is true:

$$(i, j) \in A_\pi \iff (i, j) \in P_\pi^{time} \wedge (\chi_{ij}^{in} = 1 \vee \chi_{ij}^{dep} = 1). \quad (4.9)$$

For each accepted arc, the implementation stores a small vector of operational attributes:

$$\xi_{ij} = (t_{ij}^{dead}, t_{ij}^{wait}, t_{ij}^{break}, d_{ij}^{drive}, \text{type}_{ij}). \quad (4.10)$$

These values are used by the greedy and weighted path-cover algorithms. The reported objective is still computed later from the full vehicle schedule.

Compatibility-Graph Formulation Used by the Solver

A directed arc is added only when the same vehicle can finish one selected trip and start the next trip on time.

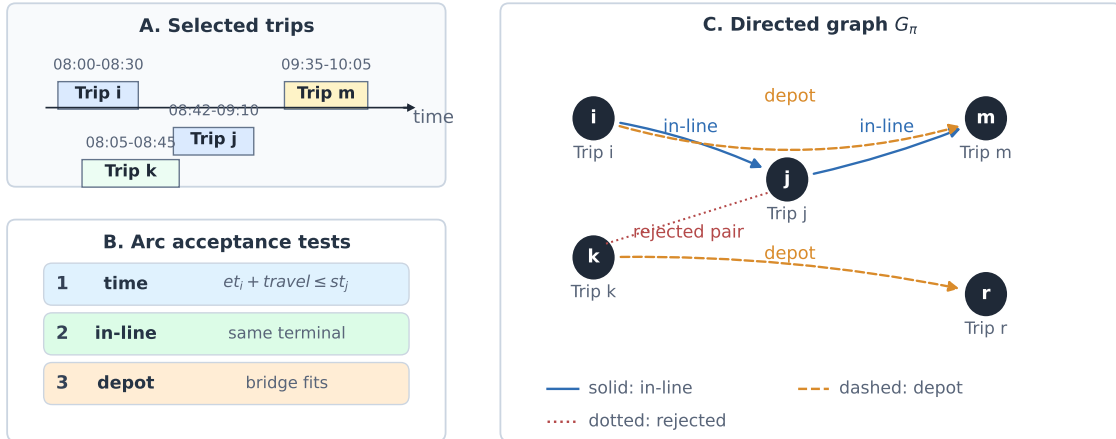


Figure 4.1: Compatibility-graph formulation.

Figure 4.1 gives the intuition behind Equation (4.9). The graph is not a drawing of the physical network. It is a graph of possible vehicle continuations. Each selected trip is a node. An arc is added when the same vehicle can serve the second trip after the first

one. In-line arcs use Equation (4.5), depot-bridge arcs use Equation (4.7), and the final acceptance rule is given in Equation (4.9). Later, the variables x_{ij} and z_{ij} select arcs from this graph. The main message is that vehicle scheduling is reduced to choosing compatible continuations, while all incompatible pairs are removed before matching is solved. This compatibility graph connects the mathematical formulation to the later matching algorithm and should be read as an operational graph, not as a geographic map.

4.3 Decision Variables and State Variables

4.3.1 Timetable Selection Variables

The selected timetable is denoted by π . Its binary variable form is

$$s_i = \begin{cases} 1, & i \in \pi, \\ 0, & i \notin \pi. \end{cases} \quad (4.11)$$

For a direction d , selected trips satisfy

$$\pi_d = \{i \in T_d : s_i = 1\}, \quad \pi = \{i \in T : s_i = 1\}. \quad (4.12)$$

4.3.2 Coverage and Chain Variables

The coverage variable links trips to vehicle blocks:

$$y_{ib} = \begin{cases} 1, & \text{trip } i \text{ is covered by block } b, \\ 0, & \text{otherwise.} \end{cases} \quad (4.13)$$

The direct continuation variable is

$$x_{ij} = \begin{cases} 1, & \text{trip } j \text{ directly follows trip } i, \\ 0, & \text{otherwise.} \end{cases} \quad (4.14)$$

This variable is the algebraic version of a selected arc in [Figure 4.1](#). It enters the predecessor and successor limits in [Equations \(4.34\)](#) and [\(4.35\)](#).

The matching model uses z_{ij} for the same predecessor-successor choice in the bipartite graph:

$$z_{ij} = 1 \iff (i^L, j^R) \text{ is selected in the matching.} \quad (4.15)$$

The variable z_{ij} is introduced separately because the implemented path-cover algorithms solve a bipartite matching problem. It appears in the unweighted matching objective [Equation \(4.36\)](#), the matching constraints [Equations \(4.37\)](#) to [\(4.39\)](#), and the weighted matching objective [Equation \(4.41\)](#).

4.3.3 Vehicle-Type Assignment Variables

For each block $b \in B$, the vehicle-type decision is represented by the general variable

$$u_{bv} \in \{0, 1\} \quad \forall b \in B, v \in V. \quad (4.16)$$

Exactly one vehicle type is assigned:

$$\sum_{v \in V} u_{bv} = 1 \quad \forall b \in B. \quad (4.17)$$

In the tested Senior instances, V contains one electric type and one ICE type. For result reporting I therefore still count EV and ICE blocks, but the mathematical notation does not rely on there being only two vehicle types.

4.3.4 Battery, Charging, and Break Variables

For each activity a in a block b , r_{ba}^{pre} and r_{ba}^{post} denote residual EV autonomy before and after the activity. Charging durations are split by mode, and p_a denotes paid-break duration:

$$r_{ba}^{pre}, r_{ba}^{post} \geq 0, \quad q_a^{slow}, q_a^{fast} \geq 0, \quad p_a \geq 0. \quad (4.18)$$

These variables are not solved in one large exact model in the implementation. They are computed and checked during EV assignment and output construction. They connect the software checks to the equations that follow. Residual autonomy is propagated in Equations (4.49) to (4.51). Charging duration appears in Equations (4.52), (4.54), (4.55) and (4.57). The paid-break variable p_a is used in the objective term Equation (4.23) and is defined more explicitly in Equations (4.64) and (4.65).

4.4 Objective Function

4.4.1 Reported Vehicle-Scheduling Cost

All reported costs follow the MINOA vehicle-scheduling objective. For a set of vehicle blocks B , the vehicle-scheduling cost is denoted by C^{VS} . The result tables report the same quantity as the cost reported by the MINOA desktop validator for the final output.

$$C^{VS} = \sum_{b \in B} \sum_{v \in V} \left[c_v^{fix} + c^{break} t_b^{break} + c_v^{pio} (t_b^{pi} + t_b^{po}) + c_v^{CO_2} d(b) \right] u_{bv}. \quad (4.19)$$

Here c_v^{fix} is the fixed cost of vehicle type v , t_b^{break} is paid break time, t_b^{pi} is pull-in time, and t_b^{po} is pull-out time. In the reported objective decomposition, $d(b)$ denotes the passenger-service driving duration to which the instance-specific CO₂ coefficient is applied:

$$d(b) = \sum_{i \in b} \tau_i^{trip}. \quad (4.20)$$

Although the written MINOA problem description refers to $d(b)$ as the total driving time of block b , the evaluation convention used for the reported outputs applies the CO₂ coefficient to passenger-service trip duration. Pull-in and pull-out durations are accounted for separately through the c_v^{pio} term. The formulation and the reported values follow this convention, and the internal objective reconstruction agrees with the external audit values.

This is different from EV autonomy, which is measured in kilometres and decreases for both passenger trips and deadhead movements. The CO₂ coefficient $c_v^{CO_2}$ is zero for EV types and positive for ICE types when specified in the input data [12, 13].

4.4.2 Objective Decomposition

For interpretation, I decompose the objective into four parts:

$$C^{VS} = C^{fix} + C^{break} + C^{pull} + C^{CO_2}. \quad (4.21)$$

The fixed vehicle cost is

$$C^{fix} = \sum_{b \in B} \sum_{v \in V} c_v^{fix} u_{bv}. \quad (4.22)$$

The paid-break cost is

$$C^{break} = c^{break} \sum_{b \in B} \sum_{\alpha \in A_b^{break}} p_{\alpha}. \quad (4.23)$$

Pull-in and pull-out cost is

$$C^{pull} = \sum_{b \in B} \sum_{v \in V} c_v^{pio} (t_b^{pi} + t_b^{po}) u_{bv}. \quad (4.24)$$

The conventional-emission cost is

$$C^{CO_2} = \sum_{b \in B} \sum_{v \in V} c_v^{CO_2} d(b) u_{bv}. \quad (4.25)$$

This decomposition separates improvements from fewer vehicles, shorter breaks, lower pull movement, and a different EV/ICE split.

4.4.3 Internal Edge Scores

The algorithms also use local edge scores, but these scores are not the reported objective. For an arc (i, j) , a generic local score is

$$\hat{c}_{ij} = \alpha_1 t_{ij}^{dead} + \alpha_2 t_{ij}^{wait} + \alpha_3 t_{ij}^{break}. \quad (4.26)$$

This score helps rank feasible continuations. The final tables, however, use C^{VS} from Equation (4.19).

4.5 Core Feasibility Constraints

4.5.1 Timetable Feasibility Constraints

For each direction d , a directed headway graph $H_d = (T_d, E_d)$ is built:

$$(i, j) \in E_d \iff a(i) < a(j) \wedge a(j) - a(i) \leq I_{ij}^{max}. \quad (4.27)$$

The achieved headway is

$$w_{ij} = a(j) - a(i), \quad (4.28)$$

and every consecutive selected pair satisfies

$$w_{ij} \leq I_{ij}^{max}. \quad (4.29)$$

The selected timetable of direction d is an initial-to-final path P_h^d in this graph. The full selected timetable is

$$\pi = \bigcup_{d \in D} \pi_d, \quad \pi_d = P_h^d. \quad (4.30)$$

For the sparse timetable construction, the dynamic recursion is

$$D_j = 1 + \min_{i: (i,j) \in E_d} D_i, \quad (4.31)$$

with $D_j = 1$ for admissible initial trips.

4.5.2 Timetable–Vehicle Schedule Linking

Every selected trip is covered once:

$$\sum_{b \in B} y_{ib} = 1 \quad \forall i \in \pi. \quad (4.32)$$

Unselected trips are not covered:

$$\sum_{b \in B} y_{ib} = 0 \quad \forall i \in T \setminus \pi. \quad (4.33)$$

In path-cover form, a selected trip can have at most one direct successor and at most one direct predecessor:

$$\sum_{j: (i,j) \in A_\pi} x_{ij} \leq 1 \quad \forall i \in \pi, \quad (4.34)$$

$$\sum_{i: (i,j) \in A_\pi} x_{ij} \leq 1 \quad \forall j \in \pi. \quad (4.35)$$

4.5.3 Matching Constraints for Path Cover

The path-cover model uses a bipartite graph with left trip copies π^L and right trip copies π^R . The unweighted matching objective is

$$\max \sum_{(i,j) \in A_\pi} z_{ij}. \quad (4.36)$$

The matching constraints are

$$\sum_{j: (i,j) \in A_\pi} z_{ij} \leq 1 \quad \forall i \in \pi, \quad (4.37)$$

$$\sum_{i: (i,j) \in A_\pi} z_{ij} \leq 1 \quad \forall j \in \pi, \quad (4.38)$$

$$z_{ij} \in \{0, 1\} \quad \forall (i, j) \in A_\pi. \quad (4.39)$$

If $m = \sum z_{ij}$ continuations are selected, then the number of vehicle blocks is

$$|B| = |\pi| - m. \quad (4.40)$$

The weighted path-cover variant keeps the same constraints and uses

$$\max \sum_{(i,j) \in A_\pi} w_{ij} z_{ij}. \quad (4.41)$$

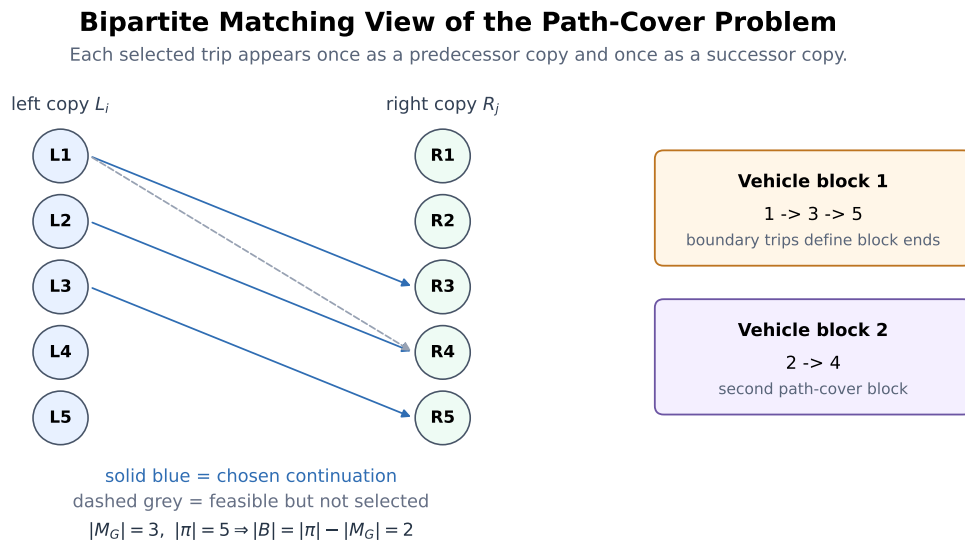


Figure 4.2: Bipartite matching transformation for path cover.

Figure 4.2 visualizes Equations (4.36) to (4.38) and (4.40). The left copy of a trip can be matched to at most one successor, and the right copy can be matched to at most one predecessor. After the matching is computed, the selected arcs are followed to reconstruct vehicle blocks. A trip with no matched predecessor starts a new block. A trip with no matched successor ends a block. This is the reason why the number of blocks is $|\pi| - |M_G|$. The figure therefore links a standard graph algorithm to the operational idea of reducing the number of vehicles. The two copies of the selected timetable do not represent two physical networks; they represent predecessor and successor positions in a vehicle block.

4.5.4 Path-Cover Theorem for the Fixed Compatibility Graph

The matching model above is exact for one fixed selected timetable and one fixed compatibility graph. The following proposition states the graph result used by the implementation.

Proposition 4.1 (Minimum path cover by bipartite matching). *Let $G_\pi = (\pi, A_\pi)$ be the directed compatibility graph of a selected timetable π . Assume that all arcs respect time order, so G_π is acyclic. Let $\mathcal{B}_\pi = (\pi^L, \pi^R, E^B)$ be the bipartite graph with one left and one right copy of every trip and with*

$$(i^L, j^R) \in E^B \iff (i, j) \in A_\pi.$$

If M^ is a maximum-cardinality matching in $\mathcal{B}_\pi$, then the minimum number of vertex-disjoint directed paths covering all trips in π is*

$$K_\pi^* = |\pi| - |M^*|. \quad (4.42)$$

Proof. Every selected matching edge (i^L, j^R) can be interpreted as the predecessor-successor relation $i \rightarrow j$. Because a matching uses each left copy and each right copy at most once, every trip has at most one selected successor and at most one selected predecessor. Since all arcs point forward in time, these selected arcs cannot form a directed cycle. They therefore form a set of directed paths. Starting with $|\pi|$ isolated trips, each selected matching edge joins two paths and reduces the path count by one. A matching of size $|M|$ therefore gives $|\pi| - |M|$ paths. Conversely, any path cover with K paths contains exactly $|\pi| - K$ predecessor-successor links, and these links form a matching in $\mathcal{B}_\pi$. Minimizing K is therefore equivalent to maximizing $|M|$, which proves [Equation \(4.42\)](#).

[Proposition 4.1](#) is important because it explains why matching is the exact subproblem inside the path-cover algorithms. Its scope is limited: it minimizes the number of blocks only for the fixed graph G_π . It does not prove optimality of timetable selection, vehicle-type assignment, charging insertion, capacity use, or the full MINOA objective.

4.5.5 Vehicle-First Weighted Matching

The weighted path-cover variant uses the same feasible edge set but prefers operationally shorter continuations among maximum-cardinality matchings. For an accepted arc (i, j) , the local transition cost is

$$\hat{c}_{ij} = t_{ij}^{dh} + t_{ij}^{wait} + \eta_{dep} t_{ij}^{dep}, \quad (4.43)$$

where t_{ij}^{dh} is deadhead time, t_{ij}^{wait} is non-depot waiting time, t_{ij}^{dep} is depot waiting time, and $0 < \eta_{dep} < 1$ gives depot waiting a smaller local penalty. In the implementation $\eta_{dep} = 0.1$. The matching edge weight is

$$w_{ij} = M^{veh} - \hat{c}_{ij}, \quad M^{veh} = 86400. \quad (4.44)$$

The value M^{veh} is one day in seconds. In addition, the implementation calls the matching routine with a maximum-cardinality requirement. Thus the cardinality priority does not rely only on the numerical size of M^{veh} . The optimization is best read lexicographically:

$$\max |M| \quad \text{first, then} \quad \min \sum_{(i,j) \in M} \hat{c}_{ij}. \quad (4.45)$$

The local edge weight is a construction score. It is not the official MINOA objective. After a full candidate schedule has been reconstructed, the complete cost C^{VS} from Equation (4.19) is computed internally. The MINOA desktop validator is then used as an audit for the final reported input-output pair.

4.6 Electric Fleet, Charging, and Capacity Constraints

4.6.1 Electric Fleet Availability

The number of blocks assigned to each electric vehicle type is bounded by the available fleet:

$$\sum_{b \in B} u_{bv} \leq N_v \quad \forall v \in V_E. \quad (4.46)$$

ICE vehicles are treated as unbounded in the Senior data [12, 13]. In the tested instances, this means the ICE count is obtained after EV assignment as

$$\sum_{b \in B} u_{b,ICE} = |B| - \sum_{b \in B} \sum_{v \in V_E} u_{bv}. \quad (4.47)$$

4.6.2 Battery Propagation

For an EV block b assigned to vehicle type $v \in V_E$, residual autonomy is measured in kilometres. The vehicle leaves the depot fully charged:

$$r_{ba_1}^{pre} = A_v^{max}. \quad (4.48)$$

For a driving activity a , which can be either a passenger trip or a deadhead movement, autonomy decreases by travelled distance:

$$r_{ba}^{post} = r_{ba}^{pre} - \ell_a. \quad (4.49)$$

For a charging activity, autonomy increases by the charged amount:

$$r_{ba}^{post} = r_{ba}^{pre} + \Delta r_{ba}^{charge}. \quad (4.50)$$

The autonomy bounds are

$$0 \leq r_{ba}^{pre}, r_{ba}^{post} \leq A_v^{max} \quad \forall b \in B, a \in A_b, v \in V_E. \quad (4.51)$$

4.6.3 Charging Feasibility

Charging duration must fit into the available break:

$$q_a^{slow} + q_a^{fast} \leq \tau_a \quad \forall a \in A_b^{break}. \quad (4.52)$$

The implementation uses minute-granular charging windows in the JSON output:

$$q_a^m = 60k_a^m, \quad k_a^m \in \mathbb{Z}_{\geq 0}, \quad m \in \{\text{slow}, \text{fast}\}. \quad (4.53)$$

Slow charging cannot exceed the time needed to reach full autonomy:

$$q_a^{slow} \leq T_v^{slow} \frac{A_v^{max} - r_{ba}^{pre}}{A_v^{max}}. \quad (4.54)$$

For fast charging, the full-charge time is $\phi_v T_v^{slow}$:

$$q_a^{fast} \leq \phi_v T_v^{slow} \frac{A_v^{max} - r_{ba}^{pre}}{A_v^{max}}. \quad (4.55)$$

The autonomy gained from charging is

$$\Delta r_{ba}^{charge} = A_v^{max} \left(\frac{q_a^{slow}}{T_v^{slow}} + \frac{q_a^{fast}}{\phi_v T_v^{slow}} \right). \quad (4.56)$$

If charging is used, the minimum charging-time rule is

$$q_a^m = 0 \quad \text{or} \quad q_a^m \geq q_v^{min,m}, \quad m \in \{\text{slow}, \text{fast}\}. \quad (4.57)$$

At most one charging mode is active during one break window. The implementation therefore writes either a slow-charging window, a fast-charging window, or a non-charging parking window [13].

4.6.4 Parking and Charging Capacity

The output distinguishes the occupied resource from active charging. Let

$$o_a^{park}, o_a^{slow}, o_a^{fast}, g_a \in \{0, 1\}. \quad (4.58)$$

For each break window, exactly one resource is occupied:

$$o_a^{park} + o_a^{slow} + o_a^{fast} = 1. \quad (4.59)$$

Active charging is possible only while occupying a charging spot:

$$g_a \leq o_a^{slow} + o_a^{fast}. \quad (4.60)$$

Let $\mathcal{A}_n(t)$ be the set of break windows active at node n at time t . Capacity counts occupied slots, not only active charging:

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{park} \leq K_n^{park}, \quad (4.61)$$

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{slow} \leq K_n^{slow}, \quad (4.62)$$

$$\sum_{a \in \mathcal{A}_n(t)} o_a^{fast} \leq K_n^{fast}. \quad (4.63)$$

The JSON field `typeSpot` records which resource is occupied, while `isCharging` records whether active charging occurs in that window [13]. A vehicle may therefore occupy a charging spot with `isCharging=false` after charging has finished, but the spot still counts against charging-slot capacity.

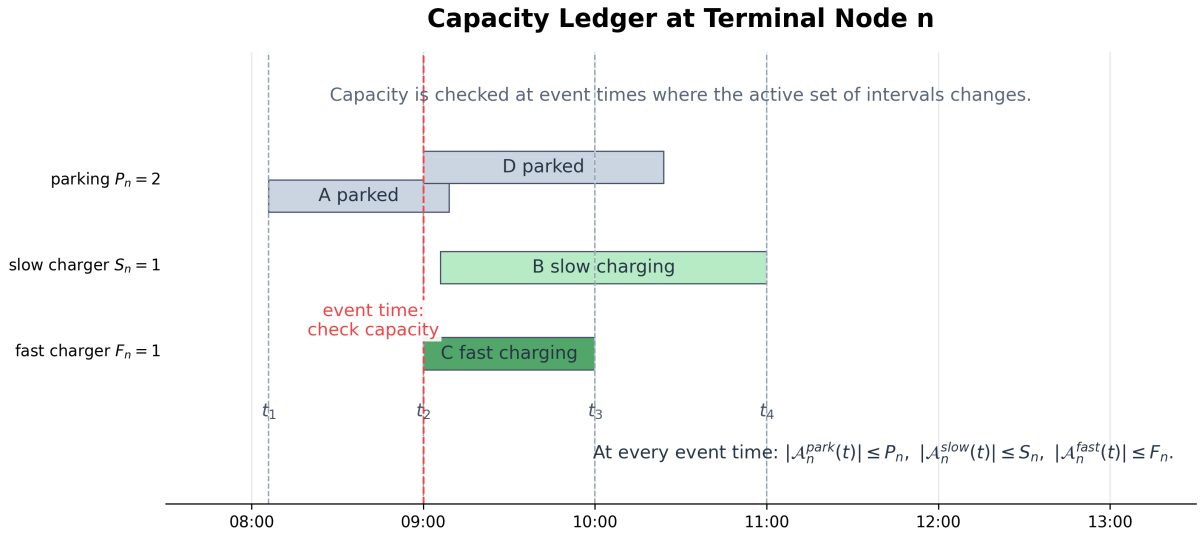


Figure 4.3: Capacity ledger at one terminal node.

Figure 4.3 gives the operational meaning of Equations (4.61) to (4.63). The method does not only check a vehicle at its arrival time. It checks interval overlap. For example, a slow-charging interval uses both time and charger capacity until it ends. If another vehicle tries to use the same charger during an overlapping interval, the schedule is capacity infeasible. The figure also explains why the implementation uses a ledger: each inserted stop reserves a resource over a time interval, and later stops must be checked against this reservation. This distinction is important for EV scheduling because a charging decision is feasible only if the charger is available for the whole occupied interval.

4.7 Paid-Break Accounting and Linearization

4.7.1 Direct Paid-Break Calculation

For a stop activity a , let τ_a be total stop time, $\tau_a^{charge} = q_a^{slow} + q_a^{fast}$ be active charging time, $n(a)$ be the node, and $w(a) = \omega(s_a)$ be the time window selected by the start of the stop. For an in-line stop between two passenger trips, the paid-break time is

$$p_a = \max \left\{ 0, \tau_a - \max \left(\tau_a^{charge}, \delta_{n(a), w(a)}^{min} \right) \right\}. \quad (4.64)$$

For stops adjacent to pull-in or pull-out movements, the official rule is different. Such a terminal stop is permitted only as a charging stop; it does not create positive paid-break time. The model description therefore uses

$$p_a = \begin{cases} \max \{ 0, \tau_a - \max(\tau_a^{charge}, \delta_{n(a), w(a)}^{min}) \}, & \text{if } a \text{ is an in-line stop,} \\ 0, & \text{if } a \text{ is adjacent to pull-in or pull-out.} \end{cases} \quad (4.65)$$

The implementation follows this distinction in the explanatory cost decomposition: `costs.py` counts paid break only for breaks between two passenger trips. The final output is then audited with the MINOA desktop validator.

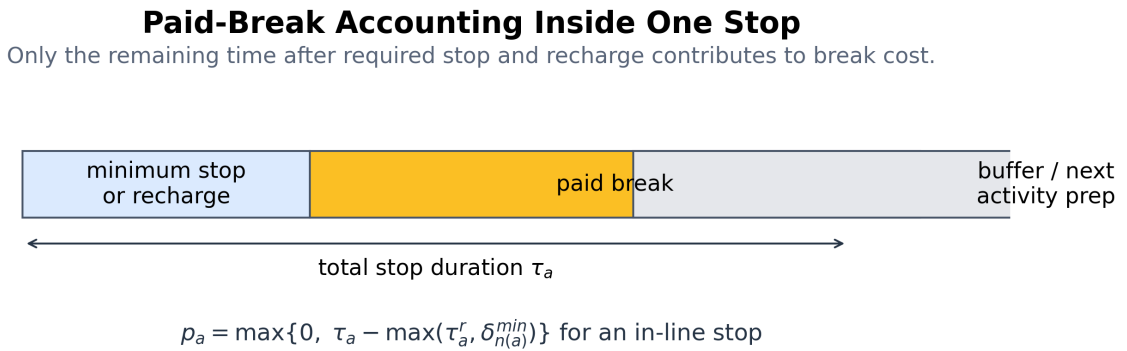


Figure 4.4: Paid-break accounting between two passenger trips.

Figure 4.4 shows how the waiting time between two trips is split. Mandatory stopping time and active charging are not counted as paid break. The remaining orange part is the

cost-relevant break time. This interpretation is later used when result figures separate fixed vehicle cost from smaller operational components. Paid-break cost is therefore not the same as total waiting time: mandatory stopping time and active charging are handled first, and only the remaining waiting time contributes to C^{break} .

4.7.2 Big-M View of the Same Logic

The direct calculation in Equation (4.64) can be written with the auxiliary variable

$$m_a = \max\left(\tau_a^{charge}, \delta_{n(a),w(a)}^{min}\right), \quad (4.66)$$

and

$$p_a = \max\{0, \tau_a - m_a\}. \quad (4.67)$$

With a sufficiently large constant M^{break} and binary variables λ_a, η_a , one linearization is

$$\begin{aligned} m_a &\geq \tau_a^{charge}, & m_a &\geq \delta_{n(a),w(a)}^{min}, \\ m_a &\leq \tau_a^{charge} + M^{break}\lambda_a, & m_a &\leq \delta_{n(a),w(a)}^{min} + M^{break}(1 - \lambda_a), \end{aligned} \quad (4.68)$$

$$\begin{aligned} p_a &\geq \tau_a - m_a, & p_a &\geq 0, \\ p_a &\leq \tau_a - m_a + M^{break}(1 - \eta_a), & p_a &\leq M^{break}\eta_a, \\ \tau_a - m_a &\leq M^{break}\eta_a, & \tau_a - m_a &\geq -M^{break}(1 - \eta_a). \end{aligned} \quad (4.69)$$

The implemented solver does not solve this as a separate MIP. The equations show that the direct accounting is consistent with an exact linearized model once the activity intervals are fixed.

4.8 Timetable Variant Scoring

4.8.1 Sparse Headway Paths

The timetable variant module selects an initial-to-final path in every direction. A path P is first ranked by its number of selected trips. Among paths with equal cardinality, a start-specific tie-breaking score is used:

$$P_h^d = \arg \min_{P \in \mathcal{P}^d} (|P|, Q_h(P)). \quad (4.70)$$

The score is

$$Q_h(P) = \sum_{(i,j) \in P} g_{ij}^h + g_{start}^h(i_0) + g_{end}^h(i_q), \quad (4.71)$$

where i_0 and i_q are the first and last selected trips of the path. Equivalently, the generic timetable-variant problem is

$$P_h^d = \arg \min_P \left(|P|, \sum_{(i,j) \in P} g_{ij}^h \right). \quad (4.72)$$

4.8.2 Examples of Variant Scores

The implementation uses several forms of g_{ij}^h :

$$g_{ij}^h \in \left\{ -a(j), a(j), a(j) - a(i), -(a(j) - a(i)), \left| (a(j) - a(i)) - \lambda_h I_{ij}^{max} \right| \right\}. \quad (4.73)$$

These scores generate different but feasible timetables. The purpose is not to add a passenger penalty to the reported objective. The purpose is to create several feasible graph structures for the multi-start path-cover method.

4.9 Model Size and Complexity Discussion

4.9.1 Graph Size

Let $n = |\pi|$ be the number of selected trips. In the worst case, every earlier trip can be tested against every later trip:

$$|P_\pi^{time}| \leq \frac{n(n-1)}{2}. \quad (4.74)$$

The compatibility graph is smaller because only feasible in-line or depot-bridge pairs become arcs:

$$|A_\pi| \leq |P_\pi^{time}|. \quad (4.75)$$

This explains why the method first selects a sparse timetable. A smaller π reduces the graph size and therefore also the matching problem.

4.9.2 Matching Problem Size

The bipartite graph contains two copies of the selected timetable:

$$|\pi^L \cup \pi^R| = 2|\pi|. \quad (4.76)$$

Its edge set has one edge for each compatibility arc:

$$|E^B| = |A_\pi|. \quad (4.77)$$

Thus, the core optimization problem is controlled by the number of selected trips and feasible continuations, not by the full candidate pool T . In practice, this reduces the graph size before matching is solved and keeps the path-cover subproblem focused on trips that are actually present in the selected timetable.

4.9.3 Why the Matheuristic is Practical

The exact integrated problem would have to decide timetable, vehicle blocks, fleet type, charging, battery states, and capacity at the same time. The implemented method separates this into smaller linked subproblems:

$$\begin{aligned} &\text{headway path + compatibility graph} \\ &+ \text{matching path cover + EV feasibility simulation.} \end{aligned} \tag{4.78}$$

Equation (4.78) explains why the later algorithm is a matheuristic. The method does not prove global optimality, but it keeps the main vehicle-count structure in the matching step. EV charging and capacity are then checked on complete vehicle blocks, where their timing can be evaluated directly.

4.10 Exact Subproblems and Heuristic Coupling

The formulation above contains two different levels of optimization. One level is exact after the timetable and compatibility graph have already been fixed. The other level is heuristic because the algorithm must still choose which timetable variant, edge score, fleet assignment, and charging insertion should be retained. Keeping these two levels separate is important for interpreting the numerical results. The matching step is solved as a well-defined graph optimization problem, but the complete integrated MINOA problem is not solved to global optimality.

4.10.1 Fixed-Graph Path-Cover Layer

For one selected timetable π , the compatibility graph $G_\pi = (\pi, A_\pi)$ is acyclic because every arc moves forward in time. A path cover of this graph is a set of directed paths that covers each selected trip exactly once. If $z_{ij} = 1$ means that trip j is served directly after trip i , the fixed-graph path-cover problem can be written as

$$\max \quad \sum_{(i,j) \in A_\pi} w_{ij} z_{ij} \quad (4.79)$$

$$\text{s.t.} \quad \sum_{j:(i,j) \in A_\pi} z_{ij} \leq 1 \quad \forall i \in \pi, \quad (4.80)$$

$$\sum_{i:(i,j) \in A_\pi} z_{ij} \leq 1 \quad \forall j \in \pi, \quad (4.81)$$

$$z_{ij} \in \{0, 1\} \quad \forall (i, j) \in A_\pi. \quad (4.82)$$

Equation (4.80) allows each trip to have at most one successor, and Equation (4.81) allows each trip to have at most one predecessor. These two restrictions are exactly the rules needed to obtain vehicle blocks from a matching. Trips with no predecessor start a block, trips with no successor end a block, and isolated trips become one-trip blocks. The number of blocks follows directly from the number of chosen continuations:

$$|B_\pi(z)| = |\pi| - \sum_{(i,j) \in A_\pi} z_{ij}. \quad (4.83)$$

Equation (4.83) is the main reason why matching is attractive in this thesis. Every accepted continuation reduces the vehicle count by one, provided that the two trips can be linked in time and space. In the unweighted version, all feasible continuations are treated equally. In the weighted version, the score w_{ij} favors continuations that are also promising with respect to waiting time, deadhead effort, EV feasibility, and charging pressure. This score is a construction score. It is not the official objective C^{VS} , which is evaluated only after complete blocks and vehicle types are known.

The fixed-graph layer can also be written as a bipartite matching problem. The left copy π^L represents possible predecessors and the right copy π^R represents possible successors. For every compatibility arc $(i, j) \in A_\pi$, the bipartite graph contains the edge (i^L, j^R) . A matching then selects a set of trip-to-trip continuations without assigning two successors to the same trip or two predecessors to the same trip. This is the computational form used by the implementation.

4.10.2 Integrated Candidate Layer

The complete solution is not determined by one fixed graph. It is selected from a family of candidate graphs, edge scores, and fleet decisions. Let $\mathcal{H}$ denote the set of timetable-variant

generators and let Θ_h denote the bounded construction settings used for variant h . For each pair (h, θ) , the implementation constructs a complete candidate solution

$$S_{h,\theta} = (\pi_h, G_{\pi_h}, z_{h,\theta}, B_{h,\theta}, u_{h,\theta}, q_{h,\theta}), \quad (4.84)$$

where π_h is the selected timetable, G_{π_h} is the compatibility graph, $z_{h,\theta}$ is the chosen matching, $B_{h,\theta}$ is the block set, $u_{h,\theta}$ assigns vehicle types, and $q_{h,\theta}$ stores the charging plan. Only after all of these objects are available can the official cost be computed:

$$C^{VS}(S_{h,\theta}) = C^{fix}(S_{h,\theta}) + C^{break}(S_{h,\theta}) + C^{pull}(S_{h,\theta}) + C^{CO_2}(S_{h,\theta}). \quad (4.85)$$

The retained incumbent is therefore selected by

$$S^* \in \arg \min \{C^{VS}(S_{h,\theta}) : h \in \mathcal{H}, \theta \in \Theta_h, S_{h,\theta} \in \mathcal{F}\}, \quad (4.86)$$

where $\mathcal{F}$ is the set of internally feasible schedules. A schedule belongs to $\mathcal{F}$ only if it satisfies timetable feasibility, vehicle-block feasibility, EV autonomy, charging-duration rules, and parking or charging capacity restrictions. The MINOA desktop validator is then used as an independent audit for the final output, not as part of [Equation \(4.86\)](#).

This separation also explains why charging-aware edge scoring is useful but limited. An edge score can estimate whether a continuation is likely to create a good block, but battery feasibility is a property of the whole chronological block. A continuation that looks attractive locally may become unattractive after several later trips are added. Conversely, a slightly more expensive continuation may create a block with better charging opportunities. For this reason, the final method combines local edge scores with full block-level simulation before a candidate can enter $\mathcal{F}$.

4.10.3 Bound Interpretation

Every feasible candidate has an official cost and therefore gives an upper bound on the unknown optimum:

$$C^* \leq C^{VS}(S_{h,\theta}) \quad \text{for every feasible } S_{h,\theta}. \quad (4.87)$$

Lower bounds have a different meaning. A globally valid lower bound LB must satisfy

$$LB \leq C^* \leq UB, \tag{4.88}$$

where UB is the best feasible cost found. Global optimality would be certified only if $LB = UB$. In contrast, a selected-timetable diagnostic bound can be useful for analysis but is conditional on the selected timetable π . It therefore does not prove global optimality for the complete integrated problem. This distinction is used again in the computational results, where the reported gaps are interpreted separately for global bounds and selected-timetable diagnostic bounds.

Chapter 5

Solution Methodology

The methodology starts from a practical construction idea. I first build a timetable that satisfies the service rules, and then I turn the selected trips into vehicle blocks with the help of a compatibility graph. In this graph, a node represents a selected trip and a directed arc means that the same vehicle can serve the second trip after the first one. This gives a compact view of the vehicle-scheduling part, because each vehicle block can be read as a path in the graph.

The four implemented algorithms should be understood as a development sequence, not as four unrelated alternatives. The greedy constructive heuristic gives a simple and transparent baseline. The unweighted path-cover method then replaces local successor choices by a global matching step. The weighted path-cover method keeps this graph structure but adds operational information to the arcs. Finally, the multi-start weighted path-cover matheuristic tests several timetable and graph variants and keeps the best validated candidate in a no-regression archive. The full method remains heuristic for the integrated MINOA problem, but for each fixed selected timetable and fixed compatibility graph, the matching subproblem is solved exactly.

5.1 Shared Construction Layer

The shared construction layer is used by all four methods. This avoids mixing algorithmic differences with different interpretations of the benchmark data. Let T be the set of candidate trips. For each direction $d \in D$, the timetable step selects an ordered subset $\pi_d \subseteq T_d$. The complete selected timetable is

$$\pi = \bigcup_{d \in D} \pi_d. \quad (5.1)$$

The first and last required trips are fixed. Between these anchor trips, the algorithm selects candidate trips so that the headway windows from [Chapter 3](#) are respected. During implementation, it became clear that one deterministic timetable rule can be too narrow. Therefore, a multi-start run generates several feasible timetable variants π^h , $h \in H$, by changing the local selection rule and the tie-breaking order.

For a fixed timetable π^h , the algorithm builds a directed compatibility graph

$$G^h = (\pi^h, A^h), \quad A^h = A^{h,\text{in}} \cup A^{h,\text{bridge}}. \quad (5.2)$$

Each node is one selected trip. An arc $(i, j) \in A^h$ is added only when one vehicle can operate trip j after trip i . In-line continuation edges represent a same-terminal layover. Depot-bridge continuation edges represent a connection through the depot, including pull-in, possible depot dwell time, and pull-out. This distinction is important because both edge types can be feasible, but they have different operational consequences.

For every candidate arc, the implementation stores the travel and resource attributes needed later by the matching and EV-feasibility steps:

$$a_{ij} = (\tau_{ij}^{\text{wait}}, \tau_{ij}^{\text{dead}}, \delta_{ij}^{\text{min}}, \delta_{ij}^{\text{max}}, \kappa_{ij}^{\text{park}}, \kappa_{ij}^{\text{chg}}). \quad (5.3)$$

Here τ_{ij}^{wait} is the waiting time, τ_{ij}^{dead} is the deadhead duration, δ_{ij}^{min} and δ_{ij}^{max} are the minimum and maximum feasible stopping times, and $\kappa_{ij}^{\text{park}}, \kappa_{ij}^{\text{chg}}$ describe the parking and charging resources that may be needed during the layover.

The shared layer keeps the comparison fair. All four methods use the same parsed trip set, headway windows, fleet limits, deadhead data, and cost coefficients. Differences in the computational results therefore come from the construction logic, not from different readings of the input files. This matters in MINOA because the same terminal can appear in several roles. It can be part of a timetable rule, a charging-capacity rule, and a cost calculation at the same time, while each rule still affects a different part of the schedule.

The construction layer also separates passenger-service feasibility from vehicle-scheduling feasibility. A timetable can satisfy the headway rules and still be difficult to operate with few vehicles. For example, two trips may be well spaced for passengers but may end and start at terminals that require a depot-bridge movement. The opposite can also occur: a timetable may be easy to chain but may contain too few trips for the required service pattern. In practice, the pipeline therefore first creates timetable-feasible candidate sets and only then evaluates vehicle reuse through the compatibility graph.

For a consecutive pair of selected trips i, j in direction d , the construction checks the maximum main-stop headway condition

$$0 < a(j) - a(i) \leq I_{ij}^{\text{max}}. \quad (5.4)$$

Here $a(i)$ and $a(j)$ are the main-stop arrival times of the two trips, and $I_{ij}^{\max}$ is determined using the time-window border rule defined in [Section 3.3.2](#). The first and final required trips are preserved as anchors. Between these anchors, the variant generator changes the density of selected trips and the tie-breaking order. This produces sparse and dense timetable alternatives without changing the benchmark data.

The multi-start candidate family is the set of complete candidates that survive these internal construction checks. It can be written as

$$\mathcal{S} = \{S(h, \theta) : h \in H, \theta \in \Theta_h, S(h, \theta) \text{ is internally feasible}\}, \quad (5.5)$$

where h identifies the timetable start. Here Θ_h collects the bounded construction settings used in the implementation, including graph-scoring mode, depot-bridge treatment, EV assignment strategy, and local repair option where enabled. The set is much smaller than all possible integrated schedules, but it is broad enough to avoid relying on one single deterministic construction.

5.2 Vehicle Blocks as a Path Cover

The vehicle-scheduling part is treated as a path-cover problem. A path is an ordered sequence of trips served by one vehicle. If a trip has no predecessor in the chosen arc set, it starts a new block from the depot. If it has no successor, the block returns to the depot after that trip. Unmatched trips are therefore not lost; they become single-trip blocks or boundary trips of longer blocks.

The graph is split into a left and a right copy of the trip set. Selecting an arc (i, j) in the matching means that trip j follows trip i in the same block. The matching variables satisfy

$$\sum_{j:(i,j) \in A^h} z_{ij} \leq 1 \quad \forall i \in \pi^h, \quad (5.6)$$

$$\sum_{i:(i,j) \in A^h} z_{ij} \leq 1 \quad \forall j \in \pi^h, \quad (5.7)$$

$$z_{ij} \in \{0, 1\} \quad \forall (i, j) \in A^h. \quad (5.8)$$

These constraints give the matching its vehicle-scheduling meaning. Each trip can have at most one successor and at most one predecessor, so the selected links form paths rather

than branching structures. These paths are then read chronologically to obtain the vehicle blocks.

For the unweighted path-cover variant, the objective maximizes the number of selected continuation arcs:

$$\max \sum_{(i,j) \in A^h} z_{ij}. \quad (5.9)$$

This objective mainly tries to reduce the number of vehicle blocks, because every chosen arc joins two trips inside the same block. The weighted variant uses a construction score w_{ij} . A generic form of the construction score is

$$w_{ij} = \alpha_0 - \alpha_1 \tau_{ij}^{wait} - \alpha_2 \tau_{ij}^{dead} - \alpha_3 \rho_{ij}^{chg} + \alpha_4 s_{ij}^{CO_2}. \quad (5.10)$$

The term ρ_{ij}^{chg} is a bounded charging-risk indicator, and $s_{ij}^{CO_2}$ rewards continuations that are more likely to be operated by an EV. The score is only a guide for constructing blocks. It is a surrogate matching score, not the official MINOA objective C^{VS} . The official objective is computed separately after complete blocks and vehicle types have been assigned and is audited later.

The matching output is converted into vehicle blocks by predecessor and successor maps:

$$succ(i) = j \quad \text{if } z_{ij} = 1, \quad (5.11)$$

$$pred(j) = i \quad \text{if } z_{ij} = 1. \quad (5.12)$$

Every trip without a predecessor starts a path. The implementation then follows $succ(\cdot)$ until no successor exists. Because the matching constraints allow at most one predecessor and at most one successor, the resulting paths cannot merge or split. Each path is written as one vehicle block.

Trips that remain unmatched are still covered. If a trip has neither a predecessor nor a successor, it becomes a single-trip block. Such a block is expensive because it requires one pull-out and one pull-in movement for little service work, but it is valid. The path-cover objective tries to avoid this situation by selecting compatible continuations whenever they are available.

The block count for a fixed selected timetable follows directly from the number of selected continuation arcs:

$$|B| = |\pi^h| - \sum_{(i,j) \in A^h} z_{ij}. \quad (5.13)$$

This equation is the main reason for using path cover in this thesis. Every selected arc joins two trips into the same vehicle block and removes one potential block start. A larger matching therefore usually means fewer vehicle blocks for the fixed selected timetable.

Network Logic of the First Two Implemented Approaches

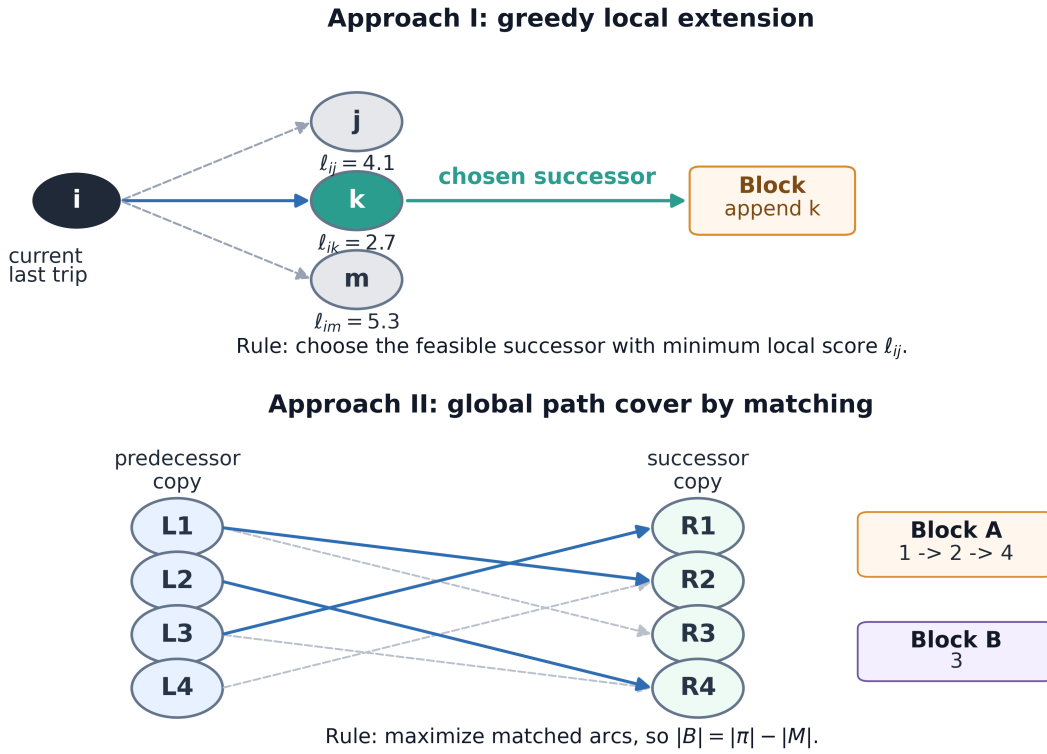


Figure 5.1: Network interpretation of local and graph-based construction.

Figure 5.1 compares the local greedy construction with the graph-based path-cover idea. The greedy method extends one block at a time, while the path-cover methods choose several continuations together. A useful way to read the figure is to focus on the consequence of one successor choice: a locally good continuation can later block a better global chain. Matching reduces this risk by selecting a consistent set of continuations over the whole selected timetable.

5.3 Four Implemented Algorithms

Figure 5.2 summarizes the four implemented algorithms in the same visual language. The figure should be read as a development path. Each method keeps the same selected-trip and compatibility-graph foundation, while adding one stronger decision layer: local successor

choice, global cardinality matching, operational edge scoring, and multi-start archive selection.

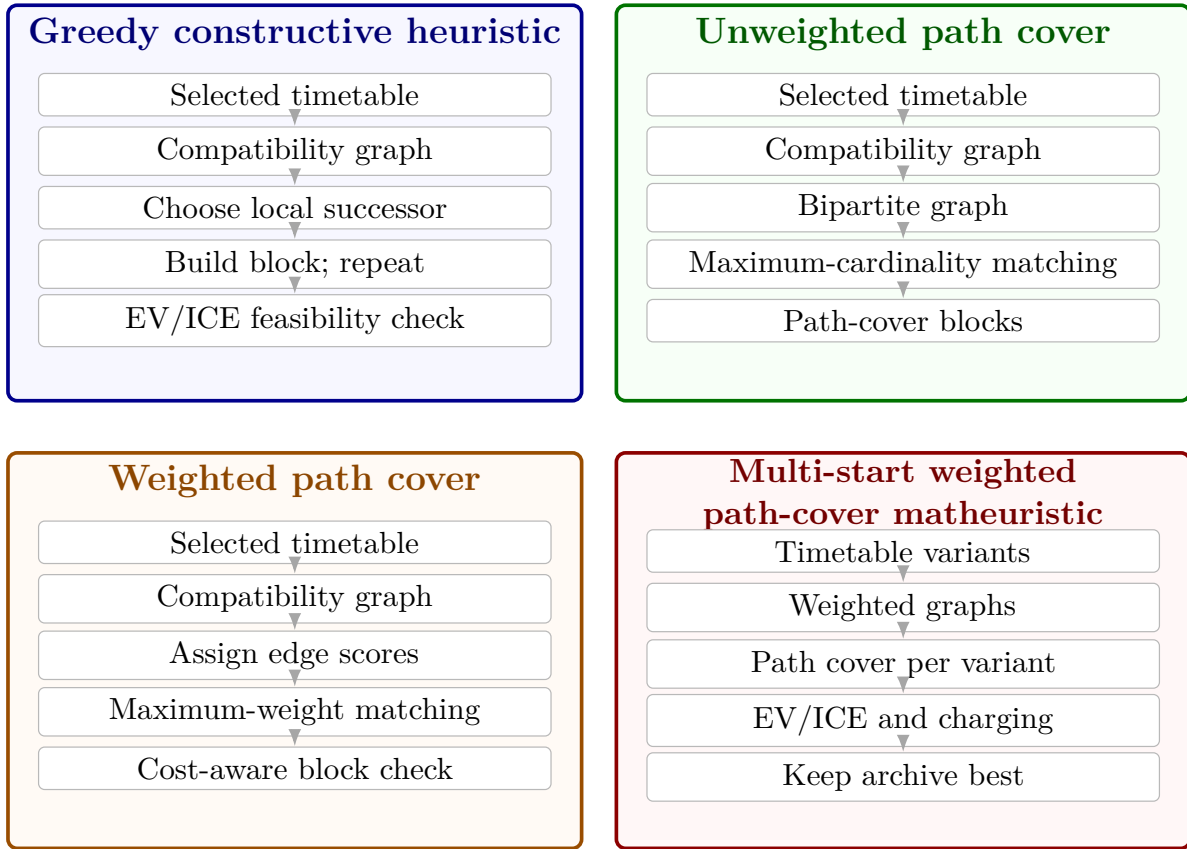


Figure 5.2: Workflow comparison of the four implemented algorithms.

The figure makes the development of the methodology visible. Greedy construction makes one local successor decision after another. Unweighted path cover replaces these local choices by one global matching step for a fixed graph. Weighted path cover keeps the same matching structure but adds operational edge scores. The final multi-start method repeats this construction over several timetable and graph variants and keeps the best validated output in the no-regression archive. This progression explains why the multi-start method is used for the main benchmark results.

5.3.1 Greedy Constructive Heuristic

The greedy constructive heuristic is the first implemented method. I use it as a transparent baseline before introducing graph matching. Its role is not to be the strongest final solver. It answers a simpler question: what kind of solution is obtained when vehicle blocks are built chronologically, one successor at a time? This baseline is useful because MINOA

combines several rules that interact with each other. Before adding a global matching step, the greedy method makes it easier to check that timetable selection, trip coverage, pull-in/out movements, and basic EV feasibility are interpreted correctly.

The input of the greedy method is one selected timetable π^h , the compatibility graph $G^h = (\pi^h, A^h)$, the deadhead data, the fleet limits, and the cost coefficients. The output is a complete vehicle schedule: a list of blocks B , an ICE/EV assignment u , and charging information where EV operation is feasible. The method keeps a set $U \subseteq \pi^h$ of uncovered trips. At the beginning, $U = \pi^h$. Whenever a trip is inserted into a vehicle block, it is removed from U . The algorithm stops only when $U = \emptyset$, so every selected trip is covered exactly once.

The construction begins a new vehicle block with the earliest uncovered trip. This rule is simple, but it is a practical way to avoid leaving early trips behind while later blocks are already being built. After the first trip i has been chosen, the algorithm searches all uncovered successors

$$\Gamma^+(i) = \{j \in U : (i, j) \in A^h\}.$$

Each candidate successor already satisfies the compatibility rules from [Chapter 4](#). The greedy step therefore does not rederive the headway or block-feasibility conditions. It chooses among continuations that are already feasible in time and space.

The greedy construction is a repeated block-building process. At each iteration, there is one open block and one current last trip. The method does not compare complete schedules. It only asks which uncovered trip can be appended next without breaking compatibility. This makes the decision easy to inspect. The selected successor must start late enough for the vehicle to move from the end of the current trip to the start of the next trip, including the required stopping time. If both trips meet at the same terminal, the continuation is in-line. If the vehicle must return to the depot between the two trips, the continuation is a depot-bridge continuation and must contain enough time for pull-in, depot dwell, and pull-out movement.

The local successor rule is

$$j^* = \arg \min_{j: (i,j) \in A^h} \left(\lambda_1 \tau_{ij}^{wait} + \lambda_2 \tau_{ij}^{dead} + \lambda_3 I_{ij}^{bridge} \right). \quad (5.14)$$

The indicator I_{ij}^{bridge} is one for depot-bridge continuations. The rule prefers short waits, low deadhead time, and in-line continuations. It is fast and easy to explain, but it also

shows the weakness of a local method: an early successor choice may prevent better vehicle reuse later.

If $\Gamma^+(i) \neq \emptyset$, the selected successor j^* is appended to the current block, removed from U , and becomes the current trip. The same search is repeated from j^* . If $\Gamma^+(i) = \emptyset$, the current block is closed. Closing a block means adding the necessary pull-in movement from the end terminal of the last trip back to the depot. When a new block starts, the corresponding pull-out movement from the depot to the first trip is added. In words, if a trip ends at a terminal and the next uncovered trip can start after a feasible layover at the same terminal, the greedy method can continue in-line. If this is not possible, it may use a depot-bridge continuation when the time is sufficient. If neither continuation is possible, the vehicle returns to the depot and a new block is opened.

The output of this phase is a set of chronological paths through the compatibility graph. Each path is one vehicle block. A block contains a pull-out movement, one or more passenger-service trips, possible in-line or depot-bridge continuations, and a final pull-in movement. The algorithm does not need a separate coverage repair at the end because a trip is removed from U only when it has been inserted into exactly one block. Trips that cannot be appended to the current block are not discarded; they remain in U and later become possible starts of new blocks.

After the chronological blocks are created, the same EV-assignment and charging feasibility logic used by the other methods is applied. Each block is simulated in chronological order. If the block can be operated by an EV without negative autonomy and without violating parking or charging capacity, an EV assignment is allowed. Otherwise, the block remains ICE, provided that conventional vehicle-scheduling feasibility is satisfied. The greedy method therefore does not ignore electric-vehicle restrictions; it simply constructs the block sequence locally before testing EV feasibility.

The main strength of the greedy method is speed and transparency. Every decision can be explained by one predecessor trip and one selected successor. This makes the method useful for debugging the data interpretation and for creating a reference point in the experiments. Its limitation is equally clear. Once a successor is selected, the choice is not reconsidered. A locally short wait can consume a trip that would have been a better successor for another block. The method compares only the next feasible successor of the current trip, not complete path covers. The next method addresses this weakness by choosing continuations jointly through matching.

The greedy heuristic also helped clarify which parts of the MINOA problem are local and which parts need a wider view. Time compatibility can be checked arc by arc, and trip coverage can be maintained with the uncovered set U . However, vehicle count, paid-break time, and charging opportunities depend on the full collection of blocks. A decision that looks good for one block can make another block longer or even force an additional vehicle. This observation motivates the move to a path-cover formulation.

5.3.2 Unweighted Path Cover

The unweighted path-cover method is the second implemented algorithm. It keeps the same timetable-generation layer as the greedy heuristic, but it replaces local block extension with a maximum-cardinality matching problem. The motivation is practical: a good local successor is not always good for the whole timetable. The path-cover method looks at all feasible trip-to-trip continuations in the selected timetable and chooses a compatible set of continuations jointly.

The input is again one selected timetable π^h and its compatibility graph $G^h = (\pi^h, A^h)$. Each selected trip is represented as one node in the directed graph. A directed arc $(i, j) \in A^h$ means that trip j can be served directly after trip i by the same vehicle. The method then builds a bipartite graph with two copies of the selected trip set. The left copy $(\pi^h)^L$ represents predecessor positions, and the right copy $(\pi^h)^R$ represents successor positions. For each compatibility arc $(i, j) \in A^h$, the bipartite graph contains the matching edge (i^L, j^R) .

This transformation enforces the vehicle-block logic through matching constraints. A trip can be matched to at most one successor, and a trip can have at most one predecessor. Therefore, the selected matching edges can be converted back into directed paths. A trip with no matched predecessor starts a block. A trip with no matched successor ends a block. A trip with neither a matched predecessor nor a matched successor becomes a one-trip block. No trip is lost, and no selected trip is covered twice.

The bipartite construction is useful because it separates the two roles that a trip can have inside a block. In the left copy, trip i asks for one successor. In the right copy, trip j asks for one predecessor. A matching edge (i^L, j^R) therefore means that the vehicle continues from i to j . Since a matching cannot use two edges incident to the same copied node, the resulting successor map cannot branch. This is exactly the structure needed for vehicle blocks: a vehicle block is a path, not a tree.

The matching objective is the unweighted maximum-cardinality objective from [Equation \(5.9\)](#). It selects as many compatible continuations as possible. Since every accepted continuation joins two trips inside the same vehicle block, maximizing the number of selected continuations tends to minimize the number of blocks for the fixed graph. The matching step is exact for this fixed path-cover subproblem. This statement is deliberately limited: it does not mean that the complete MINOA problem is solved exactly. Timetable generation, fleet assignment, charging insertion, and final candidate selection remain heuristic parts of the overall method.

For a selected timetable with $n_h = |\pi^h|$ trips, the relation between matching size and vehicle count is simple:

$$|B^h| = n_h - |M^h|, \quad (5.15)$$

where M^h is the set of selected matching edges. Each matching edge joins two trips that would otherwise require two separate path segments. Therefore, one additional continuation reduces the number of path starts by one. This equation also explains why the unweighted method is stronger than the greedy method for a fixed graph. It optimizes the number of continuations jointly instead of accepting the first locally attractive successor.

After matching, the algorithm reconstructs blocks by following the selected successor map. For each trip without a predecessor, a new block is opened. The algorithm follows the matched successor until no successor remains, closes the block with a pull-in movement, and then continues with the next unmatched block start. Once the blocks are built, the shared EV/ICE assignment and charging feasibility procedure is applied. Thus, the unweighted path-cover method improves the vehicle-block construction step, but it does not replace the later mixed-fleet feasibility checks.

The reconstruction step also handles boundary cases explicitly. If a trip has no predecessor but has a successor, it becomes the first trip of a multi-trip block. If a trip has a predecessor but no successor, it becomes the last trip of that block. If it has neither, the algorithm creates a single-trip block with one pull-out and one pull-in movement. These cases are important in sparse parts of the timetable, where no feasible continuation exists because the next trip starts too early, starts at a different terminal without enough depot time, or belongs to a different part of the service day.

The main strength of unweighted path cover is that it is more systematic than greedy construction. It chooses continuations jointly, so it can avoid some local traps. It also gives a clear mathematical link between matching size and vehicle count. The limitation is that every feasible arc has the same value. A short in-line layover and a long depot-bridge

continuation both count as one continuation. As a result, the method may reduce the number of blocks while creating long waiting times, high pull-in/out cost, or weak charging opportunities. This motivates the weighted path-cover method, which keeps the matching structure but adds operational preferences to the arcs.

Thus, the unweighted method is mainly a vehicle-count method. This is valuable because fixed vehicle cost is an important part of C^{VS} , but it is not yet a cost-aware scheduler. In the numerical experiments it provides a clean comparison point. Improvements over greedy indicate the value of global block chaining. Improvements from the weighted method then indicate the value of choosing better continuations among block covers with similar cardinality.

5.3.3 Weighted Path Cover

The weighted path-cover method is the third implemented algorithm. It was introduced because vehicle count alone does not fully explain the MINOA objective. The official objective C^{VS} contains fixed vehicle cost, paid-break cost, pull-in/out cost, and a CO₂ term for the relevant driving duration. Two schedules can therefore use the same number of vehicles but have different objective values. One may create long paid breaks or many depot-heavy transitions, while another may use the same number of blocks more compactly.

The weighted method keeps the same path-cover and matching structure as the unweighted method. The selected trips are still graph nodes, feasible continuations are still arcs, and the bipartite transformation is still used. The difference is that each arc receives a score w_{ij} , as shown in [Equation \(5.10\)](#). The matching problem then becomes a maximum-weight matching with maximum cardinality. In the implementation this is solved with the NetworkX maximum-weight matching routine described in [Chapter 6](#). The maximum-cardinality setting keeps vehicle reduction important, while the weights guide the choice among alternative continuations.

The score rewards compact and operationally attractive continuations. Short waiting time is preferred because long waiting may become paid break after the mandatory stopping time and active charging time have been accounted for. Low deadhead duration is preferred because deadhead movement consumes time and can make later connections harder. In-line continuations are often preferred over depot-bridge continuations when both are feasible. A depot-bridge movement can increase pull-in/out movement and reduce the time available

for later service. Continuations that preserve charging opportunities or avoid high battery pressure receive a better score than continuations that create a tight EV block.

The implemented weighted variant expands this generic construction score as

$$w_{ij} = \alpha_0 - \alpha_1 \tau_{ij}^{wait} - \alpha_2 \tau_{ij}^{dead} - \alpha_3 I_{ij}^{bridge} - \alpha_4 r_{ij}^{break} + \alpha_5 o_{ij}^{charge} + \alpha_6 s_{ij}^{CO_2}. \quad (5.16)$$

Here $\alpha_0, \dots, \alpha_6$ are tuning weights used only for the construction score. The term I_{ij}^{bridge} marks a depot-bridge continuation, r_{ij}^{break} estimates paid-break risk, and o_{ij}^{charge} estimates whether the layover provides a useful charging opportunity. This expression is not a replacement for the official objective. It is a local guide that helps the matching routine choose among compatible continuations. The coefficients are fixed construction-score parameters used by the implemented candidate families. They are not part of the official objective and are not interpreted as optimal dual weights. The concrete command settings and candidate-family configurations used for the reported experiments are documented in the reproducibility and experiment sections.

The score also contains a CO₂-related preference through $s_{ij}^{CO_2}$. This term does not compute the final emission cost of a complete block. It only estimates whether a continuation is likely to support an EV-feasible block later. The complete objective is still evaluated after vehicle blocks, vehicle types, and charging intervals have been constructed. This distinction is important. The matching score is a surrogate construction score, whereas C^{VS} is the official reported objective.

After the maximum-weight matching is solved, the reconstruction step is the same as in the unweighted method. Matched edges become successor links. Trips without predecessors start blocks, and trips without successors end blocks. Then the shared EV/ICE assignment and charging-feasibility procedure is applied. The weighted method can therefore improve cost even when the vehicle count does not change. If two matchings use the same number of continuations, the weighted matching can prefer the one with shorter waits, better pull structure, lower charging pressure, or better EV potential.

This method is especially useful when the selected graph contains many alternative continuations. In such cases, the unweighted matching may find several maximum-cardinality solutions with the same number of vehicles. The weighted method uses the score to choose among them. In practice, the algorithm can keep the same vehicle count but still reduce waiting, pull-in/out movement, or later EV infeasibility. It also creates a clearer link between graph optimization and the cost decomposition in [Chapter 4](#).

The main strength of weighted path cover is that it connects vehicle-count reduction with the cost structure of MINOA. It keeps an exact matching subproblem for the fixed graph, but it gives the graph solver more operational information. The main limitation is that edge weights are still local approximations. They cannot fully predict future battery states, charger capacity conflicts, or the effect of a continuation several trips later in the same block. The final method responds to this limitation by testing several timetable and graph variants instead of trusting one weighted graph.

The limitation of local scoring is most visible in battery and charging decisions. An edge may look attractive because it contains a useful layover, but the same layover may later be unavailable because another EV already uses the charger. Another edge may look unattractive locally but may help create a compact block that can be operated by ICE at lower total cost. These effects depend on the whole schedule, not only on one edge. The final method therefore uses weighted matching as a strong construction step, but it surrounds it with multiple starts and whole-solution evaluation.

5.3.4 Multi-Start Weighted Path-Cover Matheuristic

The multi-start weighted path-cover matheuristic is the final selected method. It extends the weighted path-cover method by no longer relying on one timetable and one compatibility graph. This is necessary because timetable selection changes the vehicle-scheduling problem itself. If one trip is selected instead of another, the set of feasible continuations may change, the number of possible EV blocks may change, and the amount of waiting or deadhead movement may change. A good graph for vehicle scheduling therefore depends on the timetable variant from which it is built.

Each start produces one complete candidate. First, a feasible timetable variant π^h is generated from the candidate trips while preserving the first-trip, final-trip, and headway requirements. Then the compatibility graph $G^h = (\pi^h, A^h)$ is built and the arc weights are assigned. A weighted path-cover matching is solved for this fixed graph, and the matched arcs are reconstructed into vehicle blocks. Finally, EV/ICE assignment and charging feasibility are checked on the complete blocks.

The input of one start is a timetable-variant rule, a graph-construction rule, and a set of scoring parameters. The output is either a complete internally feasible solution or a rejected candidate. Rejection can occur when the selected timetable cannot be converted into a feasible vehicle schedule under the internal checks, when the mixed-fleet assignment exceeds the available fleet, or when charging and terminal-capacity checks fail. This does

not mean that the instance is infeasible. It only means that this particular start did not produce a feasible complete schedule.

Changing the timetable can create better block possibilities. For example, two candidate departures may both satisfy the passenger headway requirements, but only one of them may allow a short in-line continuation for a vehicle. A single deterministic timetable rule can miss this alternative. The multi-start method therefore searches several structured alternatives instead of only one construction. The search is still bounded: it does not enumerate all possible timetables and does not solve a full integrated MIP. It explores a controlled candidate family that can be reproduced.

The timetable variants are generated before the vehicle blocks are constructed. This order is important. A selected trip set defines the nodes of the compatibility graph, and therefore it also defines which matching problem will be solved. If a variant removes a candidate trip or chooses a different departure within the same headway window, the graph may gain or lose arcs. A small timetable change can therefore reduce one vehicle block, create a better charging stop, or remove an expensive depot-bridge continuation.

During EV/ICE assignment, the method simulates each reconstructed block chronologically. EV assignment is attempted when the block has a feasible battery trajectory and enough charging resources. Charging is inserted or checked only during feasible stop intervals at suitable nodes. If the EV simulation fails, the block can remain ICE as long as the conventional schedule is feasible. If a candidate violates an internal feasibility rule, it is not accepted into the internally feasible candidate set.

The EV/ICE decision is made on complete blocks rather than on isolated edges. This is necessary because battery feasibility is cumulative. A block may start with enough autonomy for the first trips but become infeasible later if no suitable charging interval is available. The assignment procedure therefore checks the whole block, the charging opportunities along it, and the terminal capacity used by other blocks. Blocks that are not EV-feasible can still be part of a valid mixed-fleet schedule as ICE blocks.

Candidates are first evaluated using internal feasibility checks and the internally computed official objective C^{VS} . The MINOA desktop validator is not used to guide timetable generation, matching, EV assignment, or local candidate construction. When a candidate improves the current incumbent, it is written as a complete output file and externally audited. It replaces the archived solution only if this audit confirms feasibility and objective consistency. This keeps constructive search separate from MINOA desktop validation.

The candidate-selection rule is

$$S^* = \arg \min_{S \in \mathcal{S}^{int}} C^{VS}(S), \quad (5.17)$$

where $\mathcal{S}^{int}$ is the set of candidates that pass the internal feasibility checks. The objective C^{VS} is the official vehicle-scheduling cost decomposition introduced in [Chapter 4](#).

The final method is chosen because a single timetable and a single score configuration can be too restrictive. Some instances benefit from dense timetables, while others need sparser selections to reduce vehicle overlap. Some instances benefit from more aggressive EV assignment, while others need a more conservative treatment of charging pressure. The multi-start method tests these alternatives in a bounded and reproducible way.

The method also keeps the comparison fair across instances. Every start follows the same basic sequence: choose a feasible timetable variant, build the graph, solve the matching subproblem, reconstruct blocks, assign vehicle types, check charging, and compute the official objective internally. This common sequence makes it possible to compare Small, Medium, Large, and the additional Senior instances with the same solver structure. Differences in the results therefore come from instance structure and candidate quality, not from changing the definition of the algorithm from one instance to another.

The no-regression archive is a best-so-far storage rule applied separately to each instance. During implementation, it became clear that additional variants do not improve every benchmark file. A timetable rule or edge-scoring rule may help one instance and worsen another. The archive handles this practical issue by storing the best retained solution found so far for each instance. A new candidate can replace the stored solution only when it improves the internal objective and the external audit confirms feasibility and objective consistency. If the candidate is worse, infeasible, or not clearly better under C^{VS} , the previous archived solution is retained.

The archive is updated by the following no-regression rule:

$$S_{k+1}^{archive} = \begin{cases} S^{new}, & \text{if } S^{new} \text{ passes audit and} \\ & C^{VS}(S^{new}) < C^{VS}(S_k^{archive}), \\ S_k^{archive}, & \text{otherwise.} \end{cases} \quad (5.18)$$

This rule separates exploration from reporting. The solver may test several timetable variants and edge-scoring variants, but a later unsuccessful variant cannot overwrite a stronger validated output. The archive therefore supports reproducibility: the reported

final solution is the best retained validated candidate for the instance, not simply the last candidate generated by the search. It does not make the method exact and it is not an optimality certificate.

The method is called a matheuristic because it combines exact and heuristic parts. The path-cover matching is solved exactly for each fixed selected timetable, fixed graph, and fixed edge-score definition. The full integrated problem remains heuristic because timetable-variant generation, edge weighting, EV/ICE assignment, charging insertion, candidate selection, and archive updates do not form a global exact formulation. The final solution is therefore a feasible upper bound, not a proof of optimality.

Table 5.1: Pseudocode of the multi-start path-cover matheuristic.

Step	Operation
1	Generate a feasible timetable variant π^h .
2	Build the compatibility graph $G^h = (\pi^h, A^h)$.
3	Assign operational edge weights w_{ij} .
4	Solve the weighted path-cover matching for the fixed graph.
5	Reconstruct vehicle blocks from matched successor arcs.
6	Assign ICE/EV vehicle types and check battery feasibility.
7	Insert or check charging intervals and capacity reservations.
8	Compute C^{VS} internally for feasible candidates.
9	Audit archive-improving candidates and update the no-regression archive.
10	Report the retained archive output for the instance.

Table 5.1 summarizes the final method without exposing implementation details. The important point is the order of decisions. The method first creates a complete candidate schedule and evaluates it internally. Only archive-improving candidates are audited before they replace an already stored solution. This makes the method reproducible while keeping validation separate from timetable generation, graph construction, matching, and EV/ICE assignment.

5.3.5 Rationale for Selecting the Multi-Start Path-Cover Matheuristic

The final method was selected because it gives the best balance between feasibility, cost quality, scalability, interpretability, and reproducibility among the implemented approaches. The greedy heuristic is fast and transparent, but it depends strongly on early local choices. The unweighted path-cover method improves vehicle-block construction by solving a fixed-graph matching problem, but it ignores operating-cost differences between feasible arcs.

The weighted path-cover method adds cost awareness, but it still depends on one selected timetable and one graph structure. The multi-start weighted path-cover matheuristic addresses these limitations by testing several timetable and graph variants while preserving the exact matching component inside each fixed graph.

From a research perspective, this is a suitable compromise for the MINOA Senior benchmark. A full integrated MIP would represent more decisions jointly, but it would also be much harder to solve over all available instances. A pure greedy heuristic would be easier to implement, but it would give less systematic control over vehicle-block construction. The selected method stays between these extremes. It keeps an exact graph-optimization step, adds bounded heuristic search around timetable and graph variants, and remains understandable enough to analyze in tables and figures.

The method also aligns with the computational goals of this thesis. The aim is to produce feasible schedules, compare algorithmic variants, report validated upper bounds, and discuss lower-bound diagnostics without claiming global optimality. The multi-start matheuristic supports this goal because it can be run on all Senior instances, produces complete output files, and gives a clear reason why one candidate is retained over another: it has the best internally computed C^{VS} among the feasible candidates considered, subject to the no-regression archive.

5.4 EV Assignment and Charging Feasibility

After the vehicle blocks are constructed, each block is tested for EV operation. The test is chronological. The battery state decreases during passenger service and deadhead driving and increases during feasible charging intervals. For an activity a in block b , the update is

$$q_{b,a+1} = \min\{Q^{\max}, q_{b,a} - \Delta d_{b,a} + \Delta q_{b,a}^{\text{chg}}\}, \quad q_{b,a} \geq 0. \quad (5.19)$$

An EV is assigned only if the battery never becomes negative and the required parking, slow-charging, and fast-charging capacities are available. If a block is not EV-feasible, it remains an ICE block, provided that the conventional vehicle-scheduling rules are satisfied.

The EV decision is made for a whole block, not for a single trip. This is necessary because one physical vehicle operates the complete sequence of activities in the block. If the first trip of a block is electric, the remaining autonomy after that trip affects all later activities. The algorithm therefore simulates the complete block before assigning the vehicle type.

Charging insertion is performed only during eligible breaks. A break is eligible when the vehicle is at a node with charging infrastructure, the break is long enough for the minimum charging duration, and the relevant charger is available in the capacity ledger. If the capacity reservation fails, the charging interval is not inserted. This prevents the method from creating electric blocks that look feasible in isolation but violate terminal-resource limits when all vehicles are considered together.

The method does not maximize the number of EVs as a separate objective. It uses EVs when the complete block is feasible and when the resulting schedule is competitive under C^{VS} . Consequently, the resulting EV share may differ across instances according to block structure, autonomy, charging opportunities, fleet availability, and terminal capacities.

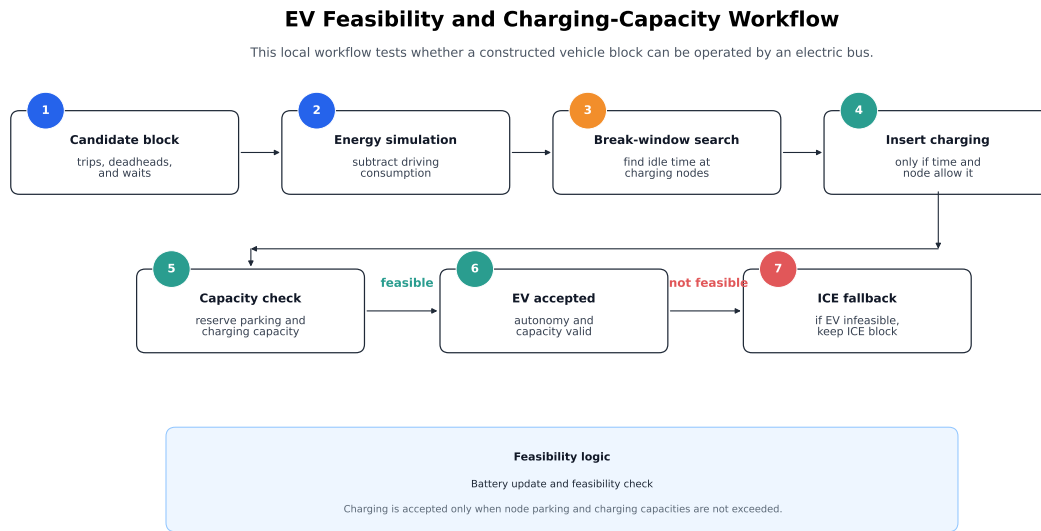


Figure 5.3: EV-feasibility check for one vehicle block.

Figure 5.3 shows the order of the EV-feasibility checks. The method first simulates autonomy, then searches for feasible charging opportunities, and finally verifies terminal capacities. This order keeps the ICE/EV assignment conservative: an electric vehicle is used only when all resource restrictions are satisfied. The figure also explains why an ICE block can remain in the final solution even when EVs are available: if charging time, autonomy, or terminal capacity fails for the complete block, the block cannot be reported as electric. EV feasibility cannot be decided from one arc alone; the whole chronological sequence must be simulated before a vehicle type can be assigned.

5.5 Comparison of Methodological Roles

Table 5.2: Methodological comparison of the four implemented algorithms.

Method	Block construction	Cost and timetable treatment	EV/charging treatment	Main strength	Main limitation
Greedy constructive heuristic	Builds each block by repeatedly choosing one local feasible successor.	Uses one timetable and no matching; arc choice is based on local waiting and deadhead measures.	EV assignment is checked after block construction.	Fast, transparent, and easy to debug.	Early local choices can block better later continuations.
Unweighted path cover	Builds blocks from a maximum-cardinality matching on one fixed compatibility graph.	Minimizes vehicle starts for the selected graph, but treats all feasible arcs equally.	EV assignment is checked after matched blocks are reconstructed.	Reduces vehicle blocks systematically for a fixed timetable.	Treats all feasible arcs equally and ignores operating-cost differences.
Weighted path cover	Builds blocks from a maximum-weight matching on one fixed compatibility graph.	Adds operational edge scores for waiting, depot use, charging opportunity, and EV potential.	Charging risk and EV potential influence edge scores; full feasibility is checked later.	Can improve cost even when vehicle count does not change.	Edge scores are local approximations and cannot predict all future capacity conflicts.
Multi-start weighted path cover	Repeats weighted path-cover construction over several timetable and graph variants.	Selects the best feasible candidate by C^{VS} and keeps the validated archive by no-regression.	Performs block-level EV/ICE assignment, charging checks, and internal feasibility tests.	Best balance of cost quality, scalability, and reproducibility among the tested methods.	Produces feasible upper bounds but does not prove optimality.

Table 5.2 summarizes the methodological development path. Each method keeps the same benchmark interpretation but changes the construction logic. Greedy construction tests local successor choices, unweighted path cover tests fixed-graph vehicle reduction, weighted path cover tests cost-aware graph construction, and the multi-start method tests several timetable and graph variants. Matching is exact for a fixed graph, but timetable generation, candidate selection, EV assignment, and charging insertion remain heuristic components of the complete integrated planning problem. The final method is therefore reported as a matheuristic that produces validated feasible upper bounds rather than optimality certificates.

Chapter 6

Software Implementation and Experimental Design

The implementation follows the mathematical structure from [Chapters 4](#) and [5](#). Input files are read as MINOA JSON instances, candidate timetables are generated, compatibility graphs are built, vehicle blocks are constructed, ICE/EV assignments are evaluated, and final solutions are written in the required output format. The same code path is used for the headline instances and for the additional Senior instances.

6.1 Software Architecture

The repository separates parsing, algorithmic construction, auditing, and reporting. This separation was used deliberately, because the same final solution must be understandable from the code, from the generated JSON output, and from the thesis tables.

Table 6.1: Main implementation components.

Component	Role in the experiment pipeline
<code>cli.py</code>	Reads the instance files and exposes trips, directions, nodes, costs, fleet limits, deadhead data, and charging capacities.
<code>timetable.py</code>	Builds timetable selections that satisfy first-trip, final-trip, and headway requirements.
<code>solver.py</code> and <code>blocks.py</code>	Builds the compatibility graph and solves the unweighted and weighted matching subproblems.
<code>ev_assignment.py</code> , <code>ev_battery.py</code> , <code>ev_charging.py</code> and <code>costs.py</code>	Checks EV autonomy, inserts charging where possible, and verifies parking and charger capacities.
<code>scripts/run_experiment.py</code>	Computes $C^{VS} = C^{fix} + C^{break} + C^{pull} + C^{CO_2}$ using the same component definitions as the reported cost audit.
	Runs the selected algorithm and scope, writes final outputs, and creates the report tables used in the thesis.

[Table 6.1](#) shows the part of the code base that is used to produce the final numerical results. Documentation drafts, local LaTeX files, and intermediate notes are not required for running the public experiment pipeline. In the implementation, the unweighted path-cover step uses `networkx.algorithms.bipartite.maximum_matching`, while the weighted variant uses `networkx.algorithms.matching.max_weight_matching` with `maxcardinality=True`.

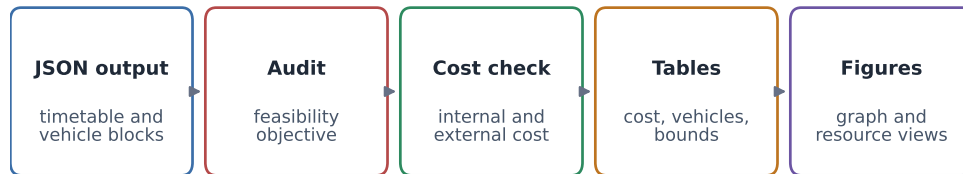
Implementation Flow from Construction to Reported Evidence

(a) Solution-construction pipeline



Construction ranks complete internally checked candidates; edge scores are surrogate matching weights.

(b) Validation, reporting, and visualization pipeline



Reporting analyzes the selected output; it does not guide graph construction or matching decisions.

Figure 6.1: Experiment pipeline used for reproducible result generation.

Figure 6.1 separates constructive optimization from external auditing. Candidate schedules are generated and ranked internally. After the final incumbent has been selected, the MINOA desktop validator is called to check feasibility and objective agreement. This check is separate from candidate construction and ranking. The figure distinguishes three activities that are easy to mix up: constructing candidates, selecting an incumbent by the internal objective, and auditing the written output externally.

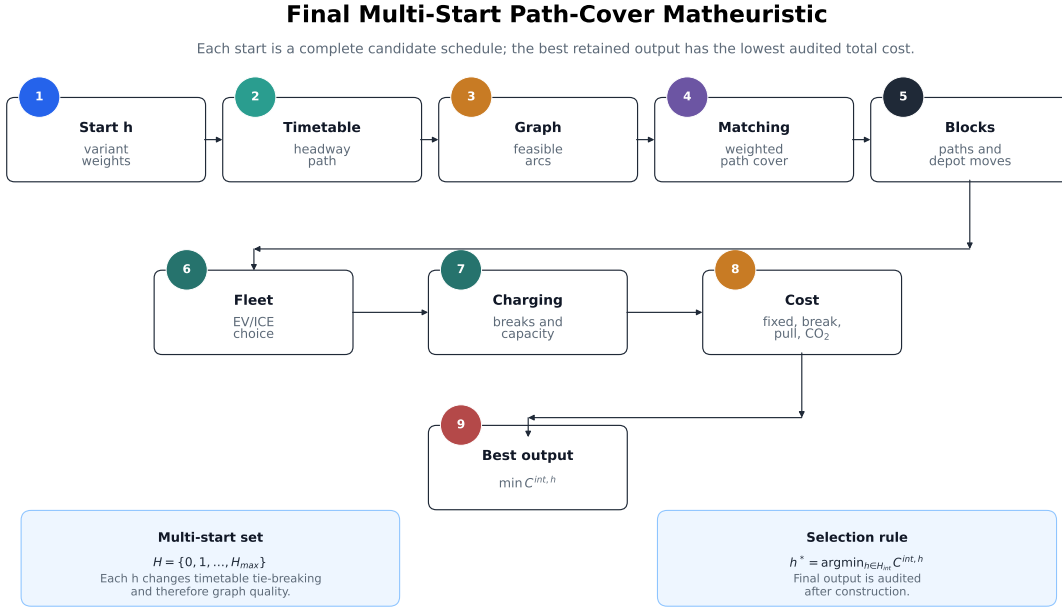


Figure 6.2: Final solver workflow from input data to archived output.

Figure 6.2 gives a second view of the same software flow at the solver level. The input file is converted into internal objects for trips, nodes, deadhead arcs, vehicle types, and charging resources. The optimization layer then creates candidate schedules. The reporting layer writes the selected output and produces the tables used in the computational chapter. It connects the mathematical objects from Chapters 3–5 to concrete software modules and generated evidence files. Parsed benchmark data become timetable objects, compatibility graphs, vehicle blocks, fleet assignments, and finally archived JSON output.

The implementation uses plain Python data structures rather than a mathematical modeling interface. This choice reflects the selected solution method. The central optimization step is graph matching, so the implementation mainly needs reliable graph construction, objective calculation, and schedule serialization. For this reason, the code is organized around small modules instead of one large script. Each module corresponds to one planning object or one feasibility layer.

The internal representation of a candidate schedule contains four linked objects:

$$S = (\pi, B, u, Q), \quad (6.1)$$

where π is the selected timetable, B is the list of vehicle blocks, u is the ICE/EV assignment, and Q is the charging and battery-state information. The output JSON file is produced

only after these four objects are available. This prevents the writer from producing a partial schedule that covers trips but lacks the corresponding vehicle or charging information.

The candidate lifecycle is:

$$\begin{aligned} &\text{parse input} \rightarrow \text{generate timetable} \rightarrow \text{solve path cover} \\ &\rightarrow \text{assign fleet} \rightarrow \text{audit objective} \rightarrow \text{write output.} \end{aligned} \quad (6.2)$$

Each arrow represents a real data transformation in the implementation. The MINOA desktop validator is called after the output has been written, so it checks the final result rather than constructing it.

6.2 Internal Objective and External Audit

The constructive search ranks candidates with internal feasibility checks and the internal objective calculation. For every internally feasible candidate S , the implementation computes

$$C^{VS}(S) = C^{fix}(S) + C^{break}(S) + C^{pull}(S) + C^{CO_2}(S). \quad (6.3)$$

The incumbent is the feasible candidate with the smallest internally computed value. The MINOA desktop validator is then applied only to the selected archived output. The reconciliation residual is

$$\Delta^{audit} = \left| C_{\text{internal}}^{VS} - C_{\text{external}}^{VS} \right|. \quad (6.4)$$

In the final runs, the tolerance for reporting equality is 10^{-2} , because the printed validator values are rounded to two decimals.

The audit is useful because it separates three quantities that can otherwise be confused. The first quantity is the internal matching score w_{ij} , which is used only to construct graph paths. The second quantity is the internal complete-schedule objective C_{internal}^{VS} , which ranks feasible candidates. The third quantity is the objective value printed by the external checker. Only the second and third quantities should match numerically. The matching score is a surrogate and has no direct unit in the official objective.

For each final instance, the implementation stores the objective components before writing the final report:

$$C_{\text{internal}}^{\text{fix}} = \sum_{b \in B} \sum_{v \in V} c_v^{\text{fix}} u_{bv}, \quad (6.5)$$

$$C_{\text{internal}}^{\text{break}} = c^{\text{break}} \sum_{b \in B} p_b, \quad (6.6)$$

$$C_{\text{internal}}^{\text{pull}} = \sum_{b \in B} \sum_{v \in V} c_v^{\text{pio}} (t_b^{\text{pi}} + t_b^{\text{po}}) u_{bv}, \quad (6.7)$$

$$C_{\text{internal}}^{\text{CO}_2} = \sum_{b \in B} \sum_{v \in V} c_v^{\text{CO}_2} d(b) u_{bv}. \quad (6.8)$$

These component values are reported in the results chapter. They also make the cost calculation easier to debug: if an output is feasible but the objective does not reconcile, the component table shows where the difference appears.

6.3 Runtime and Environment

The reported solver runtime measures the constructive optimization pipeline: timetable generation, graph construction, path-cover matching, EV assignment, charging checks, and output writing. Lower-bound experiments and external audit calls are reported separately when they are used. Therefore, runtime tables distinguish solver time, lower-bound time, and validation time instead of treating all of them as one historical competition time.

Table 6.2: Computing environment used for the reported experiments.

Item	Specification
Processor	Apple M3 processor, 8 CPU cores
Memory	16 GB
Operating system	macOS 26.5.2
Python	Python 3.9.6 virtual environment in <code>.venv</code>
External audit	MINOA desktop validator JAR
NBench indices	Not measured; placeholder metadata is not used for performance comparison

The hardware information in [Table 6.2](#) is used only to interpret reproducibility and runtime. The experiments are not presented as an exact reproduction of the historical competition environment. The reported runs were carried out in this CPU-based local environment. No GPU or HPC cluster was used for the reported experiments, and no commercial optimization solver was required by the implemented path-cover pipeline.

The runtime fields are interpreted conservatively. The constructive solver runtime measures the time needed to create the final candidate from an input instance. The lower-bound runtime measures a separate diagnostic calculation. The validation runtime measures the external checker call. File input and output time is small for the tested instances, but it is conceptually separate from the optimization time. The total wall-clock time of a complete experiment therefore depends on which of these tasks are requested.

For a reproducibility run, the most important timing distinction is between constructive time and audit time. A schedule can be generated without running the lower-bound script. It can also be generated before the MINOA desktop validator is called. This distinction keeps candidate construction separate from final output auditing.

6.4 Experiment Commands and Scopes

The modules in [Table 6.1](#) are located in `scripts/minoa_lib/`, except for the top-level experiment runner.

The same experiment runner is used for all algorithms. The algorithm argument selects the constructive method, and the scope argument selects either the three headline instances or all twelve Senior instances:

$$(\text{algorithm}, \text{scope}) \mapsto (\text{outputs}, \text{audit table}, \text{summary report}). \quad (6.9)$$

The public repository documents the concrete commands for rerunning the headline and full-benchmark experiments. This includes the command for regenerating result plots from the verified CSV files.

The scope argument changes the instance set but not the solver logic. For `scope=sm1`, the runner processes Small, Medium, and Large. For `scope=all`, it processes all twelve Senior instances. The output directory is created automatically, and each generated JSON file is paired with the corresponding input file for reporting. The table-generation script then reads these pairs and prints the same columns used in the thesis.

The algorithm argument changes the construction layer. The available values include the greedy constructive method, the unweighted path-cover method, the weighted path-cover method, and the multi-start path-cover method. This makes the algorithm comparison reproducible without editing source code. The same reporting script can therefore compare methods, instances, and archived final outputs.

The lower-bound command is separated from the main experiment command. This is deliberate because the lower-bound calculation can take substantially longer than constructing a feasible schedule. The lower-bound CSV stores the candidate count, selected trips, bound status, solver backend, and runtime. In the thesis, these values are used only for interpreting solution quality; they do not change the final feasible schedules.

6.5 Reproducibility Controls

Three controls were used to keep the reported results stable. First, the inputs are stored in the repository together with the repair note for the one malformed JSON file. The repair only closes the top-level JSON object and does not change optimization data. Second, the experiment runner retains the better validated solution when a new candidate is worse than an archived result. Third, the objective audit compares internal and external objective values for the final solution files.

These controls do not prove global optimality. They ensure that the reported solutions are reproducible feasible upper bounds under the stated experimental environment.

The final archive is the central reproducibility object. It contains one validated output per reported instance and the corresponding cost audit. When a fresh multi-start run finds an equal or better candidate, the candidate must be validated and archived explicitly before it can replace the reported row. This rule prevents accidental changes of the thesis tables caused by a different run time, different random order, or a local experimental file.

The repository intentionally keeps generated outputs separate from source code. Source files define how a result is produced; archive files record which result is reported. This distinction is useful for a thesis project because the code may continue to find improved candidates after the writing stage. The final scientific claim, however, must refer to the validated archive that is actually reported in the thesis.

Chapter 7

Computational Results

This chapter reports the validated results for the four implemented algorithms. The main reported value is the official vehicle-scheduling objective C^{VS} . All final rows in the result tables pass the independent external audit and have also been checked by the internal objective calculation.

7.1 Evaluation Metrics

The cost of a feasible solution S is

$$C^{VS}(S) = C^{fix}(S) + C^{break}(S) + C^{pull}(S) + C^{CO_2}(S). \quad (7.1)$$

Lower cost is better. The number of vehicles is also important, because fixed vehicle cost is the largest cost component in most instances. The EV share, deadhead minutes, paid-break minutes, and charging minutes are used to interpret how the schedules differ operationally.

For lower-bound analysis, the thesis distinguishes a globally valid bound from a selected-timetable diagnostic bound:

$$LB^{global} \leq C^* \leq UB, \quad (7.2)$$

where UB is the validated feasible cost and C^* is the unknown global optimum. The selected-timetable bound $LB^{TT}(\pi)$ is useful for understanding a fixed timetable, but it is not a proof of global optimality for the complete integrated problem.

7.2 Comparison of the Four Algorithms

Table 7.1: Aggregate comparison of the four implemented algorithms.

Algorithm	Cost	Vehicles	EV	ICE	Main interpretation
Greedy constructive	10993.20	138	46	92	Fast reference method with local block decisions.
Unweighted path cover	11027.94	138	39	99	Reduces chaining to matching but does not score arc quality.
Weighted path cover	10946.25	138	50	88	Uses waiting, deadhead, and charging-risk information in the graph.
Basic multi-start path-cover run (pre-archive)	10804.55	136	47	89	Best direct run before final archive-level improvements.

[Table 7.1](#) compares the four direct algorithm runs under the same reporting format. The weighted graph methods reduce cost compared with the greedy reference, and the basic multi-start run is the strongest direct run before archive-level improvements. The 136-vehicle value belongs to the basic multi-start path-cover run before archive-level improvements. The final no-regression archive keeps the best validated output per instance and reduces the full-benchmark vehicle total to 126. The final reported archive is evaluated separately in [Tables 7.4](#) and [7.5](#).

The aggregate comparison should be interpreted together with the individual instance results. A method can have the same total vehicle count as another method but still be better because it creates shorter paid breaks, shorter pull-in/out movements, or more suitable EV assignments. This is why the thesis reports both cost and operating indicators. The total cost is the final ranking criterion, while vehicle count, EV count, and time components explain where the improvement comes from.

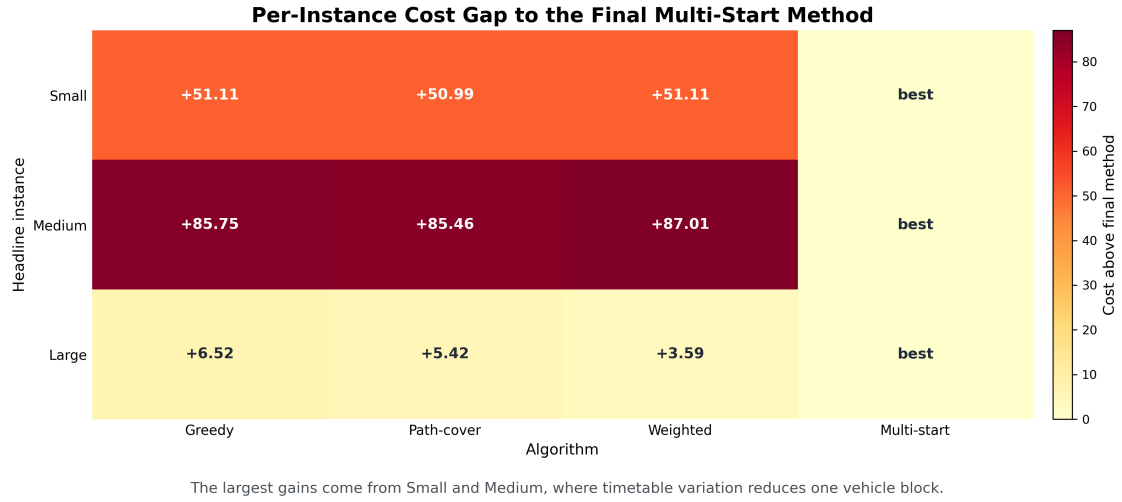


Figure 7.1: Algorithm comparison on the headline instances.

Figure 7.1 shows the same development path for Small, Medium, and Large. The four bars in each instance group correspond to Greedy construction, Unweighted Path Cover, Weighted Path Cover, and Multi-start Path Cover. Greedy construction makes local successor decisions, while Unweighted Path Cover selects compatible trip continuations jointly by matching. Weighted Path Cover keeps the same graph structure but changes the edge decision by including waiting, deadhead, and charging-risk information. Multi-start Path Cover adds several feasible timetable and graph variants before the best complete schedule is retained. The largest improvement comes from replacing local greedy chaining by graph-based path-cover construction. This supports the main methodological contribution: the improvement does not come only from trying more candidates, but from using the compatibility graph to choose trip continuations jointly. Small benefits mainly through a lower vehicle count, Medium benefits from productive electric blocks, and Large shows that weighted and multi-start decisions can still reduce cost even when the final vehicle count remains high.

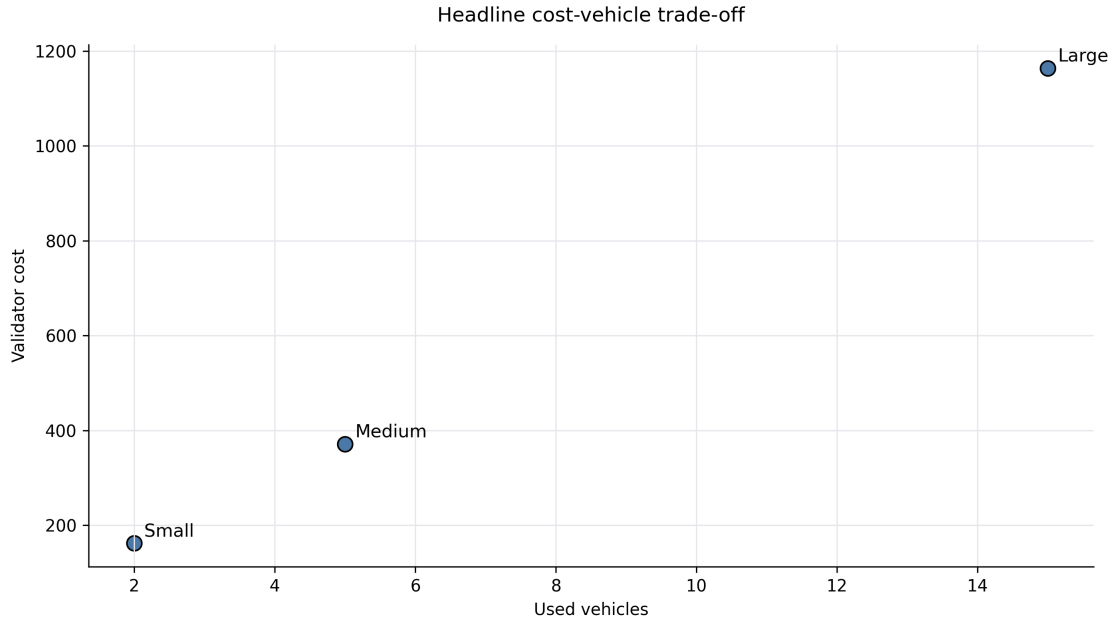


Figure 7.2: Cost and vehicle trade-off for the headline algorithm variants.

Figure 7.2 separates two aspects of solution quality for the headline variants. The vehicle count is important because the fixed cost is high, but a solution with the same vehicle count can still have different cost because of paid breaks, pull-in/out movements, and CO₂ cost. Fixed vehicle cost is therefore only one part of the comparison. The final multi-start solution is evaluated by both axes: it keeps the vehicle count low and also reduces the audited objective. The figure indicates that the final method improves solution quality through both effects where possible: it reduces vehicles for Small and Medium, and it improves the block structure and cost components for Large. This is why the thesis reports the complete C^{VS} , not only the number of vehicles.

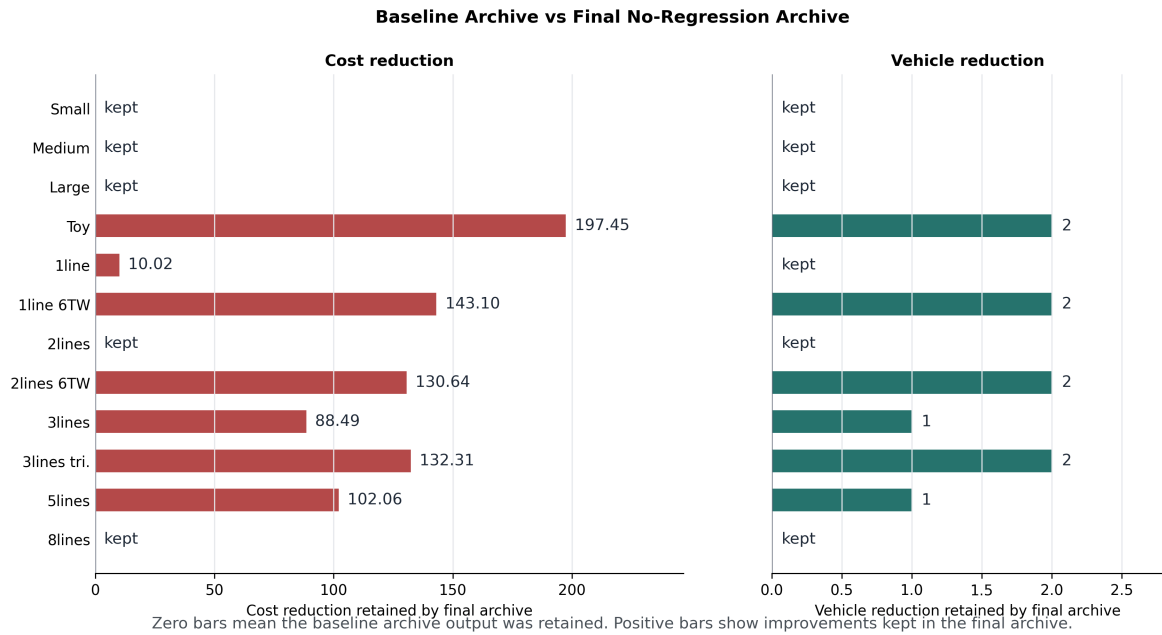


Figure 7.3: Instance-level improvement from the baseline archive to the final no-regression archive.

Figure 7.3 shows that archive improvement is instance-specific. For some instances, the final archive keeps the previous validated output because no later candidate improved it. For other instances, adaptive variants or local repair produced a better validated candidate. This supports the no-regression archive idea: the solver can explore additional variants without worsening already strong instance results. The figure does not prove global optimality; it only reports the gains retained among the tested validated candidates.

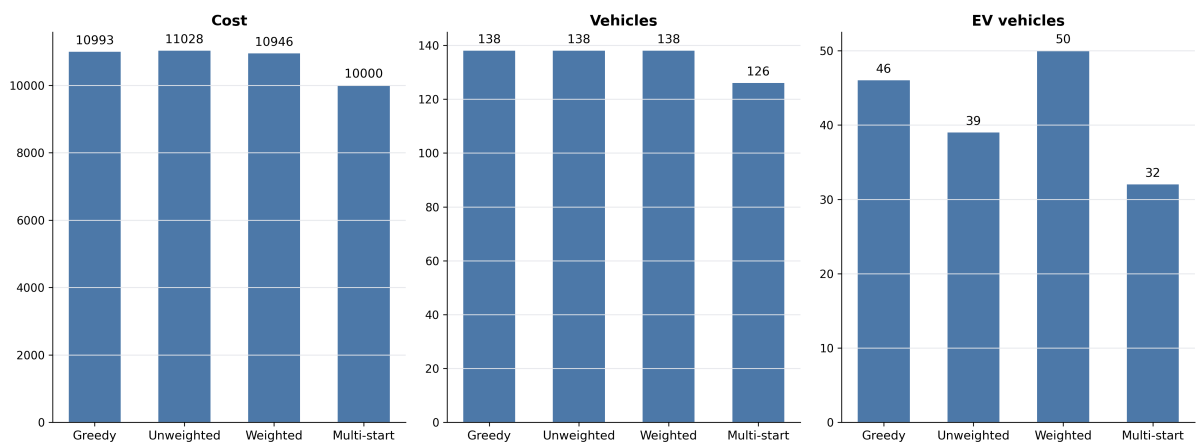


Figure 7.4: All-instance comparison of the four implemented algorithms.

Figure 7.4 complements the aggregate table by showing the algorithm comparison visually. The main interpretation is not that one method dominates every operational indicator. Instead, the figure shows the development path: local greedy construction is the simplest reference, path-cover matching improves vehicle-block construction, weighted scoring adds operational information, and multi-start search improves the final archive. The figure supports the conclusion that the final method is the strongest tested single algorithm overall, while also showing why the thesis still discusses the earlier variants as meaningful baselines.

7.3 Headline Instances

Table 7.2: Final validated results for Small, Medium, and Large.

Instance	Cost	Vehicles	EV	ICE	EV share (%)	Trips	Deadhead min	Charge min
Small	162.44	2	0	2	0.00	48	42.00	0.00
Medium	371.35	5	5	0	100.00	139	752.00	391.00
Large	1163.35	15	5	10	33.33	260	2600.00	505.00
Total	1697.15	22	10	12	45.45	447	3394.00	896.00

Table 7.2 contains the main benchmark results. Small is solved with two ICE vehicles and no charging. Medium uses five EVs and has the highest EV share. Large keeps the same vehicle count as earlier feasible solutions but lowers the validated cost to 1163.35.

The three headline instances also show why the objective should not be interpreted through vehicle count alone. Small has the lowest total cost, but it also has the smallest timetable. Medium has more selected trips than Small, yet its cost per trip is lower because the final blocks are productive and the fleet is fully electric. Large has the largest number of selected trips and requires more vehicles, so its total cost is naturally higher. The normalized view in Table 7.3 separates these effects.

Table 7.3: Normalized interpretation of the headline results.

Instance	Cost/trip	Trips/vehicle	Deadhead/trip	Charge/EV	Main interpretation
Small	3.38	24.00	0.88	–	compact ICE schedule
Medium	2.67	27.80	5.41	78.20	productive EV blocks
Large	4.47	17.33	10.00	101.00	larger mixed-fleet network

Table 7.3 reports derived values from the final validated solutions. Cost per trip is computed as $C^{VS}/|\pi|$. Trips per vehicle is computed as $|\pi|/|B|$. Deadhead per trip divides

reported deadhead minutes by the number of selected trips. Charge per EV is shown only when EVs are used. The table makes the Medium result easier to understand: it is not only good because all vehicles are electric, but also because each vehicle covers many trips. Large is more difficult because the timetable is spread over more lines and the blocks have more deadhead and charging pressure.

The EV share should therefore be interpreted together with productivity and resource use. A high EV share is attractive when it can be achieved without adding many vehicles or long charging periods. A low EV share is not automatically poor if it avoids additional vehicles and keeps the official cost low. This is why the final method does not force EV assignment whenever an EV is available. It assigns an EV only when the whole block remains feasible and the resulting candidate improves the internally computed objective.

7.4 Complete Senior Benchmark

The additional Senior instances are reported together with Small, Medium, and Large. This avoids treating them as a separate experiment and makes clear which data set is used in each row.

Table 7.4: Final validated results for all twelve Senior instances.

Instance	Cost	Vehicles	EV	ICE	EV share (%)	Trips	Break min	Charge min
Small	162.44	2	0	2	0.00	48	240	0
Medium	371.35	5	5	0	100.00	139	1717	391
Large	1163.35	15	5	10	33.33	260	2520	505
Toy Example	290.99	4	0	4	0.00	68	385	0
1line	313.61	4	1	3	25.00	102	807	67
1line 6timeWindow	387.67	5	1	4	20.00	112	970	0
2lines	910.15	12	5	7	41.67	204	2710	717
2lines 6 timeWindows	495.60	6	0	6	0.00	204	1551	0
3lines	911.50	11	3	8	27.27	306	2725	218
3linesTriangle	809.10	10	2	8	20.00	306	2811	146
5lines	1566.95	19	5	14	26.32	510	5556	965
8lines	2617.76	33	5	28	15.15	819	5552	257
Total	10000.48	126	32	94	25.40	3078	27544	3266

Table 7.4 shows that all twelve Senior instances have validated feasible solutions. The full benchmark uses 126 vehicles and has a total validated cost of 10000.48. The EV share differs strongly by instance because charging opportunities, autonomy, and terminal capacity are not uniform across the data set.

The full benchmark contains three different types of instances. The first group contains compact instances such as Small, Toy Example, and 1line, where few vehicles are sufficient

and the main question is how to chain the trips without creating unnecessary waiting. The second group contains medium-sized multi-line cases such as 2lines, 3lines, and 3lines-Triangle, where the spatial structure creates many possible depot-mediated continuations. The third group contains the large 5lines and 8lines cases, where the fleet cap for EVs becomes binding and the cost is dominated by the number of required vehicle blocks.

This grouping is helpful for reading the table. For example, 2lines and 2lines 6 timeWindows have the same number of selected trips, but they do not have the same cost or vehicle count. The reason is that time-window structure changes which trips can be chained by the same vehicle. Similarly, 3lines and 3linesTriangle both select 306 trips, but the triangle structure permits a different set of continuations and gives a lower final cost. These differences support the graph-based design of the method: the number of selected trips alone is not enough to predict schedule quality; the important object is the compatibility graph built from those trips.

Across all twelve rows, the final archive uses 32 EVs and 94 ICE vehicles. The EV count is smaller than the available EV capacity summed over all instances, because each instance is solved separately and EV feasibility is limited by block chronology. A block must have enough residual autonomy, enough time for charging, and enough terminal capacity at the required moments. If one of these conditions fails, assigning the block to ICE can produce a better feasible solution than forcing an electric assignment.

All-Instance Operational Trade-Off Summary

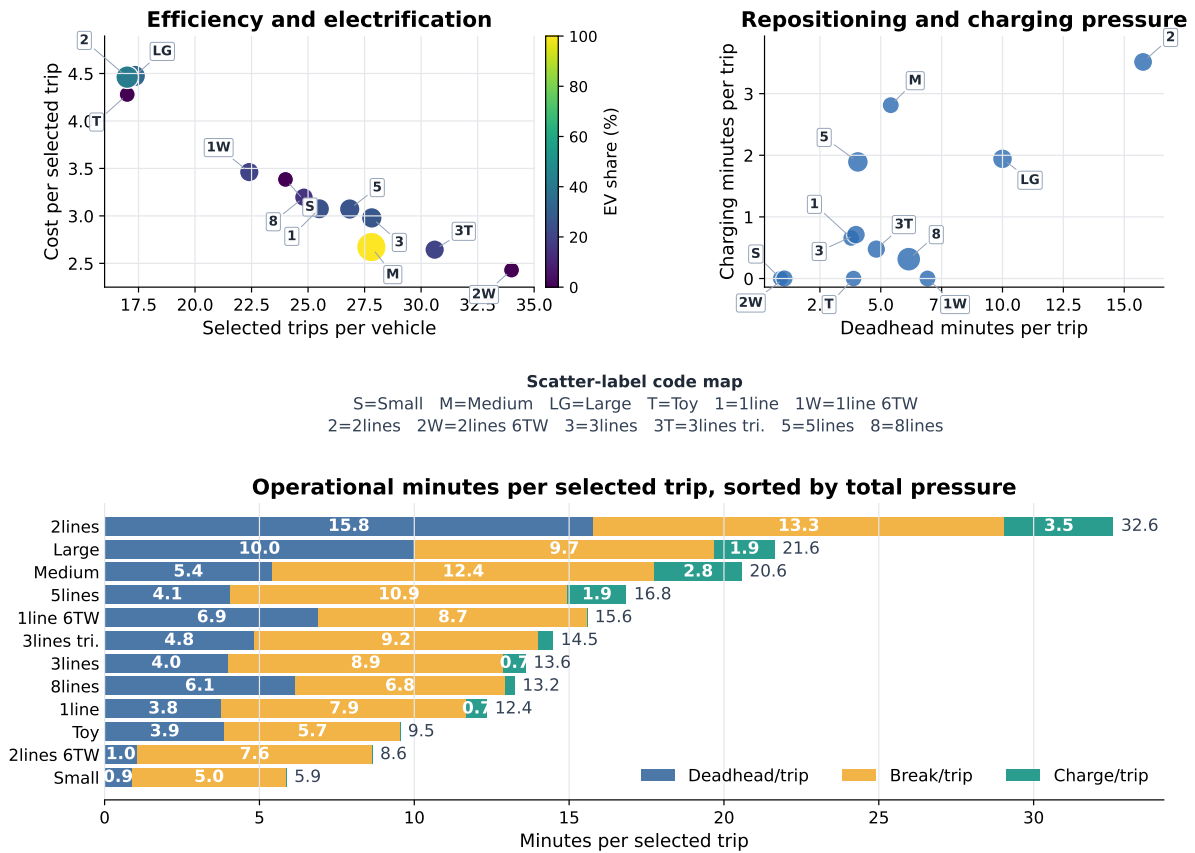


Figure 7.5: Cost, fleet mix, and efficiency over all Senior instances.

Figure 7.5 shows that cost, vehicle count, EV share, and cost per trip do not move in exactly the same way. Some instances have a high EV share but still require many vehicles, while larger instances can have lower cost per trip because fixed vehicle cost is spread over more service trips. The operational summary is therefore not a ranking by one metric. It helps separate scale effects from operational effects and shows why all twelve instances should be discussed together rather than only by total cost.

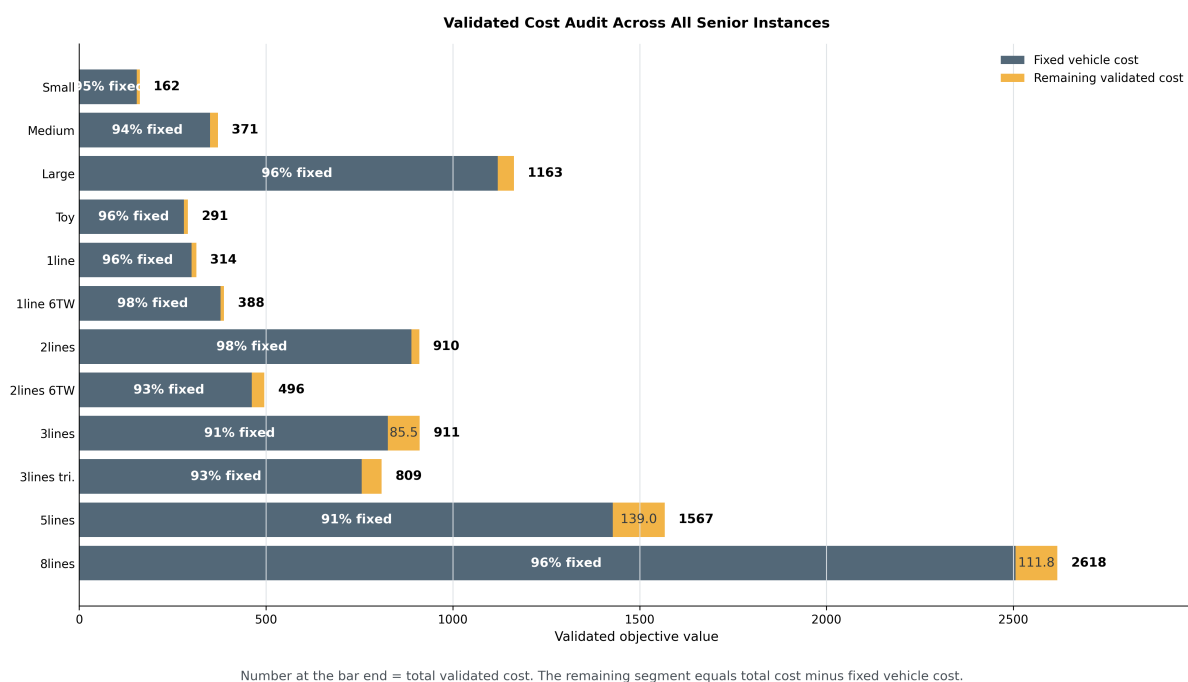


Figure 7.6: Official cost components across all Senior instances.

Figure 7.6 makes the objective decomposition visible. Fixed vehicle cost is the dominant component in most rows, which explains why path-cover construction is central to the method. The smaller components are still reported because they explain differences between solutions with the same vehicle count. In operational terms, this means that once the number of blocks is fixed, further improvements must come from shorter breaks, better pull-in/out structure, or more suitable vehicle-type assignment.

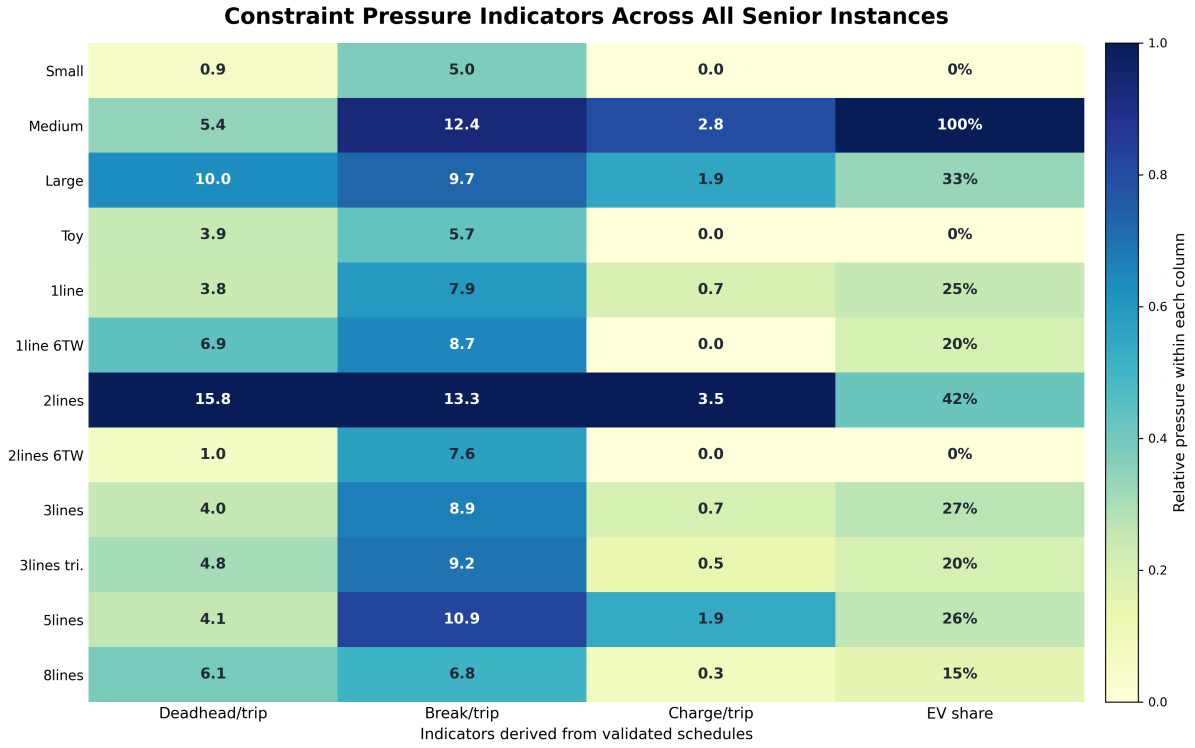


Figure 7.7: Operational pressure indicators for the Senior instances.

Figure 7.7 compares deadhead, break, charge, and EV share indicators. The color scale is column-wise, so it should not be read as one combined score. The plot shows that the difficult part of an instance can come from different sources. Some instances are dominated by waiting and paid breaks, while others are dominated by charging pressure or low EV feasibility. This matters for method design because one improvement step cannot solve every case: deadhead-heavy instances need better chaining, while charging-heavy instances need better EV assignment and resource timing.

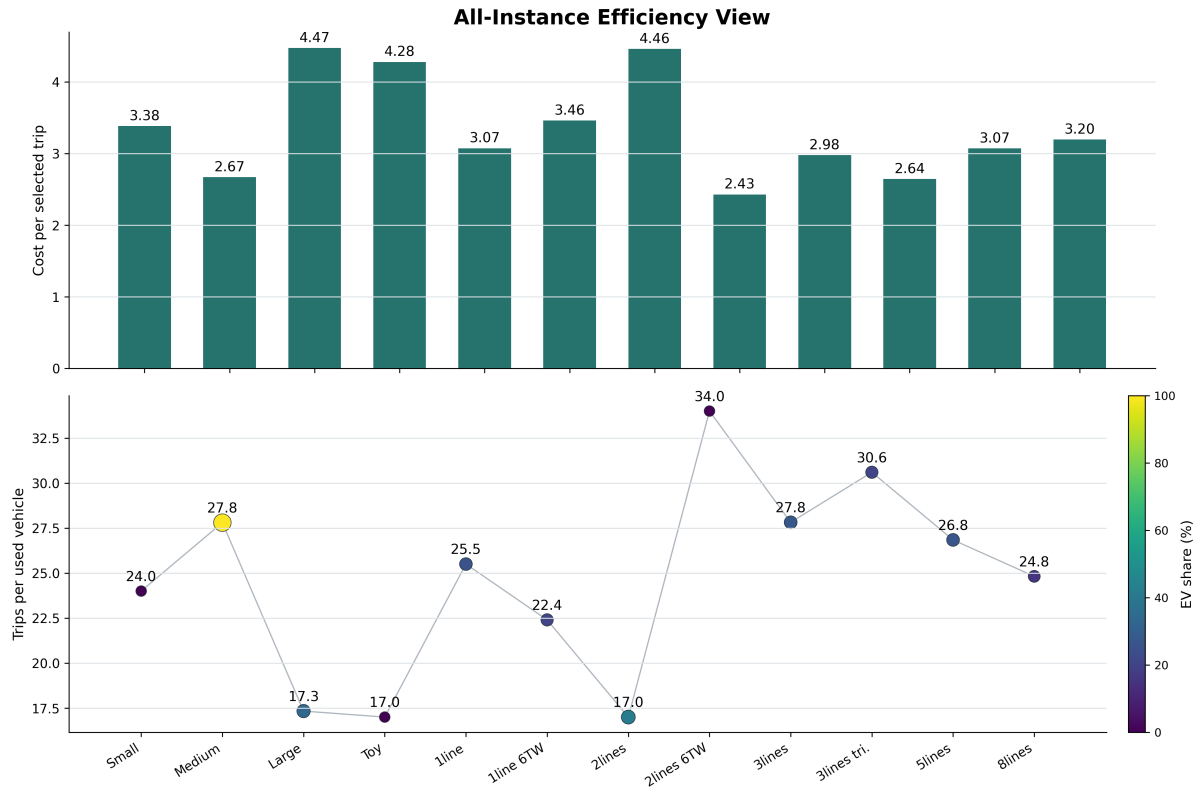


Figure 7.8: Cost per trip and vehicle productivity for all Senior instances.

Figure 7.8 normalizes the results by service volume. Cost per trip accounts for the strong differences in selected service volume across the twelve instances. Vehicle productivity shows how many selected trips are covered by each used vehicle. Together, these metrics show that a larger instance is not automatically worse: the schedule can still be efficient if vehicles are reused well. The figure also explains why EV share alone is not enough to evaluate a schedule; a highly electric solution can still be expensive if it requires weak vehicle productivity or long charging stops. Productivity is close to the path-cover objective, because each additional compatible continuation allows a vehicle to cover more trips. Instances with lower productivity indicate cases where spatial structure, time windows, or charging restrictions make vehicle reuse harder. The final archive summary remains reported in Table 7.4, where cost, vehicles, and fleet composition are listed explicitly for every instance.

7.5 Archive Improvements and Ablation Evidence

The final archive keeps the better of the previous validated output and the new candidate for each instance. Therefore, no final row is worse than the archived baseline. The final archive is not a new global optimization proof. It is the retained best validated output per instance. Compared with the basic multi-start archive, it improves the vehicle count on selected instances, reducing the total from 136 to 126 vehicles.

Table 7.5: Baseline archive versus final archive.

Instance	Baseline cost	Final cost	Cost gain	Baseline vehicles	Final vehicles	Vehicle gain
Small	162.44	162.44	0.00	2	2	0
Medium	371.35	371.35	0.00	5	5	0
Large	1163.35	1163.35	0.00	15	15	0
Toy Example	488.44	290.99	197.45	6	4	2
1line	323.63	313.61	10.02	4	4	0
1line 6timeWindow	530.77	387.67	143.10	7	5	2
2lines	910.15	910.15	0.00	12	12	0
2lines 6 timeWindows	626.24	495.60	130.64	8	6	2
3lines	999.99	911.50	88.49	12	11	1
3linesTriangle	941.41	809.10	132.31	12	10	2
5lines	1669.01	1566.95	102.06	20	19	1
8lines	2617.76	2617.76	0.00	33	33	0

Table 7.5 records where the final archive improved. The headline instances were already at the retained archive value, while several additional instances gained from adaptive candidate generation and bounded repair checks. The ten-vehicle reduction comes from improvements on Toy Example, 1line 6timeWindow, 2lines 6 timeWindows, 3lines, 3linesTriangle, and 5lines. Thus, the final 126-vehicle total is not obtained by modifying one single 136-vehicle schedule. It is obtained by retaining the best validated solution per instance across the archive comparison.

Table 7.6: Ablation evidence from adaptive candidate families.

Configuration	Valid rows	Summed cost	Interpretation
Base time scoring	11	9102.22	Broadest coverage, but not always best after archive comparison.
Charging-edge scoring	6	2181.75	Helps only on the subset where charging pressure is relevant.
Balanced-edge scoring	4	1165.01	Improves selected difficult candidates but covers fewer instances.
Look-ahead charging	4	1157.42	Best diagnostic subtotal among the tested charging-aware variants.
No short depot shortcut	4	1353.37	More conservative and more expensive on the tested subset.

Table 7.6 is not a full independent benchmark, because the configurations are valid on different subsets of instances. It is used as evidence for the local design choices: charging-aware scoring can help, but it must be protected by a no-regression archive. The table shows that the adaptive candidate families do not improve every case in the same way. Their main value is that they create additional feasible candidates for difficult instances. Together with the archive rule, these candidates can improve the reported result without replacing stronger earlier outputs on other instances.

7.6 Lower Bounds, Runtime, and Objective Audit

Table 7.7: Upper and lower bounds for the headline instances.

Instance	UB	LB^{global}	Global gap (%)	$LB^{TT}(\pi)$	Diagnostic gap (%)	LB runtime (s)
Small	162.44	140.00	13.82	140.00	13.82	1.81
Medium	371.35	210.00	43.45	280.00	24.60	120.79
Large	1163.35	0.00	100.00	700.00	39.83	120.99

Table 7.7 shows that the lower bounds are useful for diagnosis but do not certify global optimality. Small has the strongest global bound among the three headline instances. Large has no useful certified global bound in the controlled run, so the selected-timetable bound is reported only as diagnostic information. The upper bounds are the validated costs from the implemented method, while the lower bounds are reference values from relaxed or restricted computations. Larger gaps should therefore be read as a sign that the available bound is loose or that the integrated problem is hard, not as a direct proof that the feasible schedule is poor.

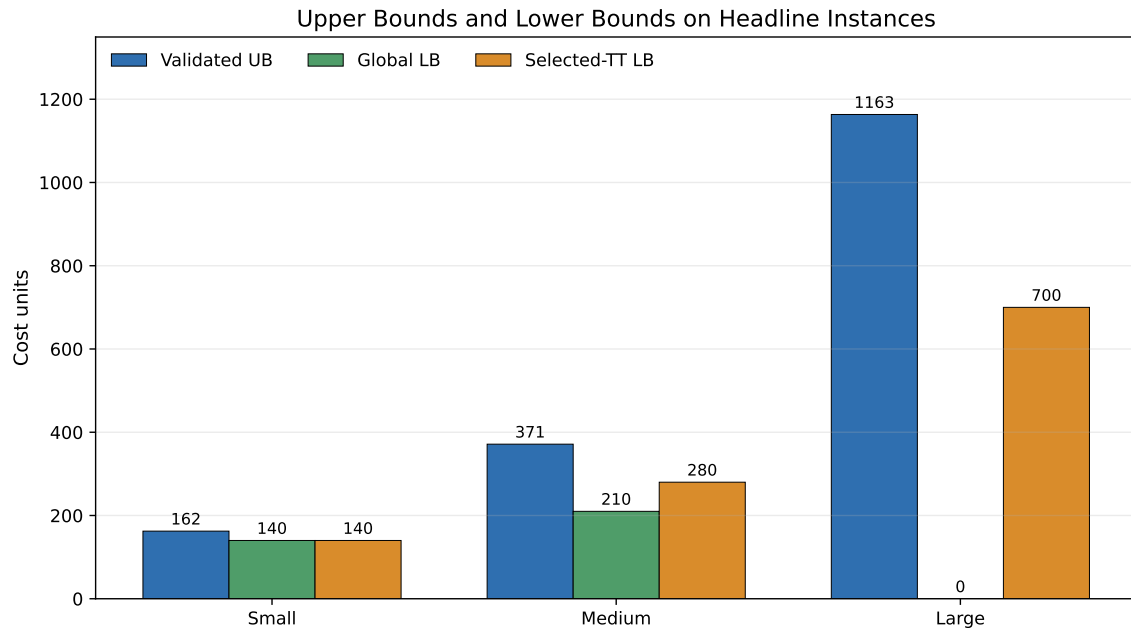


Figure 7.9: Upper bounds and lower bounds for the headline instances.

Figure 7.9 visualizes the distinction between the reported feasible upper bounds and the two lower-bound types. The global lower bound is valid for the complete integrated problem relaxation. The selected-timetable lower bound is computed after a timetable has already been chosen and is therefore diagnostic rather than globally certifying. The important reading is the distance between bars, not only their height: a smaller distance gives more evidence about solution quality, while a missing or weak global bound should be interpreted cautiously. The upper bound is the cost of a constructed schedule, while each lower bound is a relaxation-based reference value.

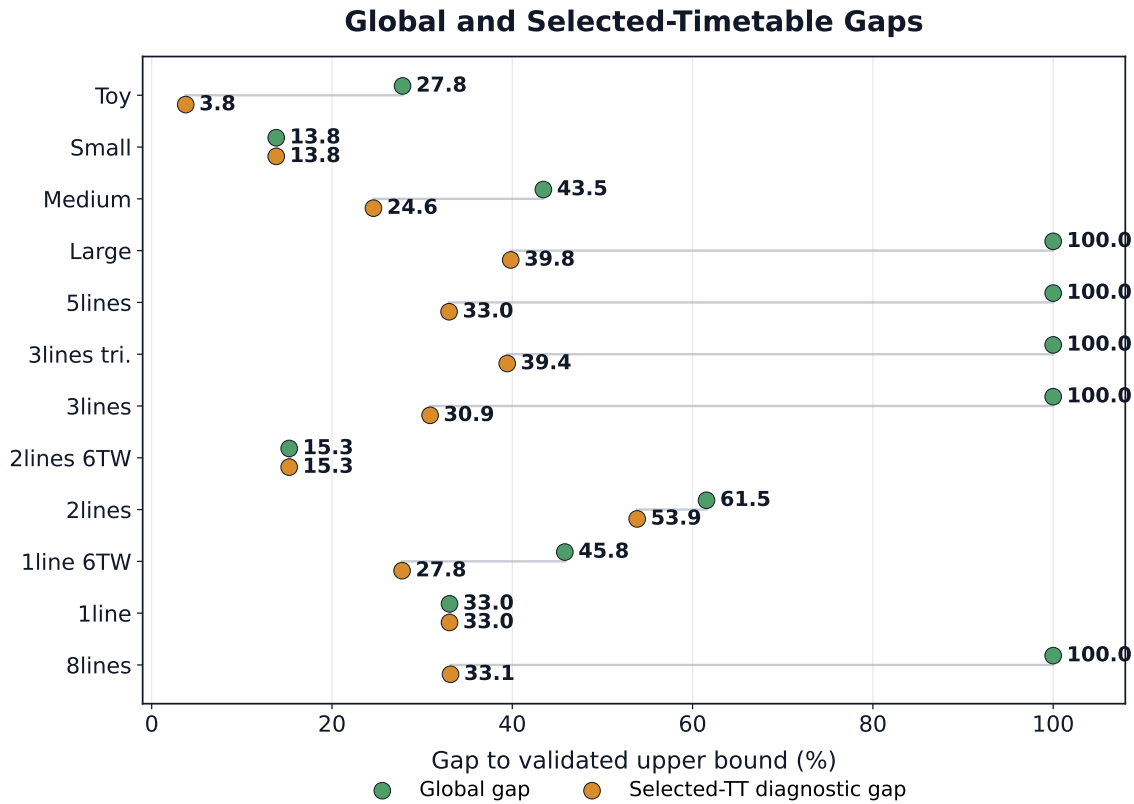


Figure 7.10: Global and selected-timetable diagnostic gaps.

Figure 7.10 shows why both gap columns are reported. A selected-timetable gap can be smaller because it evaluates a restricted problem after timetable selection. This does not mean that the complete integrated solution is near globally optimal. It only means that, for the chosen timetable, the vehicle-scheduling structure is closer to its diagnostic bound. This distinction prevents overclaiming: the figure supports interpretation of the final schedules but does not turn the heuristic method into an exact optimizer.

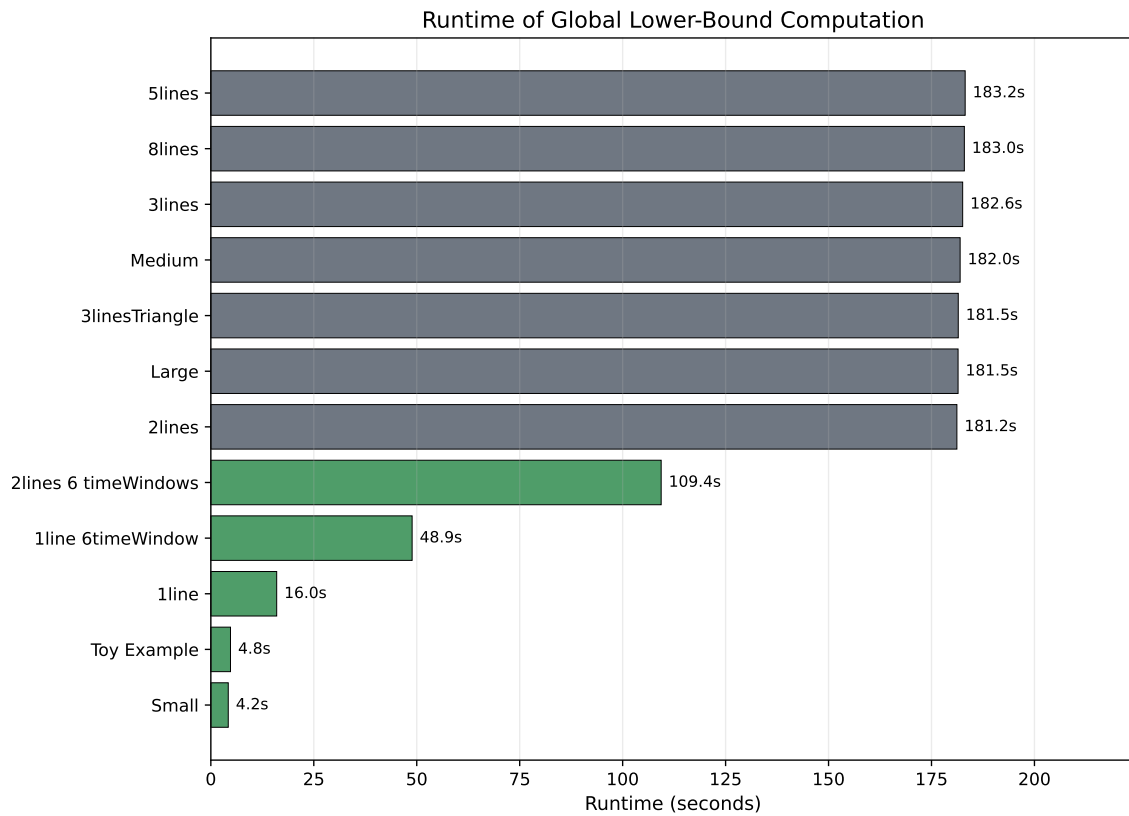


Figure 7.11: Runtime of the lower-bound computations.

Figure 7.11 explains why lower-bound reporting is kept separate from constructive solver runtime. Bound computations answer a different question: they estimate how far a feasible solution may be from a relaxation value. They are not part of constructing the final timetable and vehicle schedule. The figure also shows why exact certification is left as future work for larger instances: proving strong bounds can take substantially more time than producing a feasible schedule.

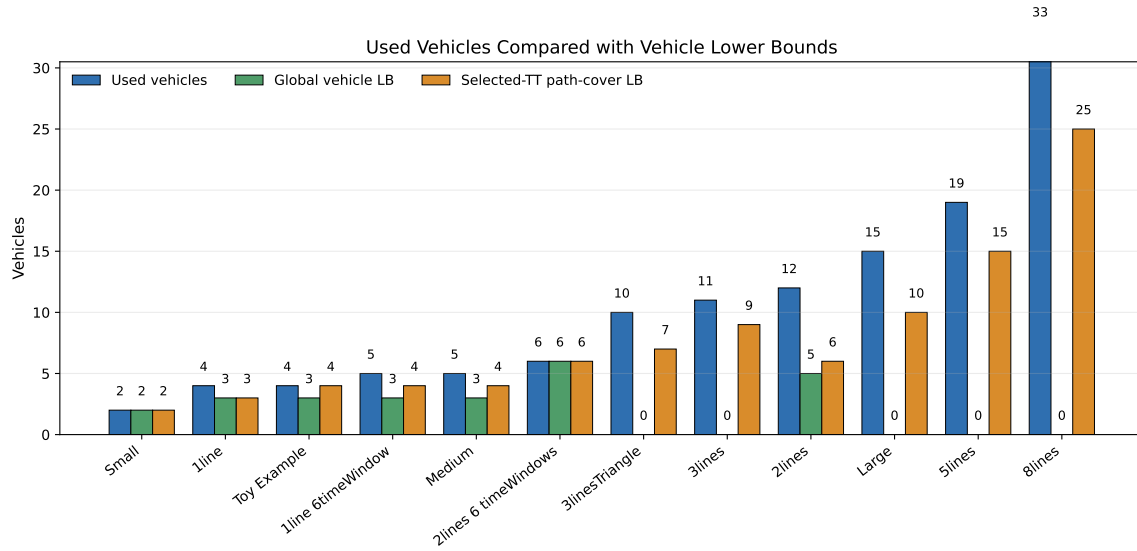


Figure 7.12: Vehicle lower-bound comparison for the headline instances.

Figure 7.12 focuses on vehicle-count bounds rather than cost bounds. Fixed vehicle cost dominates the objective, so the distance to a vehicle lower bound indicates how much room may remain in the largest cost component. If the vehicle count is close to a lower bound, then further cost improvements must mostly come from smaller components such as paid breaks, pull-in/out movements, or fleet assignment. If the vehicle count is far from the bound, then better chaining or timetable variants may still reduce the largest cost driver.

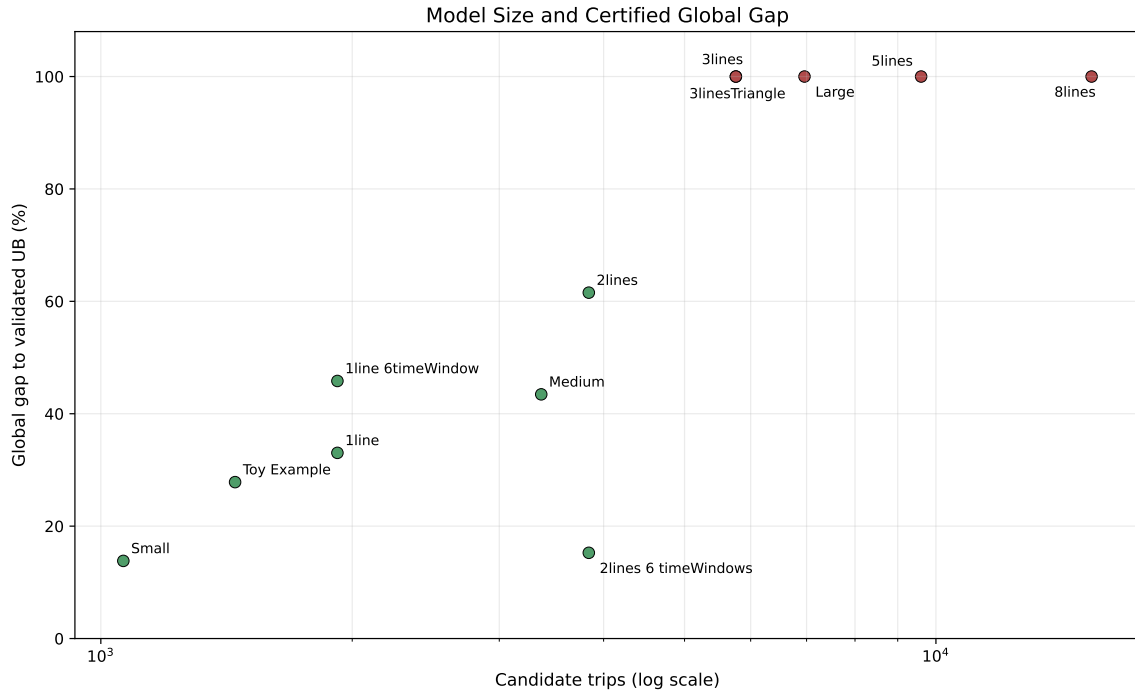


Figure 7.13: Diagnostic gap compared with candidate-trip scale.

Figure 7.13 relates bound quality to instance scale. The number of candidate trips affects the size of the timetable-selection space, but it is not the only source of difficulty. Terminal structure, headway windows, and charging feasibility also affect whether a strong bound can be obtained in the controlled run. The figure therefore supports a cautious interpretation of scalability: larger candidate pools often make the problem harder, but graph structure and resource constraints also shape computational difficulty.

The constructive runtime written into the final output files is 0.073 seconds for Small, 0.890 seconds for Medium, and 869.020 seconds for Large. The Large value is higher because the retained final headline output comes from the deeper multi-start search.

$$t^{total} = t^{solver} + t^{LB} + t^{validator} + t^{IO}. \quad (7.3)$$

Equation (7.3) is used only as a reporting convention. Unless explicitly stated, the output field `executionTime` refers to the constructive solver runtime.

The computational setting is also relevant for interpreting these timings. As reported in Table 6.2, the schedules were produced on a standard CPU-only laptop environment with an Apple M3 processor, 8 CPU cores, 16 GB memory, and macOS. No GPU acceleration, HPC cluster, or commercial solver such as Gurobi was used for the reported runs. This

is encouraging from a reproducibility perspective, because the full Senior benchmark can be handled in a standard CPU-based local environment. At the same time, the runtimes should be interpreted carefully, because NBench indices were not measured and the experiments were not run on the historical competition environment.

Table 7.8: Internal and external objective reconciliation.

Instance group	Internal total	External total	Abs. diff.	Status
Small, Medium, Large	1697.15	1697.15	0.00	reconciled
All twelve Senior instances	10000.48	10000.48	≤ 0.01	reconciled

Table 7.8 confirms that the internal cost calculation reproduces the external audit values within the stated rounding tolerance.

Table 7.9: Objective component audit for the final archive.

Scope	C^{fix}	C^{break}	C^{pull}	C^{CO_2}
Small, Medium, Large	1624.00	50.32	20.36	2.47
All twelve Senior instances	9450.00	412.36	108.29	29.84

The exact reconciliation in Table 7.9 confirms that the CO₂-duration convention used in the formulation reproduces the objective convention used by the MINOA desktop validator. Fixed vehicle cost is the dominant component, so improvements in vehicle count usually have the largest effect.

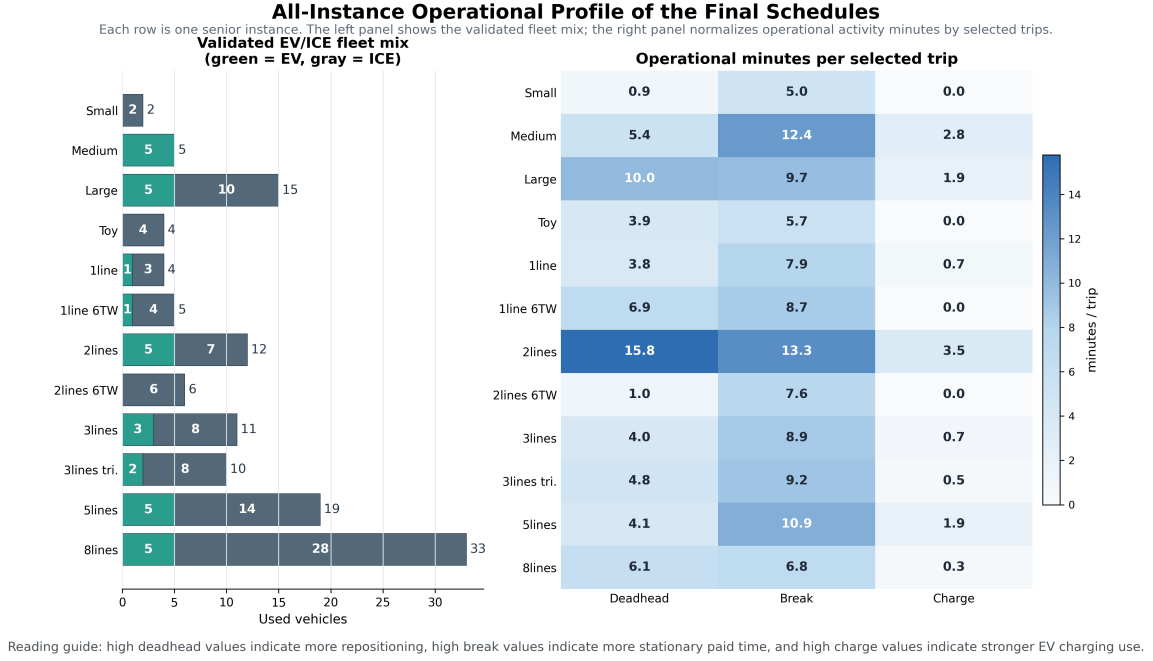


Figure 7.14: Fleet mix and operational activity profile of the final schedules.

Figure 7.14 summarizes the final schedules from two operational viewpoints. The left panel shows how many EV and ICE vehicles are used in each Senior instance, while the right panel normalizes deadhead, break, and charging minutes by the number of selected trips. The figure shows that fleet composition alone does not explain operational pressure. Some instances use a similar number of EVs but differ strongly in deadhead or charging time, which is why the reported objective must combine fleet cost, paid breaks, pull-in and pull-out cost, and the CO₂ term.

$$\Delta^{audit} = \left| C_{internal}^{VS} - C_{external}^{VS} \right|. \quad (7.4)$$

For the final archive, Δ^{audit} is within the rounding tolerance for every reported instance.

Chapter 8

Discussion

The experiments show that graph-based vehicle-block construction is the most important step in the implemented solution approach. The greedy method already finds feasible schedules, but it makes local decisions. The path-cover methods use the compatibility graph more systematically, and the multi-start weighted version gives the best validated results among the implemented methods. During the experiments, this was one of the clearest observations: improving the way trips are chained usually mattered more than changing only the fleet assignment afterwards.

8.1 Interpretation of the Main Findings

The final archive contains feasible solutions for all twelve Senior instances. The headline instances use 2, 5, and 15 vehicles for Small, Medium, and Large, respectively. Over the complete Senior benchmark, the final archive has total validated cost 10000.48 and uses 126 vehicles.

The detailed tables and plots in [Chapter 7](#) report the behaviour of the algorithms instance by instance. For the discussion, the main outcome can also be read in one compact view. [Figure 8.1](#) combines the headline costs, the all-instance archive result, the fleet split, the objective-audit consistency, and the computational setting.

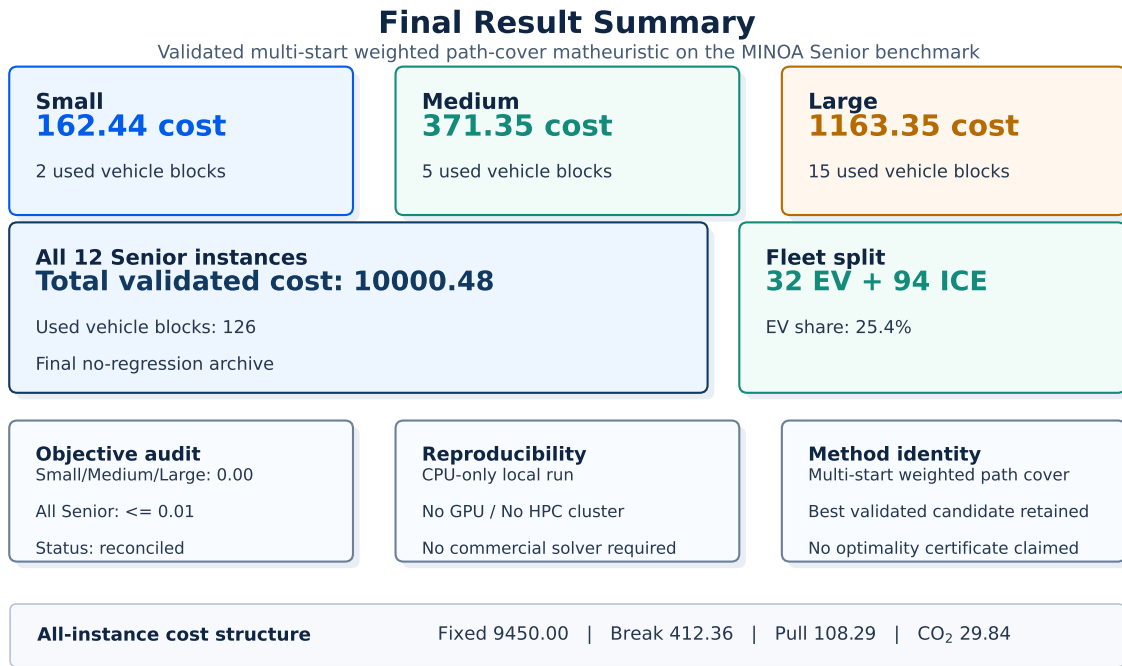


Figure 8.1: Final result summary of the implemented multi-start path-cover matheuristic.

The result summary shows that the final method is strong because several indicators point in the same direction. It gives low validated costs on Small, Medium, and Large, covers all twelve Senior instances, keeps the final archive at 10000.48 cost with 126 used vehicle blocks, and does so in a CPU-only local environment without GPU or HPC support. The EV/ICE split also shows that the method does not force electric operation everywhere. EVs are used where the block structure and charging opportunities make them feasible.

The results also show that electrification is not simply a matter of replacing as many ICE vehicles as possible. Medium can be operated with a 100% EV share in the final solution, while Small is cheaper with two ICE vehicles. This is not a contradiction. In practice, EV use depends on the structure of the blocks, available charging time, terminal capacity, and the fixed and operational cost coefficients of the instance.

Fixed vehicle cost is the largest component in the final objective audit in [Table 7.9](#). This explains why reducing the number of blocks is often more important than small changes in deadhead or charging time. At the same time, charging feasibility cannot be ignored. A low vehicle count is only useful if the resulting blocks can still satisfy autonomy and terminal-capacity requirements. Initially, I expected the EV assignment step to have a larger influence on the final objective. The experiments made this point clearer: in most

cases tested here, the structure of the vehicle blocks had to be strong before EV assignment could improve the solution.

The results therefore support a two-level interpretation. At the first level, the path-cover construction improves the schedule by joining compatible trips and reducing unnecessary block starts. This is visible in the vehicle counts and in the dominance of fixed vehicle cost. At the second level, the EV and charging checks decide which of these blocks can actually be operated by electric buses. This second level does not always reduce the number of vehicles, but it changes the fleet mix and the smaller cost components.

The Medium instance illustrates the favorable case for electrification. The final solution uses five vehicles and all of them are electric. This means that the block structure, break locations, charging times, and terminal capacities fit the EV restrictions well. The Small instance illustrates a different case. The best validated solution uses only two vehicles, both conventional. Adding EV operation would not automatically improve the official objective if it requires more vehicles or additional operational time. The Large instance lies between these two cases and uses a mixed fleet.

The all-instance results show the same pattern more broadly. EV share, vehicle productivity, and cost per trip are related but not identical. A high EV share can be operationally attractive, but it is not the only measure of solution quality. A schedule with fewer vehicles and lower cost can be better under the official objective even if its EV share is lower. This is why the thesis reports EV share together with cost, vehicle count, charging minutes, and objective components. This may not hold in general for every operator or cost setting. At least for this benchmark, however, the result suggests that electrification has to be discussed together with vehicle count and charging feasibility, not as a single percentage.

8.2 Benefits and Drawbacks of the Implemented Methods

[Table 5.2](#) summarizes the methodological role of the four implemented approaches. The greedy method is useful as a transparent baseline and as a quick feasibility constructor. Its drawback is that early successor choices can block better later combinations.

The unweighted path-cover method improves the search by solving a matching problem on the complete compatibility graph. It has a clear graph-theoretic interpretation: more

selected arcs usually means fewer vehicle blocks. However, it does not distinguish between a short clean continuation and a long operationally unattractive continuation.

The weighted path-cover method addresses this weakness by scoring arcs with waiting time, deadhead time, and bounded charging-risk information. The score is still a construction score and not the official objective. Its advantage is that the matching step becomes closer to the final cost calculation. The multi-start version adds diversification by testing several timetable and graph variants. In practice, this turned out to be helpful but not free: it makes the search more stable, but it also increases runtime, especially for Large.

The comparison also shows why a single algorithmic idea is not enough for the whole problem. Greedy construction gives feasibility quickly, but it does not look far enough ahead. Matching improves the global structure of vehicle blocks, but unweighted matching can choose unattractive continuations. Weighted matching improves the arc choice, but it still depends on the selected timetable. Multi-start search addresses this by changing the timetable and graph variants, but it cannot explore all possible combinations.

The final method is therefore a compromise. It uses exact graph optimization where the structure is clear and limited, and it uses heuristic search where the complete integrated problem would be too large. This compromise is appropriate for the MINOA Senior benchmark because the task contains several interacting decisions: trip selection, block construction, fleet assignment, battery propagation, charging insertion, and node-capacity feasibility.

The no-regression archive is another practical design choice. It prevents a new heuristic variant from replacing a better existing output. This matters because a heuristic can improve one instance and worsen another. By retaining the better validated output for each instance, the final archive reports the best solution found by the implemented search process without claiming that one single parameter setting is best for every instance. This was not always the case during testing. Some candidate variants improved one file but produced a worse block structure on another, which is why the archive rule became a useful reporting device rather than only an implementation detail.

8.3 Validation, Bounds, and Optimality

The reported schedules are feasible upper bounds. They are not claimed to be globally optimal. Matching is exact only for a fixed selected timetable, fixed compatibility graph,

and fixed arc weights. The complete problem still contains heuristic decisions in timetable generation, candidate-family construction, EV assignment, and charging insertion.

The lower-bound results in [Table 7.7](#) give useful context. For Small, the global lower bound is close enough to show that the remaining gap is moderate. For Large, the controlled run does not provide a useful certified global bound. The selected-timetable lower bound is therefore only a diagnostic value for the timetable already chosen by the heuristic.

The objective audit in [Equation \(7.4\)](#) is important for reporting reliability. It confirms that the internally computed cost and the external audit value agree within rounding tolerance. This does not prove optimality, but it supports the claim that the reported objective values are computed consistently.

The lower-bound results must be read carefully. A feasible solution gives an upper bound because it is an admissible schedule with a known cost. A lower bound gives a value that no feasible solution can beat for the corresponding relaxation. If both bounds coincide, global optimality is certified. In this thesis, the bounds do not close the complete integrated problem. They are used to interpret solution quality and to show where the remaining uncertainty is larger.

The distinction between LB^{global} and $LB^{TT}(\pi)$ is especially important. The global lower bound is valid before the timetable is fixed. The selected-timetable diagnostic lower bound assumes the timetable produced by the heuristic. It can therefore be much more informative for the vehicle-scheduling subproblem, but it is not a certificate for the full integrated problem. This distinction avoids overstating the quality of the final schedules.

The MINOA desktop validator has a different role from the lower-bound calculation. It checks whether a generated output satisfies the required format and feasibility rules and whether the reported objective value is consistent. It does not certify that the solution is globally optimal. This separation is important for the scientific interpretation of the experiments: validation confirms feasibility and objective agreement, while bounds provide information about possible optimality gaps.

8.4 Limitations and Future Work

The main limitation is the absence of a global optimality proof for the full integrated problem. The method searches over a controlled set of timetable and graph variants, not over every possible integrated schedule. A second limitation is that charging-aware

information enters the matching step through bounded scoring and later feasibility checks rather than through one fully integrated exact charging model.

Another limitation is that the multi-start family is bounded by design. The algorithm explores several timetable starts, graph scores, EV assignment strategies, and repair options, but it does not perform an unrestricted search over all successor swaps or all block recombinations. This makes the method reproducible and practical on the reported hardware, but it also means that a better solution may exist outside the explored candidate family.

The charging model is also intentionally simplified compared with a detailed electric-bus energy model. The implementation checks autonomy, minimum charging duration, slow and fast charging options, and terminal capacity. It does not model nonlinear charging curves, temperature effects, battery degradation, or stochastic energy consumption. These details are outside the MINOA Senior input data used in this thesis, but they would matter in a real-world deployment study.

The results are based on the available Senior instances. They provide useful evidence for the implemented methods, but they are not a statistical sample of all possible bus networks. Different cities may have other terminal layouts, different depot locations, different charger capacities, and different fleet cost coefficients. The graph-based method should transfer conceptually, but the numerical performance would need to be tested on those data sets. When running the larger instances, this dependence on the input structure was easy to see. Small changes in line structure or charging opportunity can change which blocks are attractive for EV operation.

Exact optimization and global optimality certification. The proposed matheuristic provides feasible solutions and therefore valid upper bounds. A future exact method could use these solutions as warm starts or initial upper bounds. Such a method would need to model timetable selection, vehicle connections, vehicle types, battery states, charging activities, and parking and charging capacities jointly. It should also avoid restricting the search only to the timetable variants generated by the current heuristic.

Possible exact directions include mixed-integer programming, branch-and-cut, branch-and-price, or decomposition. In all cases, valid lower bounds would be required in addition to

feasible upper bounds. If LB is a valid lower bound, UB is the cost of the best feasible solution, and C^* is the unknown global optimum, then

$$LB \leq C^* \leq UB. \quad (8.1)$$

Global optimality would be certified only when $LB = UB$. More commonly, an exact solver may stop with a certified remaining optimality gap. This extension could require substantially more computation time for larger instances.

Future work could also combine the current matheuristic with an exact method. The heuristic solution could provide a warm start and a strong initial upper bound. The exact method could then focus on proving whether a better solution exists. This would preserve the practical advantage of the current solver while adding a stronger certification layer.

A second extension would be a richer charging-aware neighborhood search. The current implementation uses bounded local improvements and charging-aware scores. A larger neighborhood could simultaneously move trips between blocks, change charging locations, and reassign EVs. Such a neighborhood may improve cost, but it would also require careful feasibility checks to avoid violating capacity constraints.

A third extension would be multi-objective analysis. The official objective combines fixed vehicle cost, paid break cost, pull-in/out cost, and CO₂ cost. Public transport operators may also care about separate indicators such as EV share, charging-infrastructure utilization, operational stability, or reserve battery level. Reporting these indicators separately would make the trade-offs between cost and sustainability more explicit.

Chapter 9

Conclusion

9.1 Summary of the Thesis

This thesis studied integrated timetabling and vehicle scheduling for mixed electric and conventional bus fleets in the MINOA Senior benchmark. The work started from a practical planning question. A timetable may look feasible from the passenger-service side, but it still has to be operated by vehicles that start at the depot, move between terminals, respect stopping times, and, in the case of electric buses, keep enough autonomy and find usable charging time. During implementation, this connection between timetable choice and later vehicle feasibility became the central difficulty.

The proposed solution is a graph-based matheuristic. It selects feasible timetable variants, builds compatibility graphs, solves path-cover subproblems by matching, assigns vehicle types, and checks charging and capacity feasibility. The same workflow is used for the three headline instances and for the wider set of Senior instances. This keeps the computational study reproducible and makes it possible to compare the tested algorithmic variants under the same reporting format.

9.2 Main Methodological Contribution

The final method is the multi-start path-cover matheuristic. It was selected after testing four variants: greedy construction, unweighted path cover, weighted path cover, and multi-start path cover. This development was useful because each variant exposed a different weakness. Greedy construction was fast and transparent, but it depended strongly on local successor choices. Unweighted path cover improved the global vehicle-block structure for a fixed selected timetable. Weighted path cover added operational information to the edges. The multi-start version then repeated the graph construction over several timetable and scoring variants.

The main methodological contribution is therefore not one isolated trick. It is the combination of timetable-variant generation, compatibility-graph modelling, charging-aware edge scoring, EV charging look-ahead, bounded candidate-family neighborhoods, matching-based vehicle blocking, and no-regression archiving. At least for this benchmark,

this combination worked better than relying only on a local greedy rule or on one fixed compatibility graph. It also kept the method understandable: each selected edge still means that one trip can follow another trip in the same vehicle block.

9.3 Synthesis of the Main Findings

The best validated headline solutions use 2, 5, and 15 vehicles for Small, Medium, and Large, with costs 162.44, 371.35, and 1163.35. Over all twelve Senior instances, the final archive validates all outputs and reaches total validated cost 10000.48 with 126 vehicles. These values are the main numerical outcome of the thesis. [Figure 8.1](#) summarizes this outcome across headline performance, all-instance coverage, fleet mix, objective-audit consistency, and computational resources.

The results suggest that vehicle-block construction is the most important driver of solution quality in the tested instances. This was not immediately clear at the beginning. One might expect the EV assignment step to dominate, because electric buses introduce the most visible additional restrictions. In most cases tested here, however, good chaining came first. If the blocks were poorly shaped, later EV assignment and charging checks had little room to repair the schedule.

The EV results also need a careful interpretation. Medium can be served fully electrically in the final solution. Small is cheapest with two ICE vehicles. Large uses a mixed fleet. This may seem uneven, but it follows the structure of the problem. EV use depends on the chronological block, the available charging breaks, terminal capacity, and the cost coefficients. A higher EV share is therefore not automatically the same as a lower objective value.

Each reported schedule is a feasible upper bound for its instance. The thesis also reports a globally valid timetable-overlap lower bound and a stronger selected-timetable diagnostic bound. The selected-timetable bound is helpful for understanding the vehicle-scheduling part after a timetable has been chosen, but it is not a certificate for the full integrated problem. The thesis does not claim exact global optimality.

9.4 Scientific and Practical Implications

From a methodological perspective, the thesis shows that the path-cover view is a useful way to organize the integrated problem. It connects the mathematical model to an

operational picture: trips become nodes, feasible continuations become edges, and paths become vehicle blocks. This makes the algorithm easier to inspect than a black-box construction. When a block changes, the reason can usually be traced back to a timetable choice, an edge score, or a feasibility check.

From the implementation side, the repository provides a runnable experiment pipeline for the headline instances and the wider Senior benchmark. This matters because the results are not only written in tables; they can also be checked from the generated output files, the cost audit, and the experiment commands. The reported runs were produced in a local CPU-based environment. This does not prove that the method is generally faster than other approaches, but it is useful from a reproducibility perspective.

For public-transport planning, the main practical implication is that timetable selection should not be separated too early from vehicle scheduling when EVs are present. Charging opportunities are created by the same timetable and block structure that determine the number of vehicles. In practice, a small change in selected trips can change both the compatibility graph and the charging feasibility of later blocks.

9.5 Final Remarks

The work remains limited in an important way. The implemented search is bounded and heuristic. It does not include fully integrated charging-aware matching, unrestricted block merge/split local search, or large-neighborhood repair over all possible schedules. This may not hold in general for other networks, other cost settings, or larger real-world data sets. A future exact model could check this more directly by combining the current feasible solutions with valid lower bounds.

Still, the experiments give a clear direction. Multi-start graph-based matheuristics can produce feasible and competitive schedules for the MINOA Senior benchmark while keeping the method explainable. One direction that would be interesting after this thesis is to connect the current solver with an exact certification method. The schedules found here could then be used as warm starts or upper bounds, while an exact method investigates whether better solutions exist. That would be a natural next step after the work completed in this thesis.

References

- [1] J.-Q. Li. “Transit Bus Scheduling with Limited Energy”. *Transportation Science* 48.4 (2014), pp. 521–539. DOI: [10.1287/trsc.2013.0468](https://doi.org/10.1287/trsc.2013.0468).
- [2] R. van Kooten Niekerk, J. M. van den Akker, and J. A. Hoogeveen. “Scheduling electric vehicles”. *Public Transport* 9 (2017), pp. 155–176. DOI: [10.1007/s12469-017-0164-0](https://doi.org/10.1007/s12469-017-0164-0).
- [3] M. Rogge, E. van der Hurk, A. Larsen, and D. U. Sauer. “Electric bus fleet size and mix problem with optimization of charging infrastructure”. *Applied Energy* 211 (2018), pp. 282–295. DOI: [10.1016/j.apenergy.2017.11.051](https://doi.org/10.1016/j.apenergy.2017.11.051).
- [4] A. Ceder. *Public Transit Planning and Operation: Modeling, Practice and Behavior*. CRC Press, 2016. DOI: [10.1201/b18689](https://doi.org/10.1201/b18689).
- [5] G. Desaulniers and M. D. Hickman. “Public Transit”. In: *Handbooks in Operations Research and Management Science*. Vol. 14. Elsevier, 2007, pp. 69–127. DOI: [10.1016/S0927-0507\(06\)14002-5](https://doi.org/10.1016/S0927-0507(06)14002-5).
- [6] S. Bunte and N. Kliwer. “An overview on vehicle scheduling models in public transport”. *Public Transport* 1 (2009), pp. 299–317. DOI: [10.1007/s12469-010-0018-5](https://doi.org/10.1007/s12469-010-0018-5).
- [7] M. Michaelis and A. Schobel. “Integrating Line Planning, Timetabling, and Vehicle Scheduling”. *Public Transport* 1.3 (2009), pp. 211–232. DOI: [10.1007/s12469-009-0014-9](https://doi.org/10.1007/s12469-009-0014-9).
- [8] V. Schmid and J. F. Ehmke. “Integrated Timetabling and Vehicle Scheduling with Balanced Departure Times”. *OR Spectrum* 37.4 (2015), pp. 903–928. DOI: [10.1007/s00291-015-0398-7](https://doi.org/10.1007/s00291-015-0398-7).
- [9] A. L. Dulmage and N. S. Mendelsohn. “Coverings of Bipartite Graphs”. *Canadian Journal of Mathematics* 10 (1958), pp. 517–534. DOI: [10.4153/CJM-1958-052-0](https://doi.org/10.4153/CJM-1958-052-0).
- [10] J. Edmonds. “Paths, Trees, and Flowers”. *Canadian Journal of Mathematics* 17 (1965), pp. 449–467. DOI: [10.4153/CJM-1965-045-4](https://doi.org/10.4153/CJM-1965-045-4).
- [11] Z. Galil. “Efficient Algorithms for Finding Maximum Matching in Graphs”. *ACM Computing Surveys* 18.1 (1986), pp. 23–38. DOI: [10.1145/6462.6502](https://doi.org/10.1145/6462.6502).

-
- [12] MINOA ITN. *MINOA Research Challenge: Problem Description – Senior*. PDF, Version 1.2, May 27, 2021. Accessed: 2026-07-21. [Online]. Available: https://minoa-itn.fau.de/wp-content/uploads/2021/05/MINOA_Challenge_Problem_Description_Senior.pdf. 2021.
- [13] MINOA ITN. *MINOA Research Challenge: Input/Output Description Format*. PDF, Version 1.2, Apr. 26, 2021. Accessed: 2026-07-21. [Online]. Available: https://minoa-itn.fau.de/wp-content/uploads/2021/04/MINOA_Challenge_Input_Output_format_description.pdf. 2021.
- [14] A. Schobel. *Optimization in Public Transportation*. Springer, 2006. DOI: [10.1007/978-0-387-36643-2](https://doi.org/10.1007/978-0-387-36643-2).
- [15] A. Schobel. “An Eigenmodel for Iterative Line Planning, Timetabling and Vehicle Scheduling in Public Transportation”. *Transportation Research Part C: Emerging Technologies* 74 (2017), pp. 348–365. DOI: [10.1016/j.trc.2016.11.018](https://doi.org/10.1016/j.trc.2016.11.018).
- [16] N. Kliewer, T. Mellouli, and L. Suhl. “A time-space network based exact optimization model for multi-depot bus scheduling”. *European Journal of Operational Research* 175.3 (2006), pp. 1616–1627. DOI: [10.1016/j.ejor.2005.02.030](https://doi.org/10.1016/j.ejor.2005.02.030).
- [17] G. Desaulniers, J. Desrosiers, and M. M. Solomon. *Column Generation*. Springer, 2005. DOI: [10.1007/b135457](https://doi.org/10.1007/b135457).
- [18] L. R. Ford and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962. URL: <https://dl.acm.org/doi/10.5555/1942094>.
- [19] E. L. Lawler. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart and Winston, 1976. URL: https://books.google.com/books/about/Combinatorial_Optimization.html?id=m4MvtFenVjEC.
- [20] M. Schneider, A. Stenger, and D. Goeke. “The electric vehicle-routing problem with time windows and recharging stations”. *Transportation Science* 48.4 (2014), pp. 500–520. DOI: [10.1287/trsc.2013.0490](https://doi.org/10.1287/trsc.2013.0490).
- [21] D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997. URL: <http://www.athenasc.com/linoptbook.html>.
- [22] L. A. Wolsey. *Integer Programming*. Wiley, 1998. URL: https://books.google.com/books/about/Integer_Programming.html?id=x7RvQgAACAAJ.

- [23] F. Glover and G. A. Kochenberger, eds. *Handbook of Metaheuristics*. Kluwer Academic Publishers, 2003. DOI: [10.1007/b101874](https://doi.org/10.1007/b101874).
- [24] V. Maniezzo, T. Stutzle, and S. Voss, eds. *Matheuristics: Hybridizing Metaheuristics and Mathematical Programming*. Springer, 2009. DOI: [10.1007/978-1-4419-1306-7](https://doi.org/10.1007/978-1-4419-1306-7).
- [25] M. Keskin and B. Catay. “Partial recharge strategies for the electric vehicle routing problem with time windows”. *Transportation Research Part C* 65 (2016), pp. 111–127. DOI: [10.1016/j.trc.2016.01.013](https://doi.org/10.1016/j.trc.2016.01.013).
- [26] J. Reuer, N. Kliewer, and L. Wolbeck. “The Electric Vehicle Scheduling Problem: Time-space network based and heuristic solution approaches”. In: *Proceedings of the 13th Conference on Advanced Systems in Public Transport*. 2015. URL: <http://www.rotterdam2015.caspt.org/proceedings/paper93.pdf>.
- [27] N. Olsen and N. Kliewer. “Scheduling Electric Buses in Public Transport: Modeling of the Charging Process and Analysis of Assumptions”. *Logistics Research* 13, Article 4 (2020). DOI: [10.23773/2020_4](https://doi.org/10.23773/2020_4).
- [28] X. Tang, X. Lin, and F. He. “Robust Scheduling Strategies of Electric Buses under Stochastic Traffic Conditions”. *Transportation Research Part C: Emerging Technologies* 105 (2019), pp. 163–182. DOI: [10.1016/j.trc.2019.05.032](https://doi.org/10.1016/j.trc.2019.05.032).
- [29] MINOA ITN. *MINOA Research Challenge: Rules*. PDF, Version 1.2, Sep. 8, 2021. Accessed: 2026-07-21. [Online]. Available: https://minoa-itn.fau.de/wp-content/uploads/2021/09/MINOA_Challenge_Rules.pdf. 2021.

Appendix A

Running the Code

This appendix gives the practical steps needed to reproduce the reported experiments. It is a compact execution guide for setting up the repository, running the solver, auditing output files, and regenerating the reported tables and figures.

A.1 System and Software Requirements

The reported experiments were run in a CPU-only local environment on macOS with an Apple M3 processor, 8 CPU cores, and 16 GB memory. No GPU, HPC cluster, or commercial optimization solver was required for the reported runs. The solver requires Python 3.11 or newer and the packages listed in `requirements.txt`. Java is required only for the external MINOA feasibility and objective audit.

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 python -m pip install --upgrade pip
4 python -m pip install -r requirements.txt
5 java -version
```

A.2 Repository Structure

The repository is organized as a code-first project. The most important tracked folders and files are:

1 project-root/	
2 .github/workflows/	CI checks for headline and all-instance runs
3 data/raw/minoa/senior/	Senior benchmark input files
4 scripts/	command-line runners and figure scripts
5 scripts/minoa_lib/	implementation modules
6 tests/	rule-focused tests
7 tools/minoa/desktopValidator/...jar	external MINOA checker
8 results/final_validated_results.json	canonical thesis result table
9 requirements.txt	Python dependencies

Generated working files are written outside the tracked source structure, mainly under `outputs/minoa/` and `data/processed/minoa/`. The raw Senior input files are not modified by the solver.

The public repository and the continuous-integration runs are available through the [project repository](#), the [GitHub Actions overview](#), the [headline workflow](#), the [all-instance workflow](#), and the [result consistency workflow](#).

A.3 Main Experiment Commands

The main entry point is `scripts/run_experiment.py`. The final thesis method is selected with `-algorithm multistart`. The following commands cover the main reproduction workflow:

```

1 # Headline instances: Small, Medium, Large
2 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope sml
3
4 # One headline instance
5 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope sml --only
  Small
6
7 # Final all-instance no-regression archive
8 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope all
9
10 # Optional fresh direct all-instance audit before printing the final archive
11 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope all --fresh-
  audit

```

The all-instance archive command prints the canonical final thesis table with total validated cost 10000.48 and 126 used vehicle blocks. The `-fresh-audit` option reruns the construction pipeline first and then prints the retained no-regression archive.

The comparison algorithms use the same command style:

```

1 .venv/bin/python scripts/run_experiment.py --algorithm greedy --scope sml
2 .venv/bin/python scripts/run_experiment.py --algorithm pathcover --scope sml
3 .venv/bin/python scripts/run_experiment.py --algorithm weighted --scope sml
4 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope sml

```

A.4 Output Files and External Audit

Headline outputs are written to `outputs/minoa/sml_<algorithm>/.` The final retained archive is stored under `outputs/minoa/final_archive/`. The compact machine-readable result file is `results/final_validated_results.json`.

A generated input-output pair can be audited manually with the external Java checker:

```

1 java -jar tools/minoa/desktopValidator/desktopValidator/desktopValidator.jar \
2   data/raw/minoa/senior/Small_Input_S.json \
3   outputs/minoa/sml_multistart/Small_Output_multistart.json

```

The audit checks feasibility and reports the objective value for the complete input-output pair. It is not an optimality certificate. Existing output files can be summarized in a table with:

```

1 .venv/bin/python scripts/minoa_report.py INPUT_JSON:OUTPUT_JSON

```

A.5 Tables, Figures, and Lower Bounds

The final compact thesis result table is generated from `results/final_validated_results.json`:

```

1 .venv/bin/python scripts/print_final_results_table.py

```

The numerical result graphs are regenerated with:

```

1 .venv/bin/python scripts/minoa_make_constraint_figures.py

```

Lower-bound reports are separate from feasible schedule construction:

```

1 .venv/bin/python scripts/run_lower_bounds.py \
2   --scope all \
3   --input-dir data/processed/minoa/final_adaptive_bounded \
4   --archive-csv outputs/minoa/final_archive/final_results.csv \
5   --time-limit 180 \
6   --output-csv results/lower_bounds/all_instances_lower_bounds.csv
7
8 .venv/bin/python scripts/generate_lower_bound_figures.py \
9   --csv results/lower_bounds/all_instances_lower_bounds.csv \
10  --out-dir outputs/minoa/thesis_figures

```

Only certified global lower bounds are used as global bounds. Selected-timetable bounds are diagnostic values for a fixed timetable and are not global optimality certificates.

A.6 Quick-Start Recipe

The following command sequence is sufficient for a quick reproduction run:

```

1 python3 -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -r requirements.txt
4
5 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope sml
6 .venv/bin/python scripts/run_experiment.py --algorithm multistart --scope all
7 .venv/bin/python scripts/print_final_results_table.py

```

For a manual audit of a specific output file, use the Java checker command from the previous section.

A.7 Troubleshooting

Table A.1: Common execution issues and practical fixes.

Issue	Practical fix
Python package missing	Activate <code>.venv</code> and run <code>python -m pip install -r requirements.txt</code> .
Java not found	Run <code>java -version</code> . If it fails, install a Java Runtime Environment or JDK.
Input file not found	Run commands from the project root and check <code>data/raw/minoa/senior/</code> .
Output folder missing	Use the experiment wrappers; they create output folders automatically.
Checker path changed	Pass the checker location with <code>-validator</code> .
Large run is slow	Use <code>-scope sml</code> for a quick check. Use <code>-quick</code> only for diagnostics, not for final reported results.

Appendix B

Complete Algorithm Comparison on All Senior Instances

This appendix gives the complete instance-level comparison of the four implemented algorithms. [Table B.1](#) reports the same experiment family as [Table 7.1](#), but for every Senior instance. The blue instance names mark the headline instances Small, Medium, and Large. The green cells mark the best cost and zero gap for each instance. The total rows at the end of each algorithm block summarize the all-instance cost and vehicle count.

One archived 5lines Senior benchmark file required a small input-format repair before it could be used in the automated experiment workflow. The downloaded JSON copy was structurally incomplete because the final top-level closing brace was missing. I created a repaired working copy only by closing the JSON object. No trips, times, costs, vehicle data, capacities, or other optimization-relevant values were changed.

Table B.1: Complete all-instance comparison of the four implemented algorithms.

Instance	Algorithm	Cost	Gap	Veh.	EV	ICE	EV share	Trips	Deadhead	Break	Charge
Small	Greedy	213.55	31.46	3	3	0	100.00%	48	402.00	418.00	201.00
Medium	Greedy	457.10	23.09	6	5	1	83.33%	139	826.00	2503.00	409.00
Large	Greedy	1169.87	0.56	15	5	10	33.33%	260	3215.00	2346.00	442.00
Toy Example	Greedy	488.04	67.72	6	5	1	83.33%	68	463.00	913.00	124.00
1line	Greedy	337.19	7.52	4	1	3	25.00%	102	426.00	1389.00	85.00
1line 6timeWindow	Greedy	537.93	38.76	7	2	5	28.57%	112	1432.00	1315.00	0.00
2lines	Greedy	910.21	0.01	12	5	7	41.67%	204	3218.00	2710.00	701.00
2lines 6 timeWindows	Greedy	627.38	26.59	8	3	5	37.50%	204	1150.00	2400.00	189.00
3lines	Greedy	1013.43	11.18	12	3	9	25.00%	306	1258.00	3654.00	196.00
3linesTriangle	Greedy	949.54	17.36	12	4	8	33.33%	306	1734.00	2967.00	195.00
5lines	Greedy	1672.28	6.72	20	5	15	25.00%	510	2110.00	6485.00	877.00
8lines	Greedy	2616.68	0.00	33	5	28	15.15%	819	5036.00	5552.00	288.00
Total	Greedy	10993.20	–	138	46	92	–	3078	–	–	–
Small	Path-cover	213.37	31.35	3	3	0	100.00%	48	436.00	384.00	196.00
Medium	Path-cover	456.27	22.87	6	5	1	83.33%	139	986.00	2343.00	389.00
Large	Path-cover	1168.63	0.45	15	5	10	33.33%	260	3195.00	2366.00	497.00
Toy Example	Path-cover	479.08	64.64	6	4	2	66.67%	68	463.00	913.00	107.00
1line	Path-cover	335.40	6.95	4	1	3	25.00%	102	606.00	1209.00	174.00
1line 6timeWindow	Path-cover	538.00	38.78	7	2	5	28.57%	112	1544.00	1582.00	62.00
2lines	Path-cover	925.68	1.71	12	3	9	25.00%	204	3454.00	2905.00	634.00
2lines 6 timeWindows	Path-cover	614.27	23.94	8	4	4	50.00%	204	1325.00	2276.00	780.00
3lines	Path-cover	1009.29	10.73	12	3	9	25.00%	306	1676.00	3685.00	553.00
3linesTriangle	Path-cover	962.70	18.98	12	2	10	16.67%	306	2174.00	3865.00	246.00
5lines	Path-cover	1669.81	6.56	20	5	15	25.00%	510	2470.00	6125.00	940.00
8lines	Path-cover	2655.44	1.48	33	2	31	6.06%	819	6035.00	7224.00	321.00
Total	Path-cover	11027.94	–	138	39	99	–	3078	–	–	–
Small	Weighted path-cover	213.55	31.46	3	3	0	100.00%	48	402.00	418.00	201.00
Medium	Weighted path-cover	458.36	23.43	6	5	1	83.33%	139	847.00	2482.00	357.00
Large	Weighted path-cover	1166.94	0.31	15	5	10	33.33%	260	3215.00	2346.00	510.00
Toy Example	Weighted path-cover	488.44	67.85	6	5	1	83.33%	68	463.00	913.00	126.00
1line	Weighted path-cover	323.63	3.20	4	2	2	50.00%	102	426.00	1389.00	245.00
1line 6timeWindow	Weighted path-cover	530.77	36.91	7	3	4	42.86%	112	1432.00	1315.00	349.00
2lines	Weighted path-cover	910.15	0.00	12	5	7	41.67%	204	3218.00	2710.00	717.00
2lines 6 timeWindows	Weighted path-cover	626.24	26.36	8	3	5	37.50%	204	1150.00	2400.00	215.00
3lines	Weighted path-cover	999.99	9.71	12	4	8	33.33%	306	1258.00	3654.00	351.00
3linesTriangle	Weighted path-cover	941.41	16.35	12	5	7	41.67%	306	1734.00	2967.00	651.00
5lines	Weighted path-cover	1669.01	6.51	20	5	15	25.00%	510	2110.00	6485.00	956.00
8lines	Weighted path-cover	2617.76	0.04	33	5	28	15.15%	819	5036.00	5552.00	257.00
Total	Weighted path-cover	10946.25	–	138	50	88	–	3078	–	–	–
Small	Multi-start path-cover	162.44	0.00	2	0	2	0.00%	48	42.00	240.00	0.00
Medium	Multi-start path-cover	371.35	0.00	5	5	0	100.00%	139	752.00	1717.00	391.00
Large	Multi-start path-cover	1163.35	0.00	15	5	10	33.33%	260	2600.00	2520.00	505.00
Toy Example	Multi-start path-cover	290.99	0.00	4	0	4	0.00%	68	264.00	385.00	0.00
1line	Multi-start path-cover	313.61	0.00	4	1	3	25.00%	102	386.00	807.00	67.00
1line 6timeWindow	Multi-start path-cover	387.67	0.00	5	1	4	20.00%	112	775.00	970.00	0.00
2lines	Multi-start path-cover	910.15	0.00	12	5	7	41.67%	204	3218.00	2710.00	717.00
2lines 6 timeWindows	Multi-start path-cover	495.60	0.00	6	0	6	0.00%	204	213.00	1551.00	0.00
3lines	Multi-start path-cover	911.50	0.00	11	3	8	27.27%	306	1218.00	2725.00	218.00
3linesTriangle	Multi-start path-cover	809.10	0.00	10	2	8	20.00%	306	1474.00	2811.00	146.00
5lines	Multi-start path-cover	1566.95	0.00	19	5	14	26.32%	510	2070.00	5556.00	965.00
8lines	Multi-start path-cover	2617.76	0.04	33	5	28	15.15%	819	5036.00	5552.00	257.00
Total	Multi-start path-cover	10000.48	–	126	32	94	–	3078	–	–	–

Appendix C

Selected Instance Profiles

The result chapter discusses the benchmark mainly through aggregate tables, headline instances, and cross-instance figures. This appendix adds a short profile of the instance-level patterns that are most relevant for interpreting those results.

For an instance with candidate trip set T and selected timetable π , the selected-trip share is

$$p_T = \frac{|\pi|}{|T|} \cdot 100\%.$$

The benchmark contains dense candidate pools, so the selected-trip share is small even when the timetable is feasible. Across the final all-instance archive, 3078 trips are selected from 60840 candidates, which gives

$$p_T = \frac{3078}{60840} \cdot 100\% \approx 5.06\%.$$

This value reflects the design of the input data: many departure alternatives are provided, but only a sparse subset is needed to satisfy the headway and boundary rules.

The graph identity

$$|B| = |\pi| - |M_G|$$

is the main link between the compatibility graph and the number of vehicles. Here M_G is the set of matched compatibility edges. The Small instance selects 48 trips and 46 continuations, resulting in 2 vehicle blocks. Medium selects 139 trips and 134 continuations, resulting in 5 blocks. Large selects 260 trips and 245 continuations, resulting in 15 blocks. The same identity also explains the additional benchmark instances: vehicle reductions occur when the selected timetable and matching create more feasible continuations.

The EV share differs strongly across instances. Medium is fully electrified in the final headline result, while Small uses two ICE blocks and no EV block. This difference does not mean that one instance is treated with a different objective. It means that the block lengths, charging opportunities, vehicle counts, and cost coefficients lead to different cheapest feasible fleet assignments. The main results therefore report cost, vehicles, EV share, deadhead time, break time, and charging time together.

Appendix D

Core Pseudocode

This appendix records the main algorithmic steps in compact form. The detailed motivation and mathematical formulation are given in Chapters 4 and 5.

D.1 Timetable Selection

Table D.1: Pseudocode for timetable selection in one direction.

Step	Operation
1	Sort the candidate trips $T^\ell = (i_1, \dots, i_q)$ of direction ℓ by main-stop time.
2	Build the directed headway graph $H^\ell = (T^\ell, E^\ell)$.
3	Add edge (i, j) if $a(i) < a(j)$ and the maximum headway rule is satisfied.
4	Mark admissible initial and final trips according to the boundary rules.
5	Compute feasible initial-to-final paths with minimum selected-trip count.
6	If several paths have the same count, apply the start-specific score $Q_h(P)$.
7	Return the selected path P_h^ℓ as the timetable for direction ℓ .

D.2 Compatibility Graph and Path Cover

Table D.2: Pseudocode for compatibility-graph construction and path cover.

Step	Operation
1	Let the selected timetable be π , sorted by start time.
2	Create one graph node for every selected trip.
3	For every ordered pair (i, j) , test in-line compatibility.
4	If in-line compatibility fails, test depot-bridge compatibility.
5	Store each feasible continuation as an arc $(i, j) \in A_\pi$.
6	Build the bipartite matching graph with left and right trip copies.
7	Solve maximum-cardinality or maximum-weight matching, depending on the algorithm.
8	Convert matched edges into predecessor and successor maps.
9	Start a block at every trip with no predecessor and follow successors until the path ends.
10	Treat trips without predecessor and successor as single-trip blocks.

D.3 Final Multi-Start Matheuristic

Table D.3: Pseudocode for the final multi-start path-cover matheuristic.

Step	Operation
1	Define timetable starts H and bounded construction settings $\mathcal{N}$.
2	For each $h \in H$ and $n \in \mathcal{N}$, construct a feasible timetable π^h .
3	Build $G_\pi^h = (\pi^h, A_\pi^h)$.
4	Solve the weighted path-cover matching subproblem.
5	Reconstruct vehicle blocks from the matching.
6	Simulate EV feasibility and insert charging where possible.
7	Compute the internal complete-schedule cost C^{VS} .
8	Keep the best internally feasible complete candidate.
9	Write the final output and run the external checker as an independent audit.

D.4 EV Feasibility and Charging Assignment

Table D.4: Pseudocode for EV feasibility and charging assignment.

Step	Operation
1	Sort constructed blocks by EV attractiveness and autonomy risk.
2	Initialize residual autonomy at the EV maximum.
3	Simulate all activities of the block in chronological order.
4	Subtract passenger-trip and deadhead distance from residual autonomy.
5	Test charging insertion during eligible breaks at charge-capable nodes.
6	Reserve charger capacity only if the full charging interval is available.
7	Assign the block to EV if autonomy, charging duration, and capacity remain feasible.
8	Otherwise keep the block as ICE.

Appendix E

Supporting Mathematical Derivations

This appendix collects short derivations for the graph and feasibility relations used by the implemented methods.

E.1 Why Matching Gives a Minimum Path Cover

For a fixed selected timetable π and compatibility graph $G_\pi = (\pi, A_\pi)$, a vehicle block is a directed path. A path cover is a set of vertex-disjoint paths covering every selected trip exactly once. The matching transformation creates two copies of every selected trip:

$$\pi^L = \{i^L : i \in \pi\}, \quad \pi^R = \{i^R : i \in \pi\}.$$

For every compatibility arc $(i, j) \in A_\pi$, the bipartite graph contains edge (i^L, j^R) . A matching selects edges without using a left or right copy twice:

$$\sum_j z_{ij} \leq 1, \quad \sum_i z_{ij} \leq 1.$$

The first inequality gives at most one successor per trip. The second gives at most one predecessor. Hence the selected matching edges form valid predecessor-successor links.

If no edge is selected, every selected trip is a separate vehicle block. Each matched edge joins two paths and reduces the number of blocks by one:

$$|B| = |\pi| - |M_G|.$$

For a fixed timetable and compatibility graph, minimizing the number of blocks is therefore equivalent to maximizing the matching cardinality:

$$\min |B| \iff \max |M_G|.$$

E.2 Weighted Matching as a Vehicle-First Score

The weighted path-cover variant keeps the same matching constraints but uses an edge weight

$$w_{ij} = M^{veh} - \hat{c}_{ij}.$$

The constant M^{veh} is chosen large enough that selecting one additional continuation dominates local edge-cost differences. If two matchings have the same cardinality, maximizing

$$\sum_{(i,j) \in A_\pi} w_{ij} z_{ij}$$

is equivalent to minimizing

$$\sum_{(i,j) \in A_\pi} \hat{c}_{ij} z_{ij}.$$

The score is a construction heuristic, not the reported MINOA objective. The reported cost is computed later from the complete schedule as C^{VS} .

E.3 Battery and Charging Propagation

Let an EV block contain ordered activities

$$A_b = (a_1, a_2, \dots, a_m).$$

The residual autonomy starts at the maximum value:

$$r_0 = A_v^{max}.$$

For each activity, autonomy is updated by

$$r_k = \min\{A_v^{max}, r_{k-1} - d_{a_k}^{drive} + \rho_{a_k} q_{a_k}\}.$$

Driving activities have $d_{a_k}^{drive} > 0$. Non-driving activities have $d_{a_k}^{drive} = 0$. If no charging is used, $q_{a_k} = 0$. The block is EV-feasible only if

$$r_k \geq 0 \quad k = 1, \dots, m.$$

This sequential test is stricter than checking only the total distance, because the timing of charging opportunities matters.

E.4 Capacity Checking by Event Times

Parking and charging capacities are interval resources. If a vehicle occupies resource r at node n during interval $[s_a, e_a)$, it contributes one unit to the active set during that interval. For parking, the condition is

$$|\mathcal{A}_n^{park}(t)| \leq K_n^{park} \quad \forall n, t.$$

For slow and fast chargers, the corresponding conditions are

$$|\mathcal{A}_n^{slow}(t)| \leq K_n^{slow}, \quad |\mathcal{A}_n^{fast}(t)| \leq K_n^{fast}.$$

The active set changes only when an interval starts or ends. Therefore, it is sufficient to check event times

$$H_n = \{s_a, e_a : a \text{ is an activity at node } n\}.$$

This is the reason why the implementation can check capacity with a compact event ledger.

E.5 Cost Decomposition for One Block

For one block b , assigned vehicle type $v(b)$, the cost contribution is

$$C_b = c_{v(b)}^{fix} + c^{break} t_b^{break} + c_{v(b)}^{pio} (t_b^{pi} + t_b^{po}) + c_{v(b)}^{CO_2} d(b).$$

The total cost is

$$C^{VS} = \sum_{b \in B} C_b.$$

For an EV block, $c_{v(b)}^{CO_2} = 0$. For an ICE block, the CO_2 coefficient is positive when specified by the input data. With the same block structure and break time, the difference between EV and ICE assignment is

$$C_b^{EV} - C_b^{ICE} = c_{EV}^{fix} - c_{ICE}^{fix} + (c_{EV}^{pio} - c_{ICE}^{pio})(t_b^{pi} + t_b^{po}) - c_{ICE}^{CO_2} d(b).$$

This expression explains why a higher EV share is not automatically cheaper: fixed vehicle cost, pull cost, and avoided CO_2 cost are balanced by the objective coefficients.

E.6 Global and Selected-Timetable Lower Bounds

The globally valid lower bound is computed before a final timetable is fixed. It relaxes vehicle routing, charging, stopping-time, and capacity decisions. Let $K_{LB}^{global}(I)$ be the certified lower bound on simultaneously needed vehicles for instance I . Multiplication with the cheapest fixed vehicle cost gives

$$LB_I^{global} = K_{LB}^{global}(I) \min_{v \in V} c_v^{fix}.$$

If the lower-bound run reaches a time limit, only a certified solver dual bound is used.

For a fixed selected timetable π , the diagnostic path-cover bound is

$$K_{LB}^{TT,path}(\pi) = |\pi| - |M^*(\pi)|,$$

where $M^*(\pi)$ is a maximum matching in the relaxed compatibility graph. Together with the overlap bound,

$$K_{LB}^{TT}(\pi) = \max\{K_{LB}^{TT,overlap}(\pi), K_{LB}^{TT,path}(\pi)\}.$$

The selected-timetable bound supports interpretation of one reported candidate, but it is not a global lower bound for the full integrated problem.